中国研究生创新实践系列大赛

# “华为杯”第二十三届中国研究生数学建模竞赛

| | |
|---|---|
| 学　　校 | 西安邮电大学 |
| 参赛队号 | A26116640198 |
| 队员姓名 | 1.贾朝晖 |
| | 2.何浩源 |
| | 3.李轩宇 |

中国研究生创新实践系列大赛

# “华为杯”第二十三届中国研究生

# 数学建模竞赛

## 题　目：通信与片上存储协同约束下通用神经网络处理器的计算图划分与多核调度优化


### 摘　要：

多核神经网络处理器的调度需要协调计算并行、数据搬运与片上存储，切图粒度与执行顺序共同影响整体性能。本文面向子图分别构成任务、同核数据驻留和共享二级缓存三种执行场景，建立以完工时间最小为首要目标、额外搬运量最小为次要目标的联合优化模型。提出张量共享感知与执行反馈驱动的分层启发式算法，通过代理排序、安全下界剪枝和精确事件评测，联合优化**子图划分、核心指派及各核子图顺序**。

针对问题一，研究场景 A 中跨子图数据均经 DDR 中转条件下的切图与多核调度问题。采用**张量共享感知的多粒度划分方法**，结合**关键路径列表调度**确定核心分配与任务顺序，按消费子图统计共享张量读入，避免重复估计通信代价；利用依赖路径、流水线负载和**带宽下界剪枝**，并对实测耗时最长的任务进行分层拆分，挖掘内部独立分支的并行性。在 100 个测试用例上，2 核和 5 核相对整图单核基准的平均加速比分别为 1.9344 和 3.9607。

针对问题二，研究场景 B 中同核数据驻留条件下，缓存复用、容量压力与跨核同步相互制约的调度问题。**引入驻留亲和分核与内存释放优先排序**，设计瓶颈重切、同核重排和跨核迁移邻域；依据实际流水线占用、核心完成时刻及依赖路径优先级调整候选方案，通过精确评测权衡数据复用、缓存换出换入和同步等待的综合影响。在同一测试集和相同基准下，2 核和 5 核平均加速比分别达到 2.1176 和 4.3491。

针对问题三，研究共享只读 FIFO L2 Cache 条件下，数据复用时机与带宽竞争对调度性能的影响。针对并发首次读取难以相互复用、后续读取可能错过缓存驻留期的特点，提出**时间线驱动的子图细分与热点访问相位优化算法**。依据算子实际发射、完成时刻细化子图，按**重复未命中字节**识别共享热点，在保持核心归属和依赖关系的前提下重排消费子图，协调首次填充与后续读取时机。五核下，同一方案无 L2 和有 L2 时相对整图单核基准的平均加速比分别为 4.3564 和 4.5028；逐用例计算的 L2 相对加速比均值为 1.0397，平均字节命中率为 25.24%

结果表明，场景 B 的驻留复用能够进一步提升多核调度性能，L2 Cache 可在同方案对照中带来额外收益；但缓存命中率只能反映数据复用覆盖程度，不能替代完工时间评价。本文将**张量共享关系与实际执行时序共同纳入调度决策**，实现了不同通信和存储条件下切图、分核与访问顺序的协同优化，所得方案均通过题设评测程序复核。

**关键词**： 多核调度；子图划分；张量共享感知；分层启发式算法；驻留复用；共享二级缓存

# 目录

一、问题重述 ..... 5
1.1 问题背景 ..... 5
1.2 问题重述 ..... 6
二、问题分析 ..... 8
2.1 问题一 ..... 8
2.2 问题二 ..... 9
2.3 问题三 ..... 10
2.4 问题关联性 ..... 11
三、模型假设与符号说明 ..... 13
3.1 模型假设 ..... 13
3.2 符号说明 ..... 13
四、问题一模型的建立与求解 ..... 15
4.1 模型建立 ..... 15
4.2 模型求解与结果分析 ..... 17
五、问题二模型的建立与求解 ..... 23
5.1 模型建立 ..... 23
5.2 模型求解与结果分析 ..... 25
六、问题三模型的建立与求解 ..... 30
6.1 模型建立 ..... 30
6.2 模型求解与结果分析 ..... 32
七、模型检验与分析 ..... 37
7.1 灵敏度分析 ..... 37
7.2 误差分析 ..... 39
八、模型评价与推广 ..... 41
8.1 模型的优点 ..... 41
8.2 模型的不足 ..... 41
8.3 模型的推广与改进 ..... 41
九、结论 ..... 43

参考文献.....44
附录 A 问题一逐用例结果.....47
附录 B 问题二逐用例结果.....51
附录 C 问题三无 L2 与只读 L2 Cache 逐用例结果.....55
附录 D 代码说明.....67
D.1 问题一.....67
D.2 问题二.....67
D.3 问题三.....67
附录 E 核心代码.....68
E1 问题一.....68
E2 问题二.....75
E3 问题三.....83
附录 F AI 工具使用说明.....90
F1 所用 AI 工具名称和版本.....90
F2 AI 工具的使用目的和环节.....90
F3 关键交互记录.....90
F4 使用原则.....91

# 一、问题重述

## 1.1 问题背景

题目研究由 N 个同构 AI 核心和共享核外主存构成的多核 NPU。每个核心包含矩阵计算、向量计算和两条数据搬运流水线，并配有私有片上存储。操作只有在依赖满足且对应流水线可用时才能发射；不同流水线在资源允许时可以重叠，同一流水线上的操作按顺序执行。各核心访问共享主存时受固定总带宽限制，因此操作完成时间既取决于自身数据量，也取决于同一时段其他核心的搬运请求。

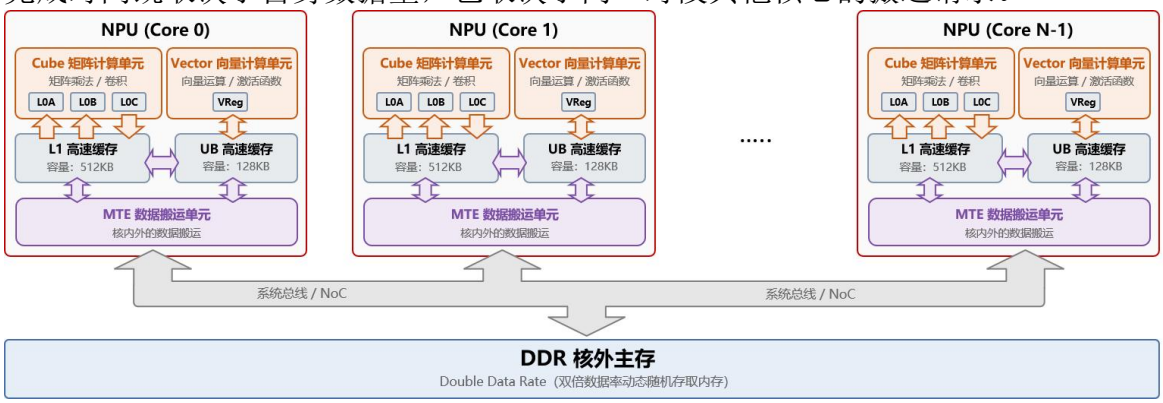
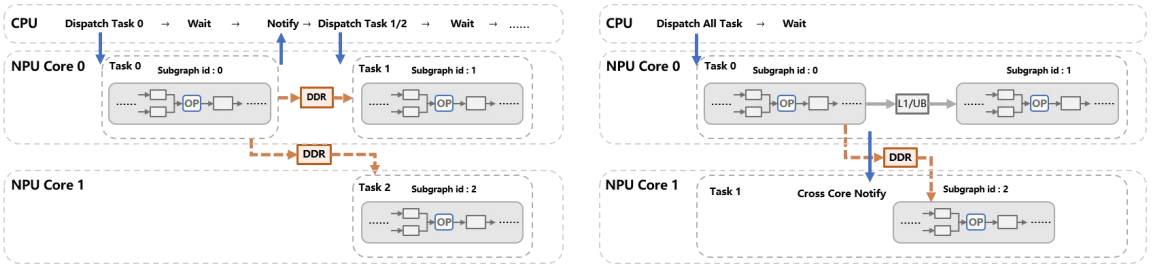



**图 1-1 多核 NPU 平台硬件架构**

图 1-1 给出了本文采用的硬件抽象。对调度而言，核心内部的计算和搬运流水线决定算子之间能否并行，片上存储容量决定中间张量能否驻留；核心之间共享的主存则决定跨核数据传输和并发访问的代价。后文的子图划分、核心映射和执行顺序均在这三类约束下确定。

输入数据由有向无环图给出。图中的算子节点记录计算或数据搬运操作，张量节点记录输入、输出及中间数据，有向边表示生产—消费依赖。本文需要共同决定三件事：将算子划分为若干子图、把子图分配给各个核心，以及确定每个核心上的子图顺序。子图过大时，中间张量复用较多，但可并行的任务数减少，且更容易受到片上存储容量限制；子图过小时，核心间并行机会增加，却可能引入更多边界搬运和同步等待。上述选择最终通过总完成时间和相对于原图增加的额外搬运量体现，因此不能只按算子数平均分配核心。




(a) 场景 A：核间同步需通过 Task 编排完成 / 子图间只允许 DDR 数据通信　(b) 场景 B：允许 Task 内部传递核间同步信号 / 同核子图间允许核内数据通信

**图 1-2 两种场景下子图间通信方式对比**

题目按通信和存储规则设置两个基础场景，并在场景 B 上增加共享二级缓存扩展。场景 A 中，每个子图单独作为一个任务，跨子图数据统一通过主存传递；即使相邻子图位于同一核心，任务切换时也不保留前一任务的缓存状态。场景 B 中，分配到同一核心的全部子图组成一个任务，在容量、依赖和执行顺序允许时复用片上存储中的驻留数据。

图 1-2 对比了两种场景的通信路径。场景 A 中跨子图数据必须经主存往返；场景 B 中同核子图在驻留条件满足时可复用片上存储，跨核依赖仍经主存并等待同步。场景 B 因而减少了部分主存搬运，但片上容量和同核顺序约束更紧。

问题三在场景 B 的调度基础上增加所有核心共享的只读二级缓存，并采用先进先出替换。多个核心读取同一输入张量时，命中缓存可减少主存访问，但缓存容量和带宽有限，替换、缓存排队以及计算流水线和跨核同步仍会影响最终时间。本文以全部核心完成计算的时间为主要指标，同时统计额外搬运量；问题三再报告按字节统计的缓存命中率及有、无二级缓存配置的加速比。

## 1.2 问题重述

题目给定一张有向无环计算图、核心数量及硬件参数。对除输入拷贝和输出拷贝外的算子，本文需要给出“算子—子图”映射；随后将每个子图分配到某个核心，并给出各核心上的子图执行顺序。提交方案必须覆盖全部待调度算子，所有子图在各核心序列中恰好出现一次，子图依赖图无环，且同一核心上的顺序不得违反依赖。官方评估器依据该映射和顺序，按对应场景生成边界拷贝、同步以及必要的缓存换入/换出操作，计算每个用例的总完成时间、额外搬运量和问题三所需的缓存指标。

三个问题共享上述三类决策变量，但通信规则和存储层次逐步增加。下面按“场景设定—决策内容—约束—输出指标”分别重述。

### 1.2.1 场景 A：独立任务下的切图与调度

每个子图单独构成一个任务，子图间的数据交换均通过主存完成；即使两个子图位于同一核心，也必须按场景 A 的规则写出和读回。对 2～5 个核心，确定子图划分、核心分配和各核心的执行顺序，使所有算子满足依赖和资源约束，并以缩短总完成时间为首要目标；在完工时间相同的方案之间，再比较额外搬运量。正文报告 1～5 个核心的平均加速比折线，单核加速比定义为 1；附录列出各用例的总完成时间和额外搬运量。

### 1.2.2 场景 B：同核任务下的切图与调度

将分配到同一核心的全部子图合并为一个任务。驻留与复用由片上存储容量、依赖和子图顺序共同决定；跨核依赖经主存搬运，并等待 500 个周期的同步延迟。对 2～5 个核心，联合确定切图、核心映射和同核顺序，以总完成时间为首要目标，并在时间相同的方案间比较额外搬运量。结果报告 1～5 个核心的平均加速比及逐用例评估结果。

### 1.2.3 共享只读二级缓存下的切图与调度

在场景 B 的任务组织和片上存储规则基础上，增加所有核心共享的只读二级缓存，容量为 1,048,576 字节，读带宽为 250 字节/周期，采用先进先出替换；缓存带宽与主存带宽相互独立，分别计入对应带宽池。输入拷贝发射时查询缓存，命中请求使用缓存带宽，未命中请求使用主存，完成后更新缓存；跨核依赖沿用场景 B 的 500 个周期同步延迟。比较 1～5 个核心下无二级缓存与有二级缓存的结果，报告相同核心数下的缓存加速比；附录给出两种配置的总完成时间、额外搬运量和按字节统计的命中率。命中率只描述访问路径，是否缩短总完成时间仍需结合替换状态、缓存带宽、主存竞争、流水线排队和跨核同步进行事件仿真[1]。

三个问题的提交文件均由两部分组成：每个算子的子图编号，以及各核心上的子图执行顺序。问题二沿用问题一的映射表示，但按同核任务规则重新评估；问题三在问题二的任务组织基础上加入二级缓存状态，并增加有、无二级缓存的对比指标。后文的模型和算法均围绕这两类输出展开。

# 二、问题分析

## 2.1 问题一

问题一的核心困难在于子图划分、核心分配与执行顺序三者耦合：划分决定跨核数据量与可并行粒度，分配与排序又决定 DDR 访问能否重叠，因此难以分步独立优化。场景 A 中，每个子图对应一个独立 Task，跨 Task 中间数据均经共享 DDR 传递。由此形成直接权衡：子图较大时边界搬运减少，但多核并行度受压缩；子图较细时任务分配更灵活，但数据读写与任务切换开销上升。同时，任务执行受前驱依赖、核心可用时间及激活间隔约束，各核心计算量均衡并不保证整体完工时间最短；多核并发访问 DDR 产生的带宽竞争，使单个任务耗时依赖整体调度，不能仅由自身计算量确定[4]。

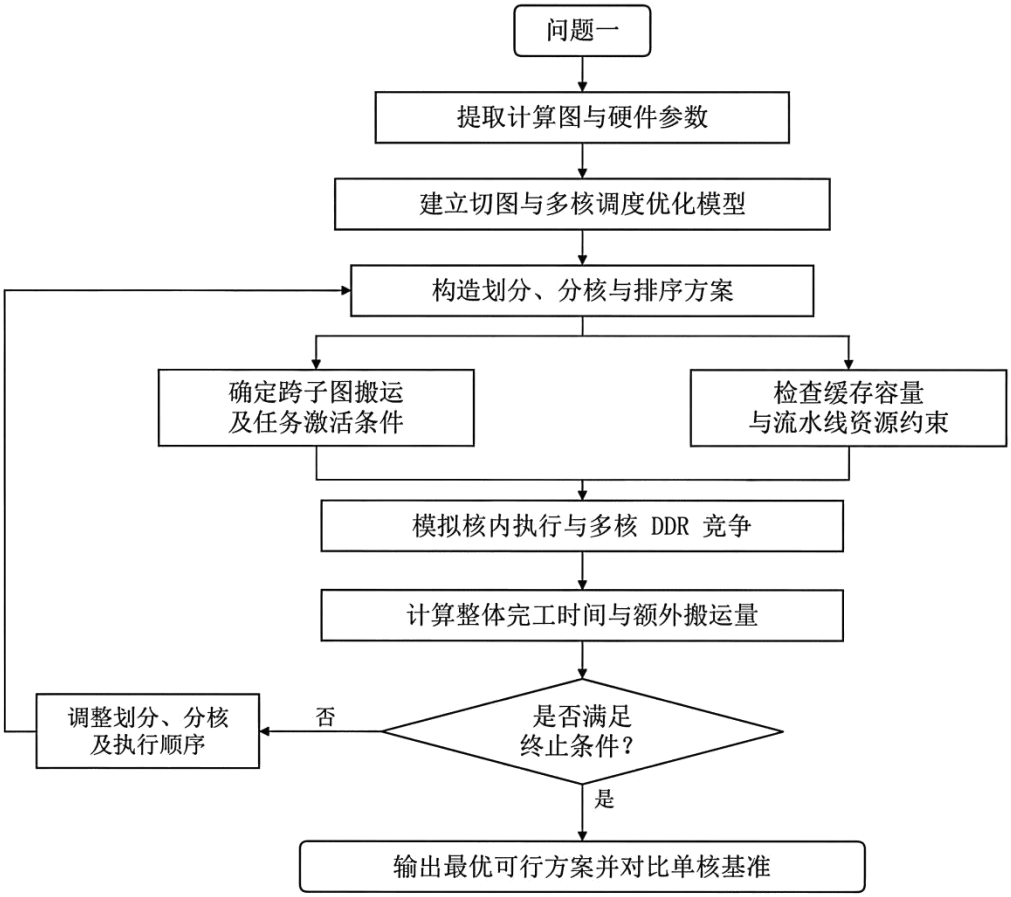



**图 2-1 问题一求解流程图**

由图 2-1 所示，本文从计算图中提取算子依赖、计算周期、张量大小及缓存需求，将子图划分、核心指派和核内排序设为联合决策变量；以整体完工时间最小为首要目标，以完工时间相同时额外搬运量最小为次目标，建立多核任务调度优化模型。求解时，先依据计算依赖与数据共享关系构造初始方案，在任务依赖、缓存容量和流水线资源约束下，结合核内执行规则与多核 DDR 带宽竞争机制建立调度模

拟器，计算各任务执行过程及整体完工时间；再根据模拟结果识别任务等待、核心空闲和数据搬运等主要耗时来源，调整子图划分、核心分配与执行顺序，迭代改进方案。输出调度结果后，以单核基准评价优化效果。

## 2.2 问题二

问题二的变化在于场景 B 下任务组织方式不同。问题一以单个子图作为一个独立 Task；本问将同一核心上的全部子图合并为一个 Task。这样，同核子图可通过本地缓存传递中间数据，减少经 DDR 的搬运；同时，子图边界不再要求前一子图整体完成后才能执行后一子图，在数据依赖、流水线和存储约束满足时，不同子图算子可以交叠执行。因此，方案需在计算并行度、同核数据复用与跨核通信开销之间权衡。

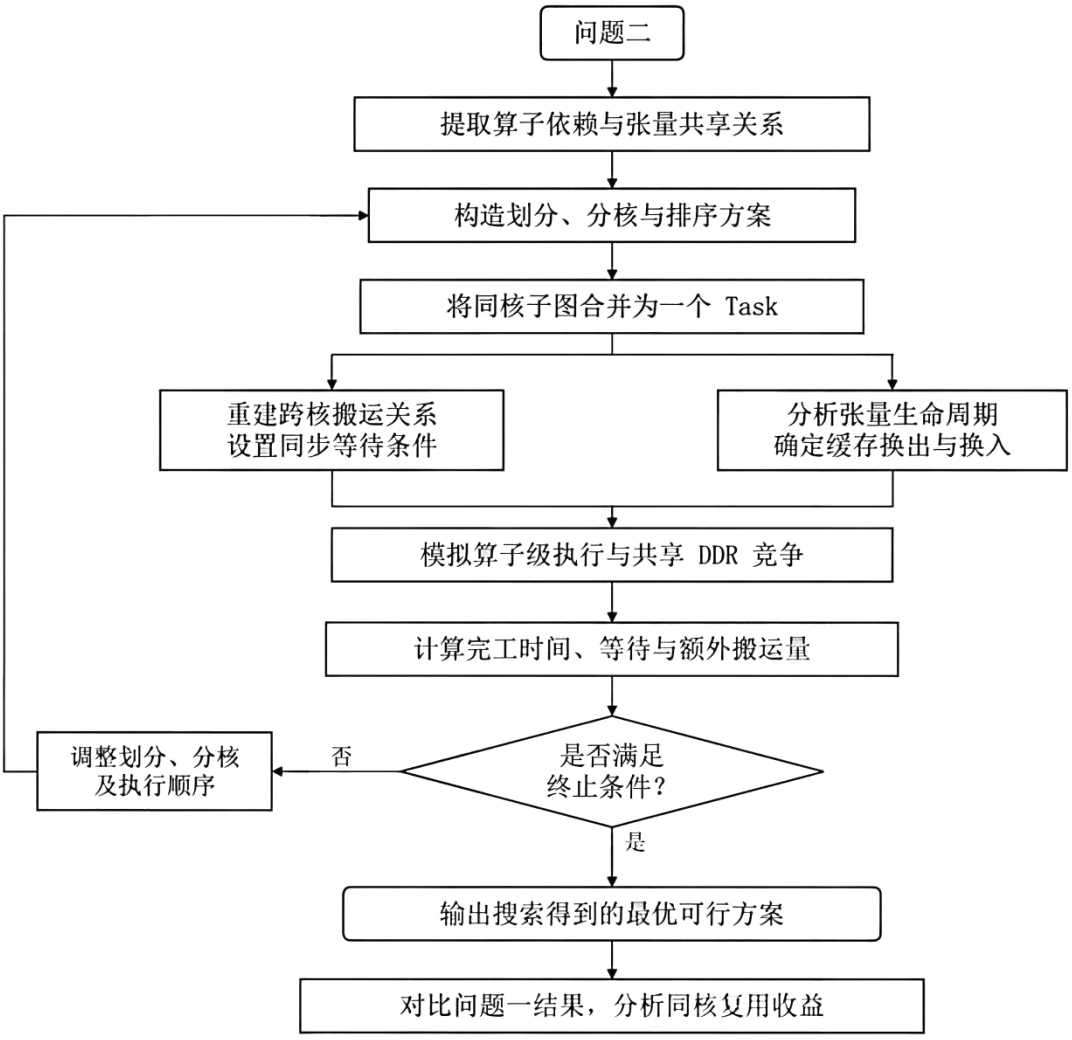



**图 2-2 问题二求解流程图**

将数据关联较强的子图分配到同一核心，可减少跨核搬运与同步等待；但过度集中会造成核心负载不均，限制并行执行。同核数据复用还受缓存容量与张量驻留时间共同约束：若张量产生后距下一次使用较远，其持续驻留会占用 L1 或 UB 空

间，容量不足时仍需换出至 DDR，并在后续使用前重新读入。因此，同核分配并不意味着数据搬运完全消除，还需结合执行顺序分析张量从产生、使用到释放的完整过程。对于跨核数据，目标核读入需等待源核写出完成并经过 500 个周期同步延迟，再结合其他执行条件判断是否就绪，不能以源核整个 Task 完成为统一起点。

如图 2-2 所示在问题一决策框架基础上，将同核数据复用与缓存驻留纳入方案评价。根据算子依赖、张量共享关系及计算负载构造子图划分、核心指派与核内顺序，并将各核心上的子图合并为 Task；再依据张量的生产者与消费者所在核心重建跨核搬运关系，避免同一张量在同一源—目标核心对之间重复建立传输。结合张量生命周期和 L1、UB 容量约束，确定必要的缓存换出与换入操作；通过多核模拟计算整体完工时间，模拟中计入算子级执行、跨核同步及共享 DDR 带宽竞争[9]。根据负载分布、通信等待与缓存换出情况调整方案，以整体完工时间最小为首要目标，时间相同时优先减少额外搬运量，并与问题一结果比较，分析任务组织方式变化带来的影响。

## 2.3 问题三

问题三在问题二的任务组织方式上引入共享只读 L2 Cache，容量 1MB，读带宽 250 B/cycle，采用 FIFO 替换策略，使多核可复用已缓存数据。此时，数据读取时间不仅取决于任务依赖与带宽竞争，还受 Cache 内容及访问顺序影响。但 Cache 不扩大各核心 L1、UB 容量，也不改变原有数据依赖与跨核同步约束。因此，调度模型需在问题二基础上显式跟踪 Cache 状态随执行过程的变化，并计入 Cache 读带宽竞争，而非仅将数据访问视为固定开销。

Cache 复用效果取决于读取与填充的先后顺序。COPY_IN 发射时，若目标张量已在 Cache 中，则通过 Cache 读带宽完成搬运，且命中不改变其 FIFO 排列位置；若未命中，则经 DDR 读取，并在搬运完成后填入 Cache。容量不足时按 FIFO 规则淘汰已有张量，单个张量超过容量时不予缓存。因此，同一张量被多次读取不必然命中：首次填充完成前的多个读取可能均访问 DDR；两次读取间隔过长，也可能因其他张量进入而被淘汰。此外，命中读取仍需竞争 Cache 带宽，过度追求复用可能增加任务等待。故命中率仅用于解释性能变化，优化目标仍为整体完工时间。

模型仍以子图划分、核心指派与核内顺序为联合决策变量，但在问题二调度评价中加入共享 Cache 状态。求解时，先根据张量大小、共享关系与算子依赖构造满足执行约束的候选方案；再在事件模拟中，按 COPY_IN 的发射与完成时刻更新命中、填充和淘汰状态，分别计算共享 DDR 与 Cache 读带宽竞争，并保留核内流水线、缓存换出换入及跨核同步规则。以整体完工时间最小为首要目标，时间相同时优先减少规定口径下的额外搬运量，据此迭代调整划分与调度方案。评价时需区分实际 DDR 访问量与额外 COPY 统计量，不能将 Cache 命中直接视为删除一次搬运。最后，在相同测试用例、核心数量和调度方案下，对比启用与关闭 L2 Cache 的完工时间，并结合按字节统计的命中率与 DDR 访问量，分析共享缓存的实际收益。

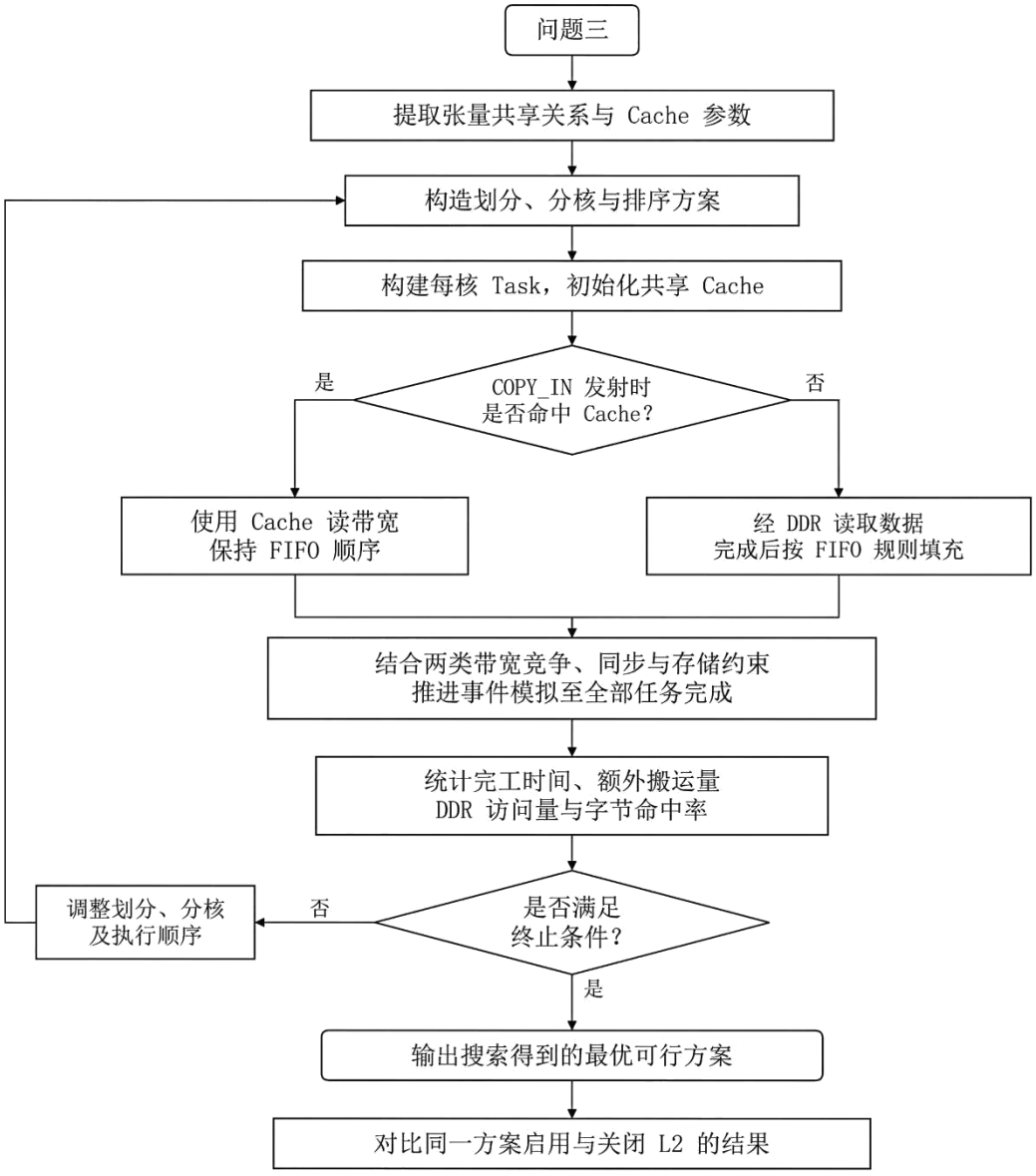



**图 2-3 问题三求解流程图**

## 2.4 问题关联性

问题的递进如图 2-4 所示，核心变化在任务边界与数据复用层次。问题一以独立子图为一个 Task，复用限于子图内部，跨 Task 中间数据均经 DDR 传递；问题二将同一核心上的子图合并为一个 Task，复用扩展至同核子图之间，并将跨核等待从 Task 完成细化为张量写出加 500 周期同步；问题三引入共享只读 L2 Cache，使不同核心的读取也可在 Cache 中复用。后一场景均保留前一场景的数据依赖与资源约束，仅新增相应状态与竞争，因此其收益需由完整执行模拟中的整体完工时间判断，不能由复用范围或命中率直接推断。

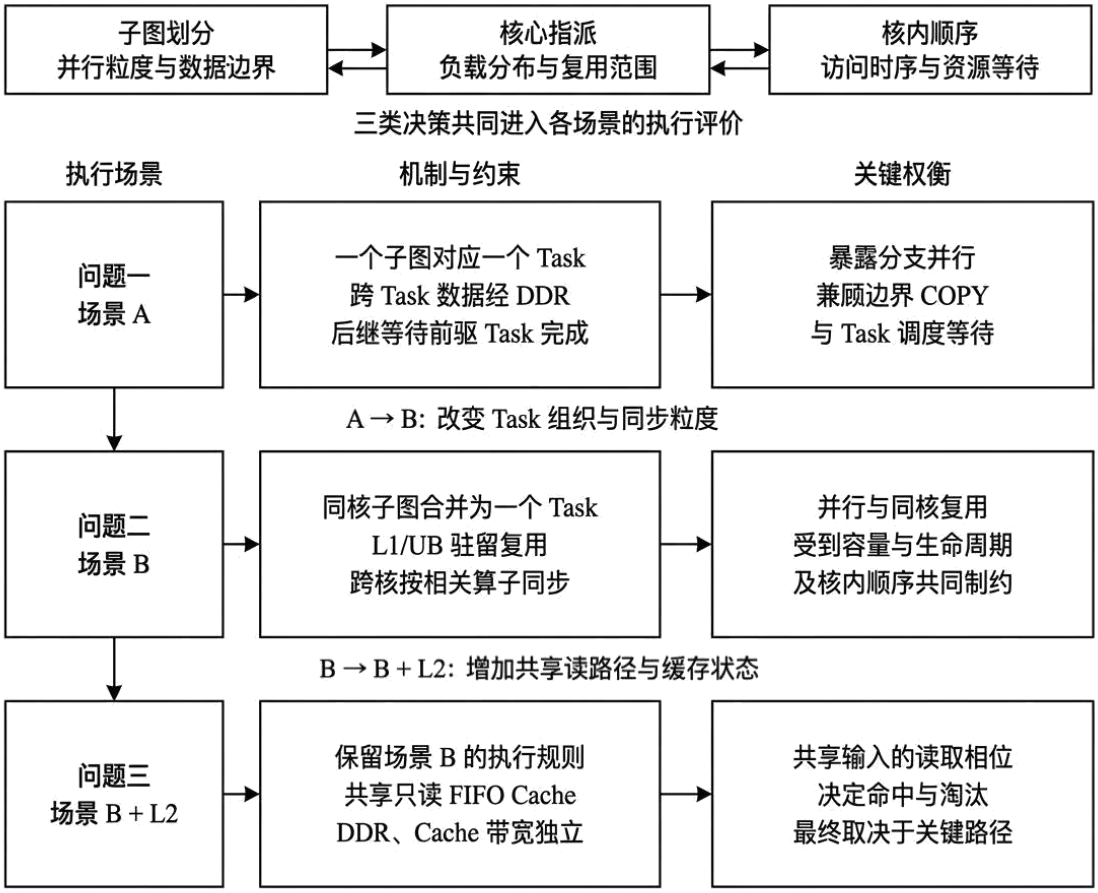



**图 2-4 问题的执行机制递进与共同决策关系**

# 三、模型假设与符号说明

## 3.1 模型假设

1. 输入计算图合法且无环，COPY_IN、COPY_OUT 仅为数据搬运，不作为切图对象。

2. 所有 AI 核心同构，计算能力及 L1、UB 容量相同，各操作仅在指定流水线上执行。

3. 仅考虑题目规定的计算、通信、同步及存储约束，忽略电压、频率和温度波动等因素。

4. 方案性能以官方评估结果为准，代理估计仅用于候选方案的筛选。

5. 求解在有限搜索预算内进行，不假定所得方案为全局最优解。

## 3.2 符号说明

本文主要符号及其含义如表 3-1 所示，三问共用一套符号体系，仅在不同场景下引入增量符号。

**表 3-1 主要符号说明**

| 符号 | 含义 |
|---|---|
| $G = (V, E)$ | 原始计算图 |
| $\mathcal{P} = (V_1, \ldots, V_m)$ | 非 COPY 算子的子图划分 |
| $b_t$ | 张量 $t$ 的字节大小 |
| $p_v$ | 算子 $v$ 的原始周期 |
| $M_{L1}$ | 每核 L1 容量 |
| $M_{UB}$ | 每核 UB 容量 |
| $B_C$ | Cache 池带宽 |
| $B_D$ | DDR 池带宽 |
| $C$ | 共享只读 L2 Cache 容量 |
| $c(s)$ | 子图 $s$ 的核心指派 |
| $Q_c$ | 核心 $c$ 上的有序子图序列 |

| 符号 | 含义 |
| --- | --- |
| $T$ | Makespan |
| $A$ | 额外搬运量 |
| $S_{i,N}$ | 用例 $i$ 在 $N$ 核下的加速比 |
| $R_{i,N}$ | 同方案 Cache 加速比 |

# 四、问题一模型的建立与求解

## 4.1 模型建立

问题一的核心不是将操作数量平均分配到各个核心，而是在满足计算图依赖、流水线资源和片上存储约束的前提下，同时确定切图边界、核心归属和执行顺序。切图会改变跨 Task 数据搬运量，核心分配会影响负载均衡和跨核等待，同核顺序又会改变 Task 的激活时刻、缓存占用及 DDR 访问的并发关系。因此，本文将三类决策统一表示为一个联合优化问题。已有多 DNN 加速研究将硬件架构、任务调度与核心映射纳入统一的协同优化框架，以处理不同决策之间的耦合关系[17]。本文在题设硬件配置固定的条件下，将子图划分、核心指派与核内顺序统一表示为联合优化问题。

### 4.1.1 计算图、划分与符号

将原始 Op—Tensor 二部图中的“生产算子—张量—消费算子”关系折叠为算子 DAG $G = (V, E)$。COPY 节点不作为划分对象，但经过被排除节点传递的依赖必须保留。对每个张量 $t$ 保存字节数 $b_t$、生产者、消费者、输入/输出标记及存储位置；对每个算子$v$保存周期$p_v$和所属 Pipe $\pi(v)$。算子 DAG 用于依赖和拓扑排序，张量关联用于边界搬运计费，二者同时保留。

核心数记为$N$，L1 和 UB 容量分别记为$M_{L1}$、$M_{UB}$，共享 DDR 带宽为 $B$。本题固定配置为 B、B、B/cycle；场景 A 的同核相邻 Task 间隔为 cycle，跨核前驱等待为 cycle。划分写为：

$$\mathcal{P} = (\mathrm{V}_1, \mathrm{V}_2, ..., \mathrm{V}_\mathrm{m}), \quad \bigcup_{\mathrm{s}=1}^{\mathrm{m}} \mathrm{V}_\mathrm{s} = \mathrm{V}, \quad \mathrm{V}_\mathrm{s} \cap \mathrm{V}_\mathrm{r} = \varnothing \ (\mathrm{s} \neq \mathrm{r}), \quad \mathrm{V}_\mathrm{s} \neq \varnothing . \tag{1}$$

令$c(s) \in \{0, ..., N-1\}$为 Task 的核心，$Q_c$为核心$c$上的有序 Task 序列。所有 Task 必须恰好出现在一个 $Q_c$ 中，允许部分核心为空；是否使用全部核心由目标函数决定，而不是预先强制。

### 4.1.2 目标函数、Task 依赖与搬运约束

将划分收缩为 Task 节点得到 Task 依赖图$G_{\mathcal{P}}$，在每个核心序列的相邻 Task 之间加入顺序边，得到增广图：

$$\mathrm{G}^{+} = \mathrm{G}_{\mathcal{P}} \cup \bigcup_{\mathrm{c}=0}^{\mathrm{N}-1} \{\mathrm{Q_c[j]} \rightarrow \mathrm{Q_c[j+1]}\} \tag{2}$$

要求$G^{+}$无环。仅检查 Task 数据依赖图并不能排除多个核心之间由排序共同产生的循环等待，因此所有顺序边必须合并后统一判环。

记$a_s$、$C_s$分别为 Task 的激活和完成时刻，整体完工时间为$T = max_s C_s$，完整评估函数记为：

$$(\mathrm{T}, \mathrm{A}) = \mathrm{Eval_A}(\mathrm{G}, \mathcal{P}, \mathrm{c}, \mathrm{Q}) \tag{3}$$

采用字典序目标：

$$\min_{\mathcal{P},c,Q} (T, A)_{lex} \tag{4}$$

额外搬运量严格按官方口径计算。设$D_{original}$为原图 COPY 字节总量，$D_{scheduled}$为切图、换出换入及边界操作全部完成后的 COPY 字节总量，则：

$$A = D_{scheduled} - D_{original} = A_{partition} + A_{spill}.$$

边界 COPY 按“张量—生产 Task—消费 Task”去重。同一张量从 Task 传给 Task 内的多个消费者时，只计一次写出和一次读入；外部输入与最终输出也必须计入。$A_{partition}$只反映切图边界，$A_{spill}$由 L1/UB 容量不足时官方 Step2 插入的换出换入产生，二者不能混为一谈。

4.1.3 执行约束与时间下界

令$q(s)$为 Task 在本核心序列中的直接前项，其本核释放时刻为：

$$L_s = \begin{cases} 0, & s \text{ 是该核心的首个 Task,} \\ C_{q(s)} + \delta_s, & \text{否则.} \end{cases} \tag{5}$$

跨核前驱释放时刻为：

$$R_s = \max_{u \in pred(s),\, c(u) \neq c(s)} (C_u + \delta_x) \tag{6}$$

空集合的最大值取 0，故 Task 的激活时刻是 $a_s = max(L_s, R_s)$。多个前驱的等待取最大值而非逐项相加；同核相邻 Task 即使没有数据依赖也必须满足$\delta_s$。

对每个 Task，官方核内流程先确定访问顺序，再依据 L1/UB 容量插入必要的换出换入，最后在四条 Pipe 上建立操作依赖和 FIFO 顺序。记展开后操作集合为$\Omega_s$，操作$i$的开始、结束时刻为$\alpha_i, \beta_i$，则：

$$\alpha_j \geq \beta_i \ (i \to j), \quad \alpha_i \geq a_s \ (i \in \Omega_s), \quad C_s = \max_{i \in \Omega_s} \beta_i \tag{7}$$

核内实际时长不能简化为四条 Pipe 累计负载的最大值；跨 Pipe 的数据依赖、Pipe 队首阻塞、地址复用的 WAR/WAW 约束及换出换入均可能增加时长。因而，将某 Task 单独展开所得时长$d_s^{iso}$仅作为列表调度参考，不能直接相加得到多核 Makespan。

对于涉及 DDR 的 COPY，先按满带宽工作量：

$$w_i = \max\left(1, \left\lceil \frac{b_i}{B} \right\rceil\right) \tag{8}$$

进行统计。当$k$个 COPY 同时存在剩余工作量时，共享带宽按 $1/k$ 的比例服务各传输；新 COPY 加入或旧 COPY 完成时重新计算剩余工作和结束事件。所有核心的 DDR COPY 共用同一带宽池，而非 COPY 操作不会仅因属于 MTE Pipe 就被计入该池。对每个核心和存储层次$h \in \{L1, UB\}$，任意时刻还需满足：

$$\sum_{t \in \mathcal{R}_{c,h}(\tau)} b_t \le M_h \tag{9}$$

在这些执行约束下，为降低候选筛选成本，统计 Task 的四类 Pipe 工作量、内部依赖路径和边界 DDR 工作量，构造乐观代理时长：

$$\hat{d}_s = \max\left(L_s^{int}, \max_p W_{s,p}, W_s^{DDR}\right) \tag{10}$$

其中 $L_s^{int}$ 为 Task 内部计算依赖路径，$W_{s,p}$ 为 Task 在 Pipe 上的工作量，$W_s^{DDR}$为其边界 DDR 工作量。固定划分的安全时间下界为：

$$LB(\mathcal{P}, N) = \left\lceil \max\left( \max_{P \in Paths(G_{\mathcal{P}})} \sum_{s \in P} \hat{d}_s,\ \frac{\sum_s \hat{d}_s}{N},\ \sum_s W_s^{DDR} \right) \right\rceil \tag{11}$$

三项分别对应 Task 依赖路径、核心总工作量和共享 DDR 服务量。只有$LB$严格大于当前最好$T$时才可安全跳过该固定划分；代理成本的排序本身不构成剪枝证明。

## 4.2 模型求解与结果分析

### 4.2.1 求解算法与结果检验

针对多 DNN 在多加速器核心上的映射问题，MAGMA 通过专门设计的搜索算子提高大规模映射空间的探索效率[16]。其结构化搜索思想为本文在有限预算下构造候选方案提供了参考。由于同时枚举所有划分、核心指派和排列的代价过高，程序采用”结构候选—代理筛选—精确模拟—局部调整”的有限搜索。首先保留整图单 Task 方案作为可行基线；随后依据安全链、拓扑序、独立连通分量、拓扑深度层和共享输入亲和关系生成候选[7]。候选经过完整覆盖检查和 Task 增广图判环后，才进入代理评价。

对一个候选划分，按 Task 依赖图逆拓扑计算后向优先级：

$$r_s = \hat{d}_s + \max_{u \in succ(s)} r_u \tag{12}$$

列表调度每次从前驱已被安排的 Task 中选取后向路径最长者，分别计算其追加至各核心后的预计完成时刻，选择最小者[8]。估计时使用同核间隔和跨核等待；核内展开完成后，再以$d_s^{iso}$更新列表顺序，并保留关键路径优先与最长 Task 优先两类结果。该阶段只产生候选，不预测全部多核 DDR 竞争。

精确评价时，Engine.prepare()依据题目 Step1/2/3 生成每个 Task 的固定核内执行图；Engine.simulate()再对给定的核心序列维护 Task 激活、Pipe 队首、操作前驱和全局 DDR 剩余工作量，采用离散事件推进至所有 Task 完成。当前方案附近还尝试拆分、合并和跨核迁移；只有完整模拟得到的$(T, A)$严格按字典序改善时才接受。对固定划分的下界、候选去重及未改变子图的核内展开缓存用于控制计算量，不能改变最终评分规则。图 4-1 给出了问题一的求解流程。

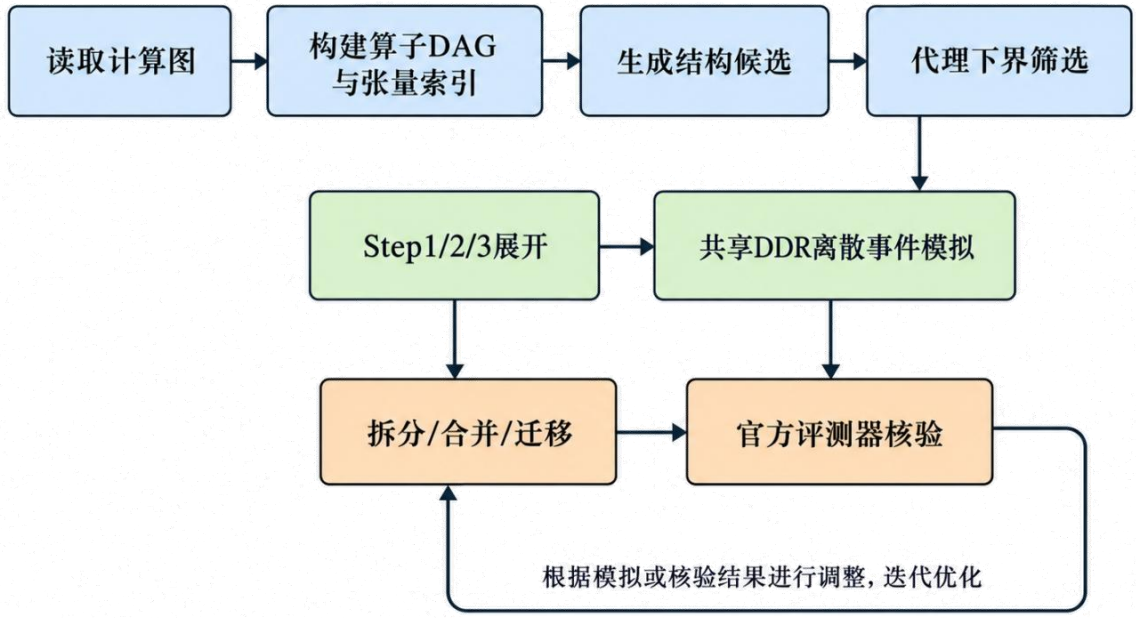



**图 4-1 问题一求解流程图**

为保证候选方案满足题目要求，在写出结果前进行两层检验：结构层检查非 COPY 操作是否唯一覆盖、每个 Task 是否唯一进入某个核心序列以及增广 Task 图是否无环；执行层检查 L1/UB 峰值、Pipe 顺序、地址复用依赖、Task 激活和共享 DDR 事件是否能够推进至全部 Task 完成。批量结果中的 verified 字段均为真，表 4-1 和表 4-2 所列结果均通过官方评测器核验，未以代理时长替代实际执行周期。

### 4.2.2 实验结果与模型分析

实验使用 100 个用例、2～5 核共 400 条多核记录，所有记录字段 verified=True。加速比按逐用例先计算、再对 100 个用例取算术平均：

$$S_{i,N}=\frac{T_{i,1}}{T_{i,N}},\quad \overline{S}_N=\frac{1}{100}\sum_{i=1}^{100}S_{i,N},\quad \overline{S}_1=1 \tag{13}$$

表 4-1 给出主要统计量，其中 added_bytes 为官方总额外搬运量，partition_added_bytes 与 spill_added_bytes 分别表示切图边界和换出换入贡献；“相对链式基线改善”为最优方案相对链式基线，即按链式结构组织的初始候选方案的完工时间之比；搜索时间是算法运行成本，不等同于官方评测时间。

**表 4-1 问题一在 100 个用例上的主要统计量**

| 核数 | 平均加速比 | 相对链式基线平均改善 | 平均额外搬运量/B | 其中切图/B | 其中 Spill/B | 平均 Task 数 | 搜索时间中位数/s |
|---|---|---|---|---|---|---|---|
| 2 | 1.9344 | 1.6909 | 9,657,279 | 7,302,823 | 2,354,455 | 32.1 | 5.50 |
| 3 | 2.6896 | 2.4038 | 10,223,649 | 7,467,256 | 2,756,393 | 48.8 | 5.83 |
| 4 | 3.3808 | 2.6735 | 10,020,066 | 8,038,956 | 1,981,110 | 61.7 | 5.26 |
| 5 | 3.9607 | 3.2943 | 10,853,293 | 8,630,585 | 2,222,708 | 78.1 | 5.42 |

由表 4-1 可见，平均加速比随核心数由 1.9344 增至 3.9607，但增长并非理想线性。加速比仍受原始依赖、Task 等待、共享 DDR 和片上容量约束共同限制。平均额外搬运量约为 $0.97\times10^7\sim1.09\times10^7$B，表明并行收益通常以新增边界流量为代价；切图贡献在四种核数下均占主要部分，但 Spill 并非可以忽略的固定项。

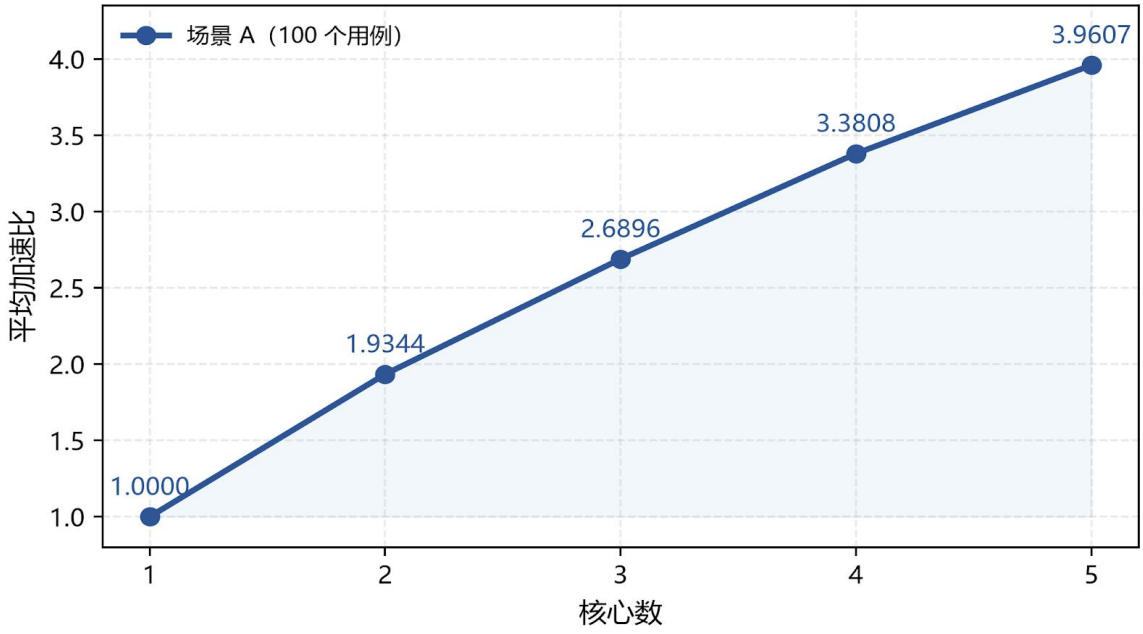



**图 4-2 1～5 核平均加速比折线图**

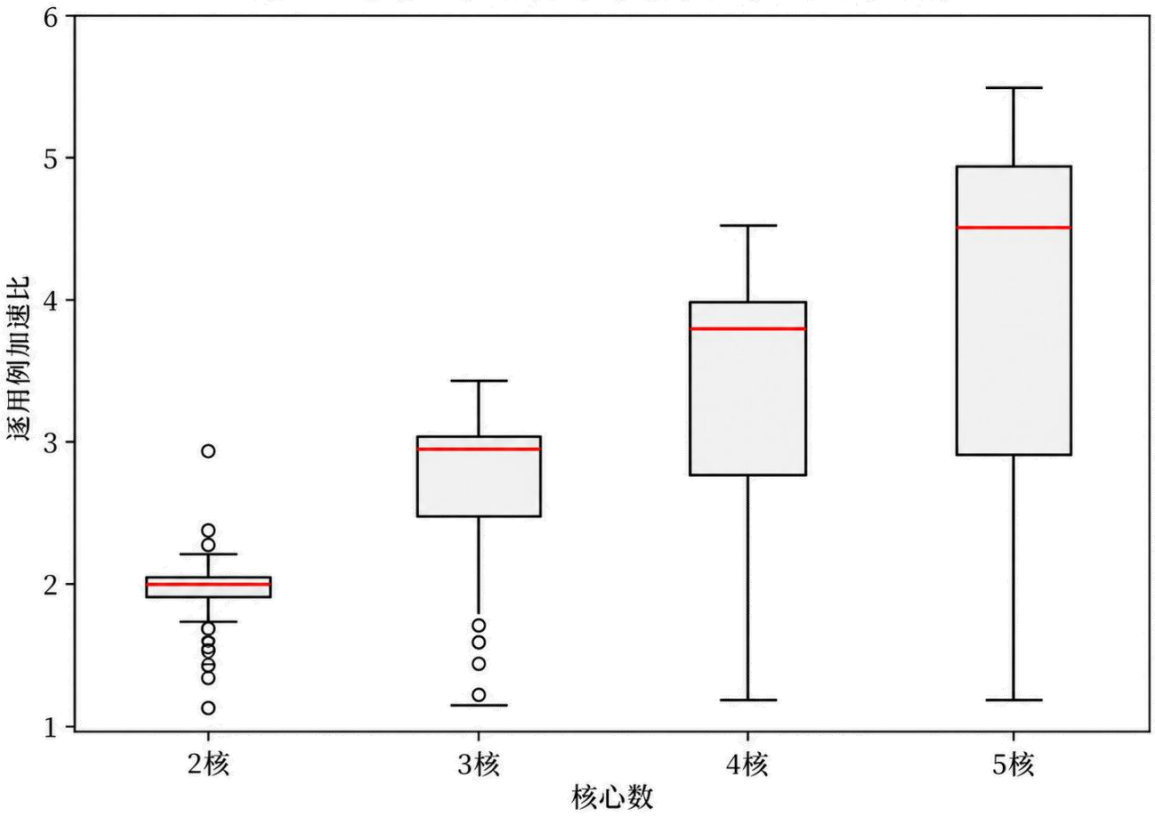



**图 4-3 按核心数分组的逐用例加速比分布箱线图**

**表 4-2 问题一代表用例计算结果**

| 用例与配置 | 单核 $T_1$ | 链式基线 $T_{base}$ | 最终 $T_N$ | 加速比 | 额外搬运/B | Spill/B | Task 数 | 平均 Task 占用率 |
|---|---|---|---|---|---|---|---|---|
| case_055,4 核 | 118,835 | 77,077 | 31,297 | 3.7970 | 206,340 | 0 | 8 | 0.8721 |
| case_064,4 核 | 19,116 | 20,493 | 16,304 | 1.1725 | 118,024 | 0 | 24 | 0.3172 |
| case_084,2 核 | 2,507,412 | 1,254,926 | 856,125 | 2.9288 | 17,316,288 | 4,700,160 | 7 | 0.9983 |

case_055 在四核下由链式基线的 77,077 cycle 降至 31,297 cycle，最终方案与基线的 Task 数均为 8，改善主要来自重新安排核心序列和瓶颈 Task 的结构调整，而不是简单增加 Task 数；其平均 Task 占用率为 0.8721，说明仍存在依赖或通信造成的空闲区间。case_064 的四核结果仅获得 1.1725 倍加速，平均 Task 占用率为 0.3172，说明图中可利用的并行工作有限，或切分后等待、边界搬运占比较高；五核时其 Makespan 与四核基本相同，继续增加核心几乎不能降低由依赖和通信形成的瓶颈。case_084 在两核下由链式基线的 1,254,926 cycle 降至 856,125 cycle，相对

单核取得 2.9288 倍加速，但同时产生 4,700,160 B 的 Spill 额外搬运，说明较高加速不能统一解释为 Spill 减少，多核加速还可能来自原图单核的资源串行、分支并行以及计算与搬运重叠。




**图 4-4 case_055 四核最终方案的核心 Task 甘特图**

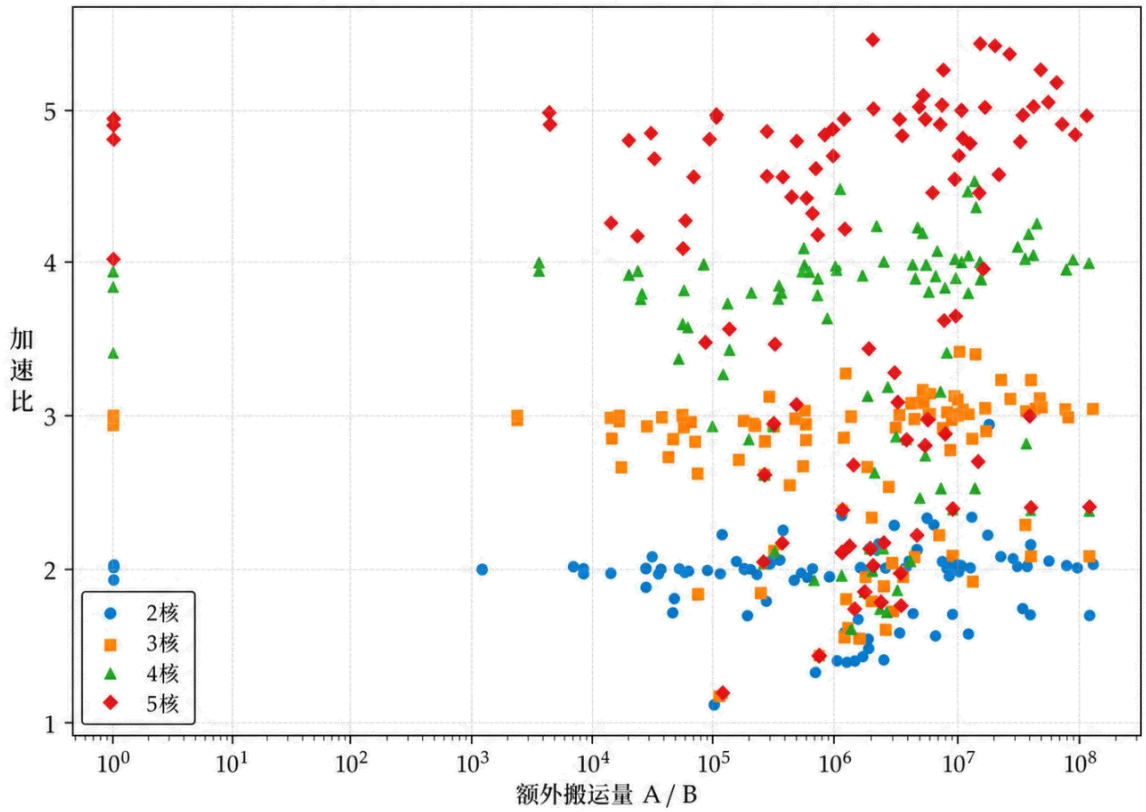



**图 4-5 逐用例加速比与额外搬运量散点图**

结合上述实验还需明确方法的适用边界：搜索只覆盖有限结构候选和有限局部邻域，不能证明得到全局最优；代理成本可能改变候选优先级，但不替代精确评分；共享 DDR 使一个 Task 的实际时长依赖其他核心的重叠传输，不能把独立 Task 时长直接相加。因此，实验结果是在给定搜索预算下通过官方规则核验的优良解。

# 五、问题二模型的建立与求解

## 5.1 模型建立

问题二与问题一的决策对象相同，仍需同时确定操作的子图划分、子图的核心归属以及各核心上的子图顺序。两者的区别在于执行语义：问题一中每个子图独立构成一个 Task，而问题二中同一核心上的全部子图合并为一个 Task，同核子图之间可以复用 L1 或 UB 中的驻留数据。因此，问题二的性能不再只由计算负载和边界搬运决定，还取决于子图顺序对张量生命周期、片上容量和换入换出行为的影响。

### 5.1.1 图表示与优化目标

原始图由算子节点、张量节点及其连接边组成。去除不参与划分的原始 COPY_IN、COPY_OUT 节点后，将“算子—张量—算子”关系折叠为算子依赖图$G = (V, E)$，其中$V$为非 COPY 算子集合，$E$ 为必须保持的先后关系。原始 COPY 的输入输出含义保存在张量索引中，用于后续边界搬运计算。预处理同时建立前驱、后继、拓扑序、弱连通分量和安全链索引。

对每个张量$t$，记录字节数$b_t$、原始存储位置、生产者集合$Prod(t)$、消费者集合$Cons(t)$，以及是否需要最终写回 DDR 的标志$F_t$。按拓扑序计算节点深度，按逆拓扑序计算后向路径秩；对 Pipe 统计原始算子工作量：

$$d_v = \max(1, p_v), \quad W_p = \sum_{v:\, \pi(v)=p} d_v \tag{14}$$

上述预处理只服务于候选构造、拓扑检查和下界计算；确定方案后，按官方 Step1—Step3 重新展开访问序列、驻留集合、Spill/Reload 以及通信事件，不能用静态张量总量代替实际状态。

设 $\mathcal{P} = (V_1, \ldots, V_m)$为划分，$c(s) \in \{0, \ldots, N-1\}$为子图$V_s$的核心，$Q_c$为核心$c$上的有序子图列表。划分满足：

$$\bigcup_{s=1}^{m} V_s = V, \quad V_s \cap V_r = \varnothing \ (s \neq r), \quad V_s \neq \varnothing \tag{15}$$

每个非 COPY 算子恰好属于一个子图，每个子图恰好出现在一个核心序列中，允许部分核心为空。若$s(v)$表示算子$v$所属子图，则其核心归属为$x(v) = c(s(v))$。同核子图依赖必须与$Q_c$的顺序一致；子图图和展开后的操作图均不得出现无法执行的循环等待。

对完整评估函数记$(T(P), A(P)) = Eval_B(G, \mathcal{P}, c, Q)$，问题二的优化模型为：

$$\min_{P \in \mathcal{F}_N} (T(P), A(P))_{\text{lex}}, \quad T(P) = \max_{c} \max_{i \in \Omega_c} \beta_i, \quad A(P) = D_{\text{scheduled}}(P) - D_{\text{original}} \tag{16}$$

其中$\mathcal{F}_N$是通过划分检查、核内展开和完整事件评估的$N$核可行方案集合，$\beta_i$是实际操作完成时刻。

5.1.2 执行约束与时间下界

对张量$t$，定义生产核心集合和消费核心集合：

$$P_t = \{x(v): v \in Prod(t)\}, \quad C_t = \{x(v): v \in Cons(t)\} \tag{17}$$

若 $P_t = \varnothing$，则每个消费核心需要一次图输入读入；若$F_t = 1$，则每个生产核心需要一次最终写回。对于 $p \in P_t, c \in C_t, p \neq c$，补充一对跨核写出和读入 COPY。同一目标核心的多个消费者共享该源—目标传输。设一次 COPY 的字节数为$b$，DDR 带宽为 B/cycle，其独占工作量为：

$$q(b) = \max\left(1, \left\lceil \frac{b}{B} \right\rceil\right) \tag{18}$$

跨核传输中，源写出完成时刻为$\beta_o$，目标读入的最早释放时刻为：

$$r_i = \beta_o + \delta, \quad \alpha_i \geq r_i, \quad \delta = 500 \text{ cycles},$$

其中$\alpha_i$还需同时满足目标核的数据依赖、Pipe 队首和 DDR 资源条件。若同时有$k$个 DDR COPY 活动，则每个 COPY 按 $1/k$的速率消耗其独占工作量；新传输加入或旧传输完成时，重新计算所有活动 COPY 的预计完成时刻。必要边界读写可形成代理量：

$$\widehat{D}(\mathcal{P}) = \sum_{t,c} b_t \left(n_{t,c}^{in} + n_{t,c}^{out}\right) \tag{19}$$

但$\widehat{D}(\mathcal{P})$不包含容量不足造成的实际 Spill，不能替代最终的$A(P)$。设$\mathcal{R}_{c,h}(\tau)$为时刻$\tau$核心$c$在存储层$h \in \{L1, UB\}$中实际占用的张量副本集合，则：

$$\sum_{t \in \mathcal{R}_{c,h}(\tau)} b_t \leq M_h, \quad M_{L1} = 524288, \; M_{UB} = 131072 \text{ (byte)} \tag{20}$$

容量不足时按官方规则插入 Spill/Reload，并对空间复用增加 WAR/WAW 等操作依赖。因而“同核生产—消费”只是具备驻留亲和的结构条件，是否真正减少搬运仍由访问顺序和精确内存展开决定。

除存储容量和跨核同步外，操作执行还需满足依赖关系与 Pipe 资源约束。对展开后的执行依赖$(i, j)$和同一 Pipe 队列中的相邻操作，分别有：

$$\alpha_j \geq \beta_i, \quad \alpha_{i_{k+1}} \geq \beta_{i_k}, \quad \beta_i = \alpha_i + d_i \tag{21}$$

每核的计算、向量、MTE2 和 MTE3 四条 Pipe 各至多执行一个操作；不同 Pipe 只有在依赖和存储约束允许时才能重叠。子图边界不额外施加整体屏障，实际顺序由官方 Step1 的访问序列、子图稳定分桶、Step2 的容量处理和 Step3 的 Pipe 队列共同确定。在多模型推理场景中，Layerweaver 通过交错执行计算密集型与访存密集型网络层，增加计算与数据传输的重叠，减少资源空闲时间[14]。这一思路说明，调度优化需要同时考虑计算资源与存储带宽的利用情况。

为减少精确模拟次数，对固定方案将必要 COPY 工作加入对应核心的 MTE2/MTE3 负载，记各 Pipe 增广工作量为$W'_{c,p}$，必要 DDR 独占工作总量为$Q(\mathcal{P})$，则资源下界为：

$$\mathrm{L_{res}}(\mathcal{P}) = \max\left(\max_{\mathrm{c,p}} \mathrm{W'_{c,p}},\ \mathrm{Q}(\mathcal{P})\right) \le \mathrm{T}(\mathcal{P}) \tag{22}$$

若所有收缩依赖都能由实际操作边或张量关系解释，再沿拓扑路径加入跨核的固定延迟和必要 COPY 工作，得到路径下界$L_{path}(\mathcal{P})$，并取$L(\mathcal{P}) = max(L_{res}(\mathcal{P}), L_{path}(\mathcal{P}))$。若资格检查不通过，只将资源项用于严格剪枝，路径项仅用于启发式排序。仅当$L(\mathcal{P}) > T_{best}$时淘汰候选，等号仍保留，以便寻找时间相同但搬运量更少的方案。

## 5.2 模型求解与结果分析

### 5.2.1 联合搜索方法

首先把整图置于一个核心，调用官方场景 B 入口得到单核基准。对$N = 2, 3, 4, 5$依次求解，并以已有较少核心方案补充空核心作为可行起点。随后构造多类候选：按弱连通分量装箱以暴露分支并行，按拓扑层段或安全链连续分段以保持局部性，按驻留亲和关系分组以减少潜在跨核输入，并补充深度窗口和两阶段流水候选。分组后按照后向路径秩进行列表分核，预计代价只用于候选优先级，最终仍须精确评估。

候选先做完整方案去重和结构合法性检查，不合法者直接舍弃；通过后再经下界剪枝，仅保留者进入核内精确展开。精评时，将同一核心上的非 COPY 操作合并为一个 Task，补上图输入、图输出和跨核 COPY；按访问顺序执行官方内存展开，插入 Spill/Reload 和空间复用依赖；最后在全局事件循环中模拟四条 Pipe、同步释放及共享 DDR。方案接受规则为：

$$\mathrm{accept(P)} \Leftrightarrow \mathrm{T(P)} < \mathrm{T_{best}} \tag{23}$$

在当前最好方案基础上，算法根据真实时间线选择 Pipe 忙碌量最大的瓶颈子图进行重切，再尝试同核顺序调整、相邻子图细分或合并，以及负载和同步迁移。候选配额分别为 24、16、8，搜索阶段的用户预算为 45s；已开始的候选评估和最终官方核验会继续完成，因此记录的 solve_seconds 可以略高于搜索预算。该过程属于有限邻域启发式搜索，不能据此宣称遍历了全部划分和排列。

### 5.2.2 实验设计与总体结果

实验使用 100 个用例，对 2 至 5 核分别求解，共形成 400 条配置记录。单核基准取同一用例整图单 Task 的官方完工时间；每个用例的加速比为：

$$S_{i,N} = \frac{T_{i,1}}{T_{i,N}}, \quad \overline{S}_N = \frac{1}{100}\sum_{i=1}^{100} S_{i,N} \tag{24}$$

平均值按用例加速比的算术平均计算，不将不同规模用例的完工时间先求和再取比值。表 5-1 给出最终求解结果。

**表 5-1 场景 B 在 100 个用例上的最终求解结果**

| 核心数 | 平均加速比 | 中位数加速比 | 平均求解时间/s | 官方核验 |
|---|---|---|---|---|
| 2 | 2.1168 | 2.0055 | 20.86 | 100/100 |
| 3 | 2.9616 | 2.9708 | 21.22 | 100/100 |
| 4 | 3.7309 | 3.8888 | 20.93 | 100/100 |
| 5 | 4.3471 | 4.7560 | 21.59 | 100/100 |

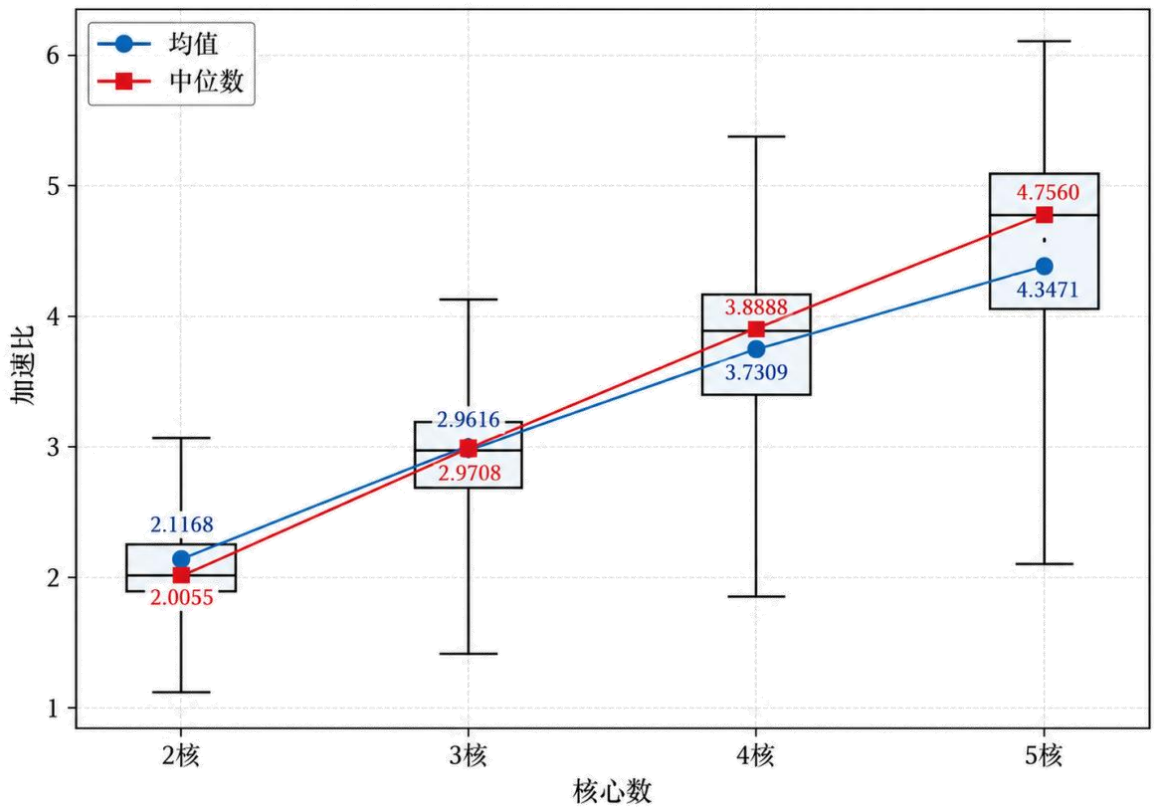



**图 5-1 2~5 核心数下的加速比统计**

部分用例的加速比高于核心数。本文加速比以整图单 Task 的官方执行结果为基准；单核下的片上容量压力、Spill、流水线重叠和访问顺序，与多核划分后的执行状态并不相同。超线性只说明相对该基准的完工时间下降，不能据此把收益归因到某个优化环节。

### 5.2.3 搜索效果与模型评价

记录初始、排序调整后与全部迁移结束后的方案，以观察局部调整的累计效果。初始方案已包含多结构构造和必要的瓶颈重切，因此后续数字为累计阶段记录，不构成严格的单因素消融。排序阶段在 2、3、4、5 核分别缩短 46、46、45、45 个用例的完工时间；迁移阶段在此基础上进一步改善 15、17、9、9 个用例。表 5-2 给出初始与最终平均加速比。

**表 5-2 问题二搜索阶段的累计改善记录**

| 核心数 | 初始平均加速比 | 最终平均加速比 | 排序改善数 | 迁移改善数 |
|---|---|---|---|---|
| 2 | 2.0697 | 2.1168 | 46 | 15 |
| 3 | 2.9001 | 2.9616 | 46 | 17 |
| 4 | 3.6358 | 3.7309 | 45 | 9 |
| 5 | 4.2452 | 4.3471 | 45 | 9 |

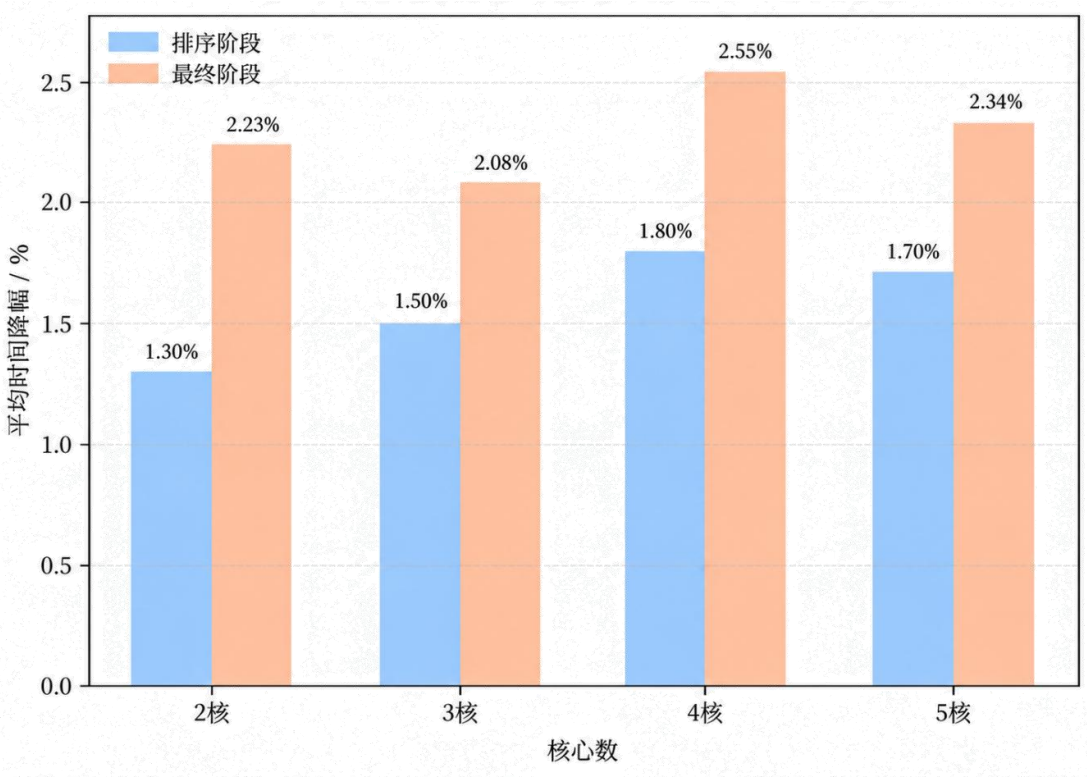



**图 5-2 搜索阶段相对初始方案的平均时间降幅**

新增搬运量呈现明显的结构权衡。按 added_bytes 的平均值换算，2、3、4、5 核分别为 6.179、7.222、7.282、7.155 MiB；其中 spill_added_bytes 的平均值分别为 4.738、4.309、3.380、2.831 MiB，Spill 占新增量的比例依次为 76.68%、59.66%、46.41%和 39.56%。随着核心数增加，边界搬运通常增加，而容量分散使 Spill 部分下降；这是总体统计，不能推断每个用例都具有相同趋势。

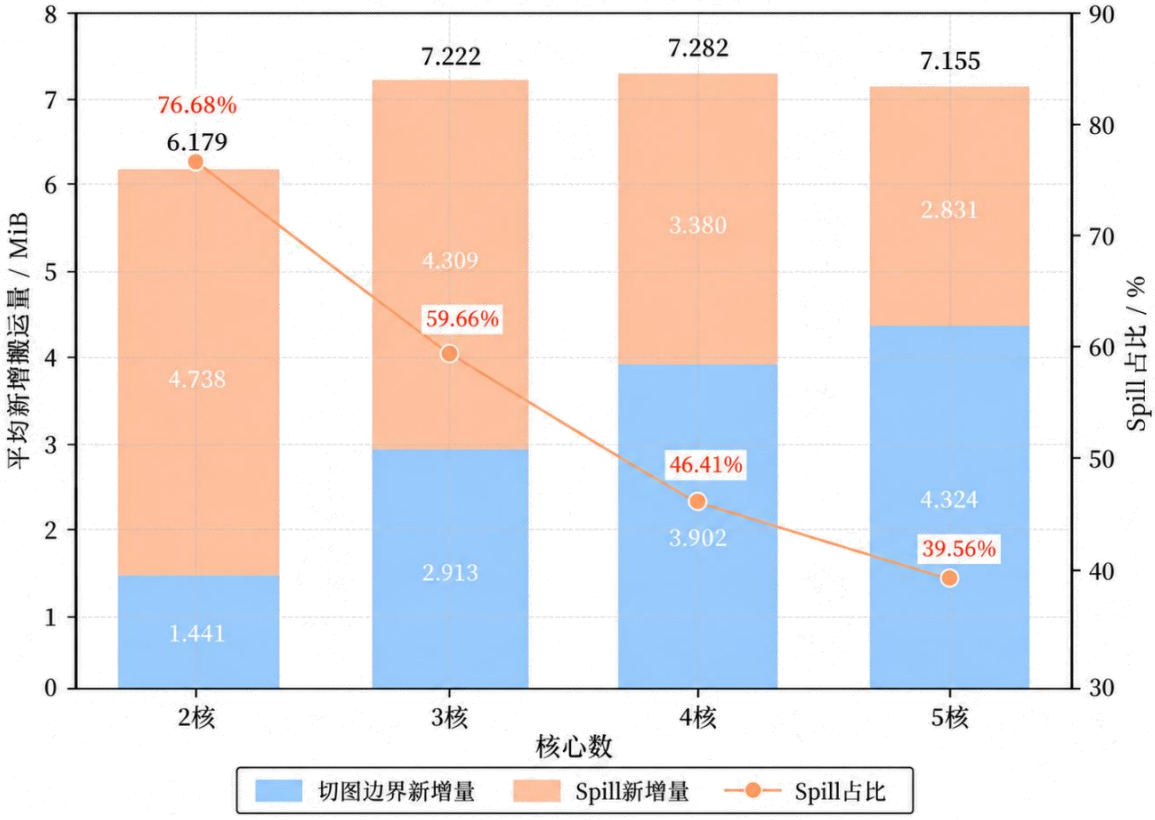



**图 5-3 新增搬运量的组成**

为进一步解释收益差异，按计算图结构考察求解结果。从 family 分组均值看，attention 类用例在 2 至 5 核的平均加速比分别约为 3.622、4.769、5.776 和 6.528，表现出较强的分支并行性；transformer 类分别约为 1.902、2.244、2.675 和 2.824，新增核心收益较弱，说明关键路径、同步和粒度细化对其限制更明显。该比较用于解释结构差异，不替代逐用例结果。

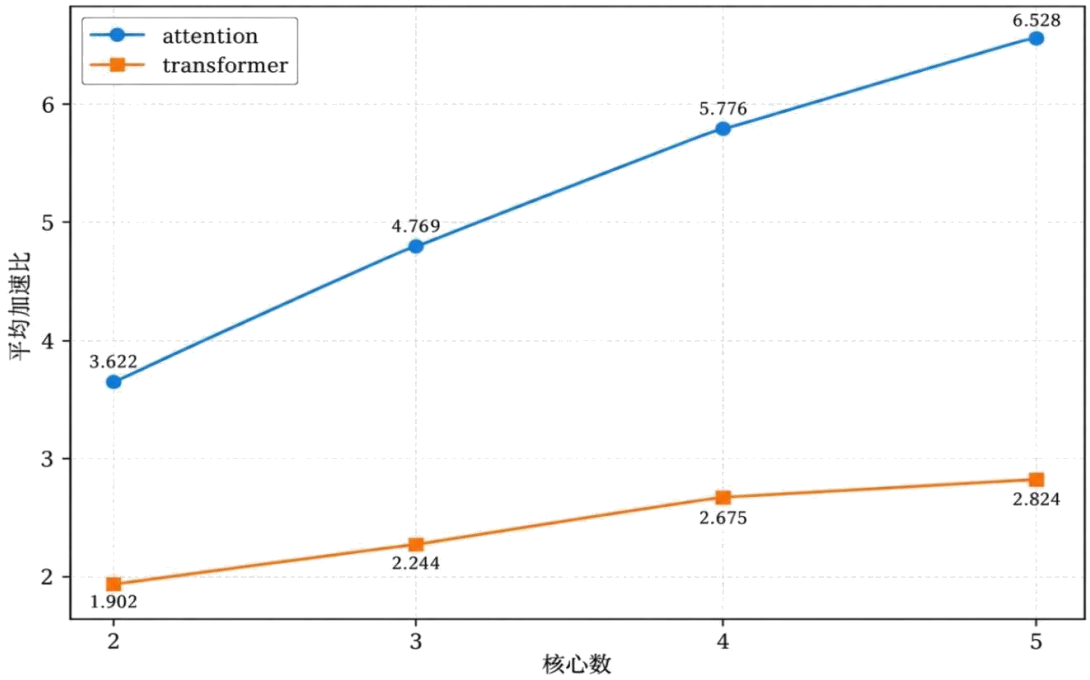



**图 5-4 不同计算图族的加速比对比**

模型的精确性来自最终官方复核：当前 400 条记录的 verified=True，说明输出方案、操作时间线和搬运统计与官方入口一致。与此同时，算法仍受候选结构、有限局部邻域和时间预算限制，near_optimal 只在全局时间下界满足条件时记录近优停止，不能把所有结果解释为全局最优。代理代价、路径下界和静态驻留亲和只服务于搜索，最终结论以精确评估的$T(P)$和$A(P)$为准。

# 六、问题三模型的建立与求解

## 6.1 模型建立

问题三在问题二场景 B 的基础上引入所有核心共享的只读 L2 Cache。仍需同时确定非 COPY_IN、COPY_OUT 操作的子图划分、核心分配以及各核心上的子图顺序。与问题二相比，方案不仅影响计算负载、跨核通信和 L1/UB 驻留，还会改变各核心发起 COPY_IN 的时刻，从而影响 L2 Cache 的命中状态、FIFO 淘汰过程以及 L2 带宽竞争。

### 6.1.1 划分、映射与执行约束

划分满足 $\bigcup_{s=1}^{m} V_s = V, V_s \cap V_r = \varnothing \ (s \neq r), V_s \neq \varnothing$ 并且 $c(s) \in \{0, ..., N-1\}$，$\pi_c$恰好排列分配给核心 c 的全部子图。若 $u \to v \in E$ 且两者不在同一子图，则在子图商图中保留相应有向边；商图必须无环，同核顺序不得反转原始依赖。

将官方展开后的所有操作（包括生成的 COPY、Spill 和 Reload）记为 $\Omega(P)$。对每个操作$i$记发射时刻为$\alpha_i$、完成时刻为 $\beta_i$。对数据或存储依赖$i \to j$，有$\alpha_j \geq \beta_i$。跨核心传递必须等待生产端 COPY_OUT 和固定同步延迟 cycle，因而$\alpha_{in} \geq \beta_{out} + \delta$，该约束即使在 COPY_IN 命中 L2 Cache 时仍然成立。不同 Pipe 可以重叠，但同一 Pipe 按官方队列串行；场景 B 不添加“前一子图全部完成后后一子图才能启动”的整子图屏障。

对任意时刻$\tau$、核心$c$以及片上空间 $h \in \{L1, UB\}$，令$\mathcal{R}_{c,h}(\tau)$ 为实际占用该空间的数据集合，则：

$$\sum_{t \in \mathcal{R}_{c,h}(\tau)} b_t \leq M_h, \quad M_{L1} = 524288,\ M_{UB} = 131072 \text{ (byte)}. \tag{25}$$

容量不足时由官方 Step2 插入 Spill/Reload，Step3 再将存储复用转化为 WAR/WAW 等依赖；不能以单个子图的静态张量大小之和替代这一动态约束。




**图 6-1 问题三候选生成、Cache 反馈与双配置复核流程**

### 6.1.2 FIFO Cache 与双带宽事件模型

用队列：

$$Q(\tau) = \left[(\kappa_1, b_1), (\kappa_2, b_2), ..., (\kappa_q, b_q)\right] \tag{26}$$

表示时刻$\tau$的 Cache 内容，队首为最早插入对象。队列必须满足：

$$\sum_{(\kappa,b)\in Q(\tau)} b \le C, \quad C = 1048576\ (B) \tag{27}$$

对具有有效逻辑张量键$\kappa_i$的 COPY_IN，在发射前按：

$$H_i = 1(\kappa_i \in Q(\alpha_i^-)) \tag{28}$$

判定命中，其中$\alpha_i^-$表示该发射事件前、已按官方顺序处理同刻退休和填充事件的状态。命中时不刷新 FIFO 位置；未命中的 COPY_IN 在完成时，若 $0 < b_i \le C$，先从队首逐个淘汰对象直至有足够空间，再把$\kappa_i$插入队尾。已经驻留的对象不重复插入，超容量张量不进入 Cache，COPY_OUT 也不填充 Cache。若同一张量的多个冷读同时发射，它们分别查询同一时刻的队列，不作请求合并。这一状态转移说明”命中率最大化”并不是本题目标：命中率只统计有效 COPY_IN 的字节比例 $h = D_{hit}/(D_{hit} + D_{miss})$，而$T(P)$还受 Pipe 阻塞、同步、DDR 写出、Cache 带宽和整数周期取整影响。如图 6-2 所示，给出 FIFO Cache 的状态转移。

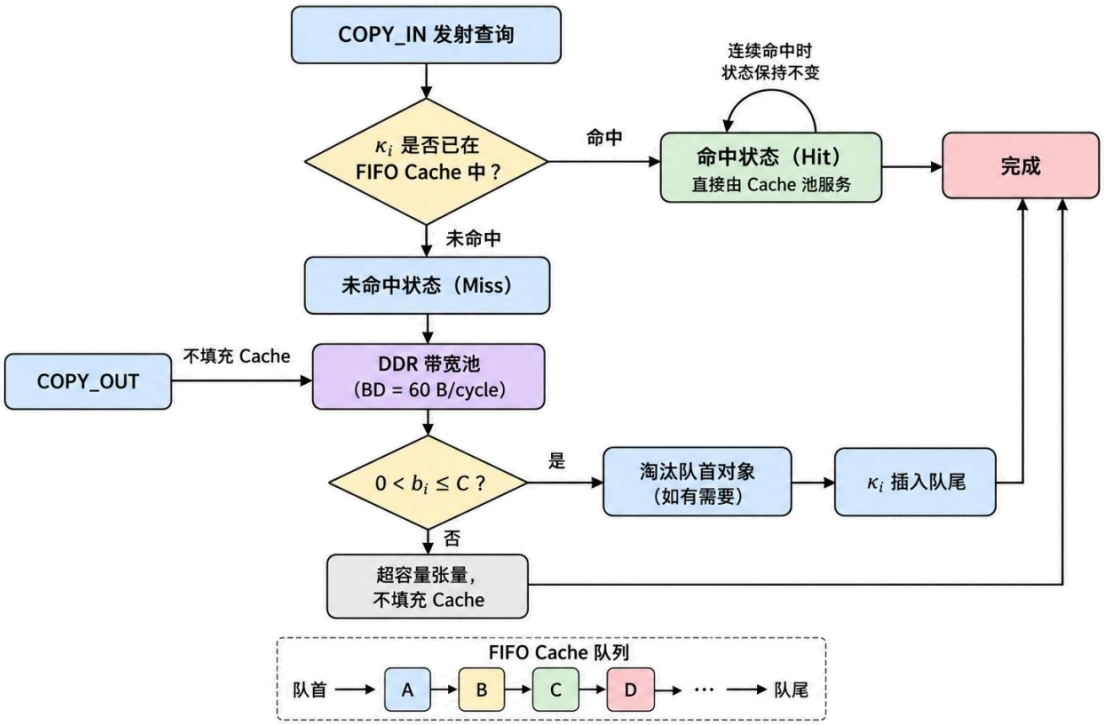



**图 6-2 FIFO Cache 状态转移图**

设张量字节数为$b_i$，DDR 带宽和 Cache 带宽分别为 B/cycle、B/cycle。满带宽下的最小传输工作量为：

$$q_i^{(D)} = \max\left(1, \left\lceil \frac{b_i}{B_D} \right\rceil\right), \quad q_i^{(C)} = \max\left(1, \left\lceil \frac{b_i}{B_C} \right\rceil\right) \tag{29}$$

命中请求进入 Cache 池，未命中读取与所有 DDR 写出进入 DDR 池。若同一池在时刻$\tau$中有$k_b(\tau)$个剩余工作量为正的传输，则每个传输按公平规则获得 $1/k_b(\tau)$的服务率；新操作加入或旧操作完成时重新计算剩余工作量和预计完成时刻，并按官方规则向上取整到整数周期。事件推进顺序为“推进带宽池—退休完成操作—更新依赖与 FIFO—发射就绪操作—重算池内完成时刻—跳转至最近事件”。这样处理可以保留命中释放 DDR 带宽的间接收益，也不会把总字节除以带宽这种静态估算误当作完工时间。

6.1.3 下界与接受规则

设各 Pipe 的总计算工作量为$W_p$，必要冷输入和最终写出的 DDR 时间下界为$M_0$，不计通信的最长依赖路径为$CP_0$，则核数为$N$时采用：

$$\mathrm{LB_{global}(N)} = \max\left(\left\lceil \frac{\max_{\mathrm{p}} \mathrm{W_p}}{\mathrm{N}} \right\rceil + \mathrm{I_0},\ \mathrm{M_0},\ \mathrm{CP_0}\right) \tag{30}$$

作为全局剪枝下界。对允许驻留的读取，固定方案下界可用最乐观的$q_F(t) = min\{q_t^{(D)}, max(1, \lceil b_t/B_C \rceil)\}$ 估计，但该式只用于证明候选不可能优于当前最好时间，不能宣称实际访问必然命中。候选经结构检查和下界筛选后，仍须执行完整官方展开与双带宽 FIFO 仿真；只有$T_{new} < T_{best}$或（$T_{new} = T_{best}$且$A_{new} < A_{best}$）时才接受新方案。报告中的$T/LB - 1$是下界差距上界，不是实验误差；启发式搜索一般不提供全局最优保证。

## 6.2 模型求解与结果分析

6.2.1 候选生成与精确求解流程

算法首先从原图提取算子 DAG、张量生产—消费集合、关键路径、Pipe 工作量和存储释放优先序。随后生成独立分支、DFS/拓扑分块、深度阶段、驻留分配和分叉—归约等多类初始方案；对精英方案再进行连续瓶颈重切、核内子图重排、细分/合并和跨核迁移。最后根据真实 Cache 事件定位重复冷读和“填充后再次未命中”的热点，调整消费相位、合法拓扑顺序及热点消费者归属。

每个候选均按”覆盖与商图检查—片上容量和同核顺序检查—安全下界剪枝—官方 Step1/2/3 展开—双带宽池与 FIFO 精确仿真”的顺序处理。当前正式实验采用每种核数 45s 的软预算、18 个初始候选、32 个局部结构候选、16 个 Cache 反馈候选和最多 2 轮反馈。导出方案后重新读取，并由原版 L2 评测器和同方案无 L2 评测器分别复核时间线、搬运统计、内存峰值和 Cache 事件。

6.2.2 实验设置与数据可靠性

本节所有数字均由 metrics.csv 和 summary.json 重新汇总得到，共 100 个用例、1～5 核 500 个配置，覆盖 12 类网络结构。固定参数为 B、B、B/cycle、cycle、B、B/cycle。500 个配置均标记为 verified；最终官方双配置调用数为 1000，累计搜索时间为 17552.87s，累计核验时间为 2927.66s。只有 7 个配置同时达到程序给出的全局时间下界，故其余配置只报告启发式方案及下界差距，不宣称全局最优。

表 6-1 问题三 1～5 核双配置汇总（100 用例平均）

| 核数 | 平均 L2 归一化时间 | 平均 L2 并行加速比 | 平均 Cache 加速比 | 平均字节命中率 | Cache 赢/负（100） |
|---|---|---|---|---|---|
| 1 | 0.904446 | 1.127131 | 1.000654 | 5.3801% | 29/0 |
| 2 | 0.475512 | 2.147554 | 1.002710 | 11.9815% | 50/0 |
| 3 | 0.340334 | 3.026942 | 1.008824 | 20.3731% | 63/0 |
| 4 | 0.276488 | 3.789150 | 1.025459 | 22.5383% | 66/0 |
| 5 | 0.239340 | 4.502776 | 1.039698 | 25.2430% | 73/2 |

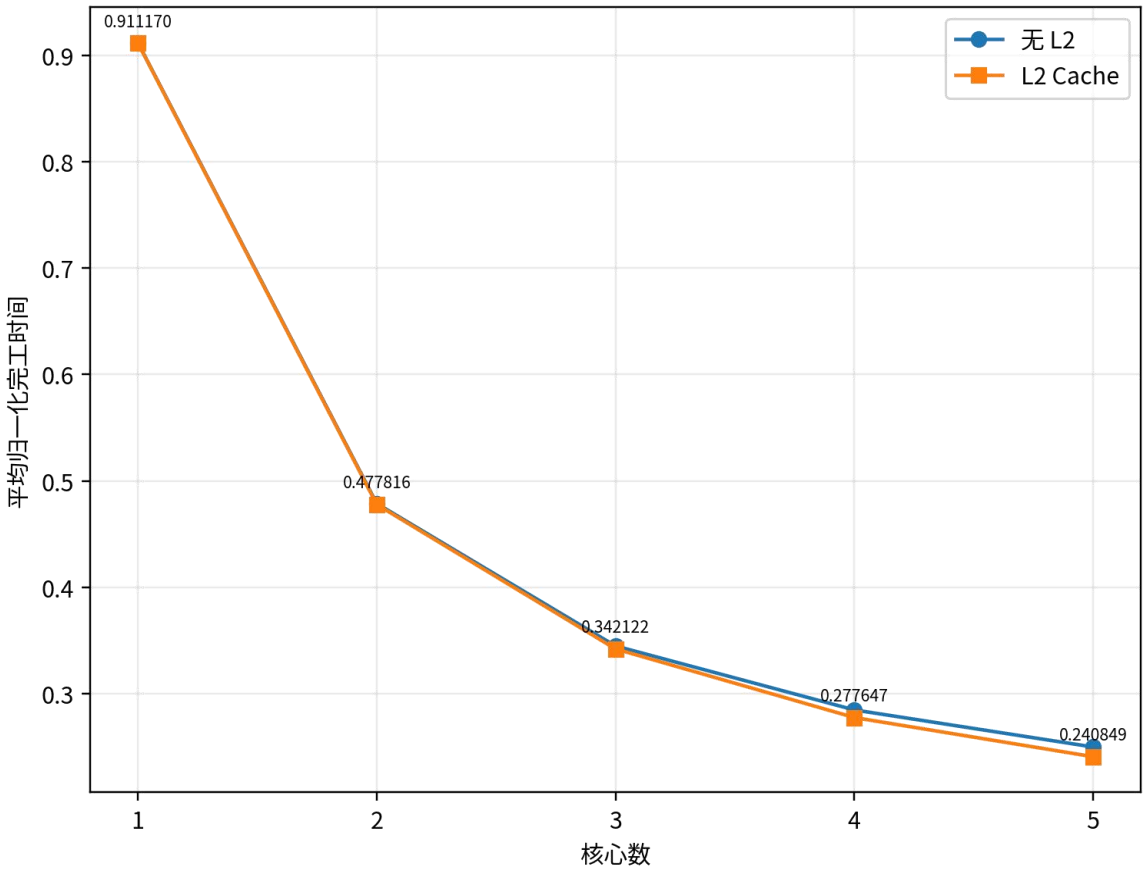



图 6-3 L2/无 L2 归一化完工时间

为比较不同核心数下的完成时间，先以每个用例的整图单核无 L2 结果作为基准，定义归一化完工时间$Z(i,N) = T_L2(i,N)/T_whole,noL2(i)$；无 L2 方案的曲线采用同一分母，将分子替换为$T_noL2(i,N)$。这里的 L2 与无 L2 结果使用完全相同的划分、核心映射和核内操作顺序，仅切换只读 L2 访问路径，因此表中的差异可归因于缓存事件和双带宽服务，而不是重新搜索得到的两套方案。

Cache 加速比按同一方案计算为$R(i,N) = T_noL2(i,N)/T_L2(i,N)$。逐用例比较时，严格小于 1 的情况记为“负”，严格大于 1 的情况记为“赢”，两者相等记为“持

平”；持平不计入任一方。命中率只统计有效 COPY_IN 的字节比例，用于解释访问路径，不替代完工时间指标。

表 6-1 给出逐用例指标先计算、再按核数取算术平均的结果；Cache 加速比按同一方案计算，命中率为有效 COPY_IN 的字节命中率；“赢/持平/负”分别表示 L2 完工时间严格小于、等于或大于同方案无 L2 完工时间。随着核心数增加，平均 L2 并行加速比由 1.127131 提高到 4.502776，Cache 平均加速比由 1.000654 提高到 1.039698，字节命中率由 5.3801%提高到 25.2430%。这三项指标的基准和含义不同，不能相互替代；五核仍有 2 个配置略微变慢，说明共享 Cache 不保证逐配置单调改善。

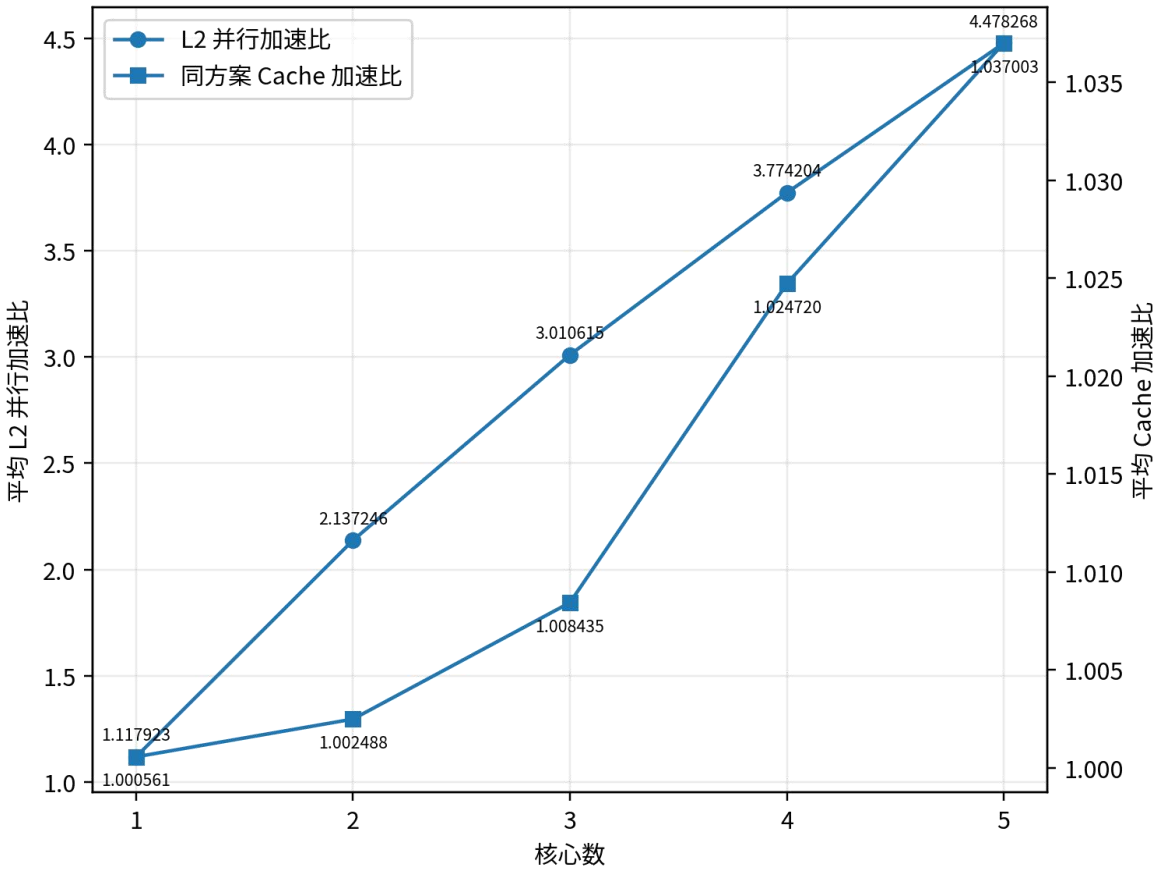



**图 6-4 并行加速比与同方案 Cache 加速比**

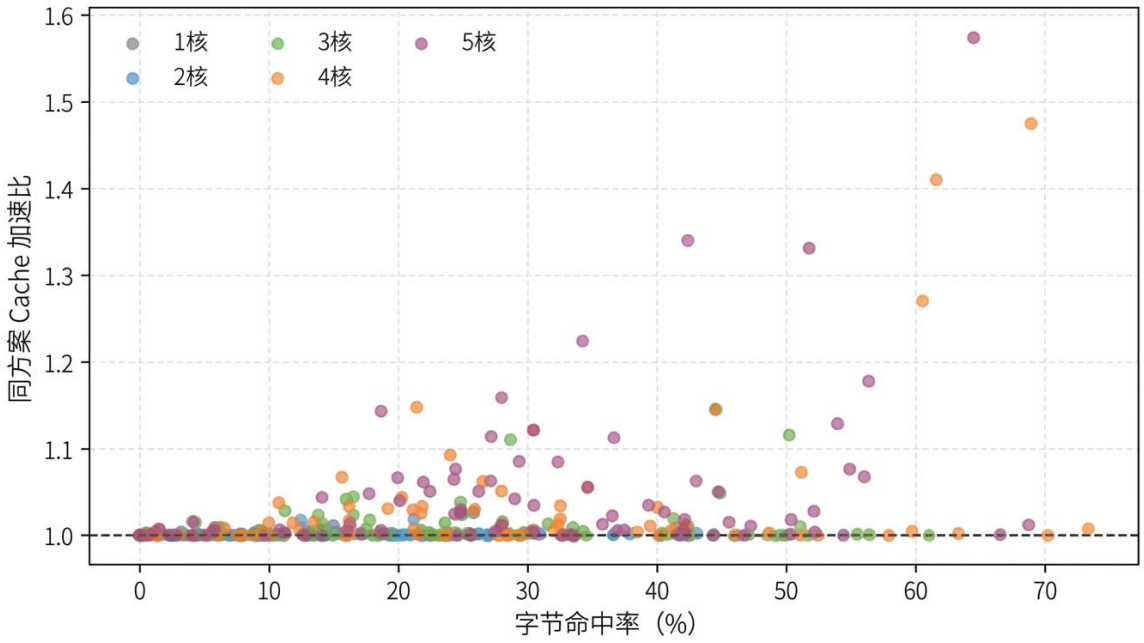



**图 6-5 字节命中率与同方案 Cache 加速比散点图**

6.2.3 典型用例与模型评价

表 6-2 选取具有代表性的五核结果。case_001 为浅层卷积、高并行分支，主要受计算 Pipe 约束；case_044 和 case_046 的共享读更集中，Cache 命中对 DDR 竞争缓解更明显；同时列出 case_074 作为命中但轻微变慢的反例。

**表 6-2 问题三典型五核用例结果**

| 用例 | 五核 L2 周期 | 同方案无 L2 周期 | Cache 加速比 | 字节命中率 | 额外 COPY/B | 解释 |
|---|---|---|---|---|---|---|
| case_001 | 47,502 | 47,502 | 1.000000 | 0.00% | 4,608 | 共享输入少，计算 Pipe 接近饱和，Cache 无可见收益 |
| case_044 | 37,817 | 50,676 | 1.340032 | 42.37% | 1,581,952 | 共享读和多核并行同时存在，命中缓解 DDR 竞争 |
| case_046 | 56,442 | 88,823 | 1.573704 | 64.46% | 3,037,568 | 命中覆盖高，Cache 池承担重复读取，收益最明显 |
| case_074 | 332,174 | 331,814 | 0.998916 | 33.56% | 4,821,002 | 命中存在但同步、排队和分区搬运抵消收益 |

case_046 在二、三、四、五核的 Cache 加速比分别为 1.018027、1.116156、1.475083、1.573704，说明增加核心数使共享读取机会增多，但收益还取决于消费相

位是否使对象在读取前已经完成填充。case_074 的命中率达到 33.56%，完工时间仍从 331814 增至 332174，说明不能以命中率替代完整事件仿真。类似地，额外 COPY 字节包含分区和 Spill 产生的官方搬运量，不能直接用命中字节相减得到。

问题三将问题二的驻留式场景 B 与共享只读 FIFO L2 Cache 统一到一个离散事件模型中。模型的核心不是给所有读取附加固定加速系数，而是在方案确定后按照真实发射顺序维护 FIFO 状态，分别推进 DDR 和 Cache 带宽池，并保留 Pipe、片上容量、跨核同步和官方搬运统计。全量 100 用例结果表明，多核切图与排序决定总体加速的主要部分，Cache 在高并行、共享读取明显的结构上提供额外收益，但其增益具有用例和核数依赖性，不能宣称所有配置均改善。最终结果均经过原版 L2 与同方案无 L2 双配置核验，正文中的最优性表述仅限于达到安全下界的少数配置，其余结果应称为经精确评估的启发式优良解。

# 七、模型检验与分析

## 7.1 灵敏度分析

灵敏度分析考察关键参数与输入特征对调度结果的影响程度，说明结论的稳健性与主要敏感来源。本文从核心数、计算图结构与 L2 Cache 参数三方面进行分析。

(1)核心数的影响。核心数是本文最主要的决策参数。三个问题中，随着核数增加，平均加速比整体上升，但边际收益递减。以问题一为例，2 核到 5 核的平均加速比依次为 1.9344、2.6896、3.3808、3.9607，相邻增量分别为 0.7552、0.6912、0.5799，单核并行度收益逐步放缓。其原因在于：切图引入的边界搬运、跨核同步与 DDR 带宽竞争随核数同步增长，抵消了部分计算并行收益，使加速比无法随核数线性增长。

(2)计算图结构的影响。图结构是加速比最强的敏感因素。表 7-1 给出问题一在 4 核配置下 12 类图结构的平均加速比：可并行的规约类（gemm_reduce，4.057）与注意力、浅卷积类（attention 3.981、conv_shallow 3.970）收益显著，而长串行链结构（vector_chain 2.384、transformer 1.986）收益有限。同一核数下结构差异可达 2 倍以上，说明“关键路径长度+可并行分支数”决定了多核收益的上限——切图只能释放并行度、不能压缩串行链。因此，对串行占优的图，单纯增加核数收益有限，应优先考虑核内 Pipe 并行与驻留复用。图 7-1 给出了这一结构差异的直观对比。

**表 7-1 问题一 4 核配置下各图结构的平均加速比**

| 图结构 | 用例数 | 平均加速比 | 最小 | 最大 |
|---|---|---|---|---|
| gemm_reduce | 10 | 4.057 | 3.183 | 4.478 |
| attention | 3 | 3.981 | 3.966 | 4.000 |
| conv_shallow | 2 | 3.970 | 3.952 | 3.988 |
| vector_elementwise | 2 | 3.962 | 3.940 | 3.985 |
| gated | 3 | 3.914 | 3.766 | 3.994 |
| mlp | 3 | 3.864 | 3.811 | 3.902 |
| gemm_shallow | 2 | 3.846 | 3.752 | 3.941 |
| conv_deep | 10 | 3.745 | 2.380 | 4.531 |
| vector_norm | 2 | 3.620 | 3.405 | 3.834 |
| mixed | 50 | 3.340 | 1.717 | 4.199 |
| vector_chain | 3 | 2.384 | 2.380 | 2.390 |
| transformer | 10 | 1.986 | 1.172 | 2.790 |

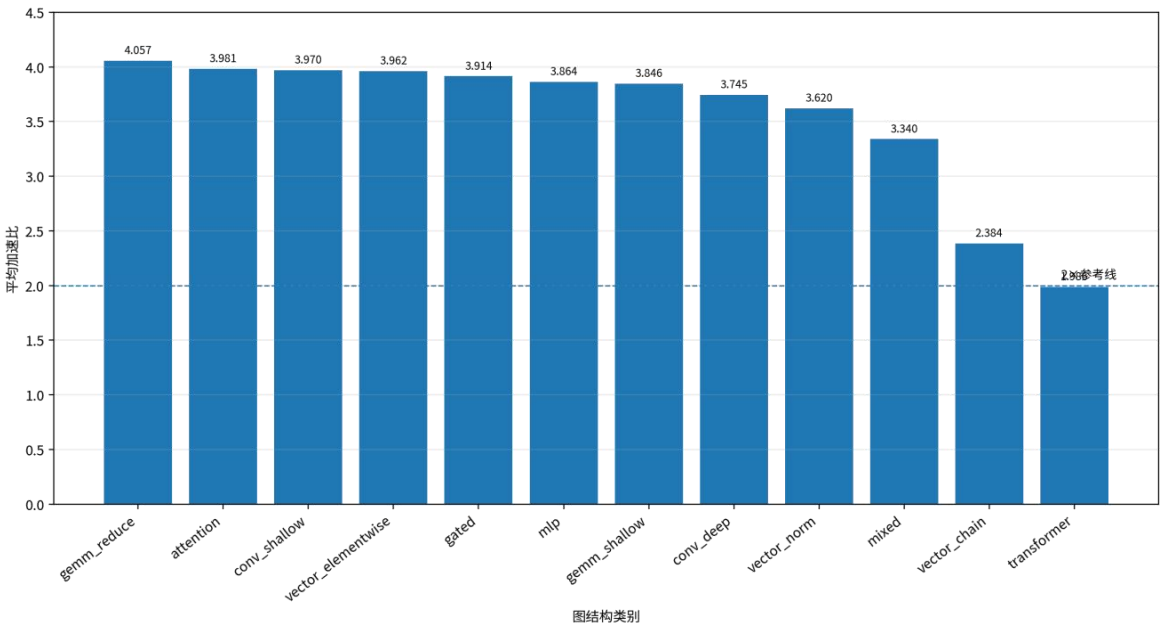



**图 7-1 问题一 4 核配置下 12 类图结构的平均加速比对比图**

(3)L2 Cache 参数的影响（问题三）。Cache 的收益随核数增加而增强：加权字节命中率由 1 核的 8.14%升至 5 核的 32.24%，同方案 Cache 加速比由 1.0007 升至 1.0397（表 7-2）。同时，在 100 个用例中，Cache 相对无 Cache 的“获胜”次数由 1 核的 29 增至 5 核的 73，而“劣化”次数始终不超过 2，说明 FIFO Cache 在本题配置下是近似“安全”的优化——几乎不产生负收益，但绝对收益受命中率上限与 FIFO 淘汰的约束。图 7-2 给出命中率与 Cache 加速比随核数的变化趋势，二者同向增长但幅度不同：命中率增长近 3.0 倍，而 Cache 加速比仅增长约 4.0 个百分点，印证了“命中率与完工时间并不等价”的结论。

**表 7-2 问题三 命中率与 Cache 加速比随核数的变化**

| 核数 | 平均命中率 | 加权命中率 | Cache 加速比 | Cache 胜例 | Cache 负例 |
|---|---|---|---|---|---|
| 1 | 5.38% | 8.14% | 1.0007 | 29 | 0 |
| 2 | 11.98% | 18.76% | 1.0027 | 50 | 0 |
| 3 | 20.37% | 23.61% | 1.0088 | 63 | 0 |
| 4 | 22.54% | 26.23% | 1.0255 | 66 | 0 |
| 5 | 25.24% | 32.24% | 1.0397 | 73 | 2 |

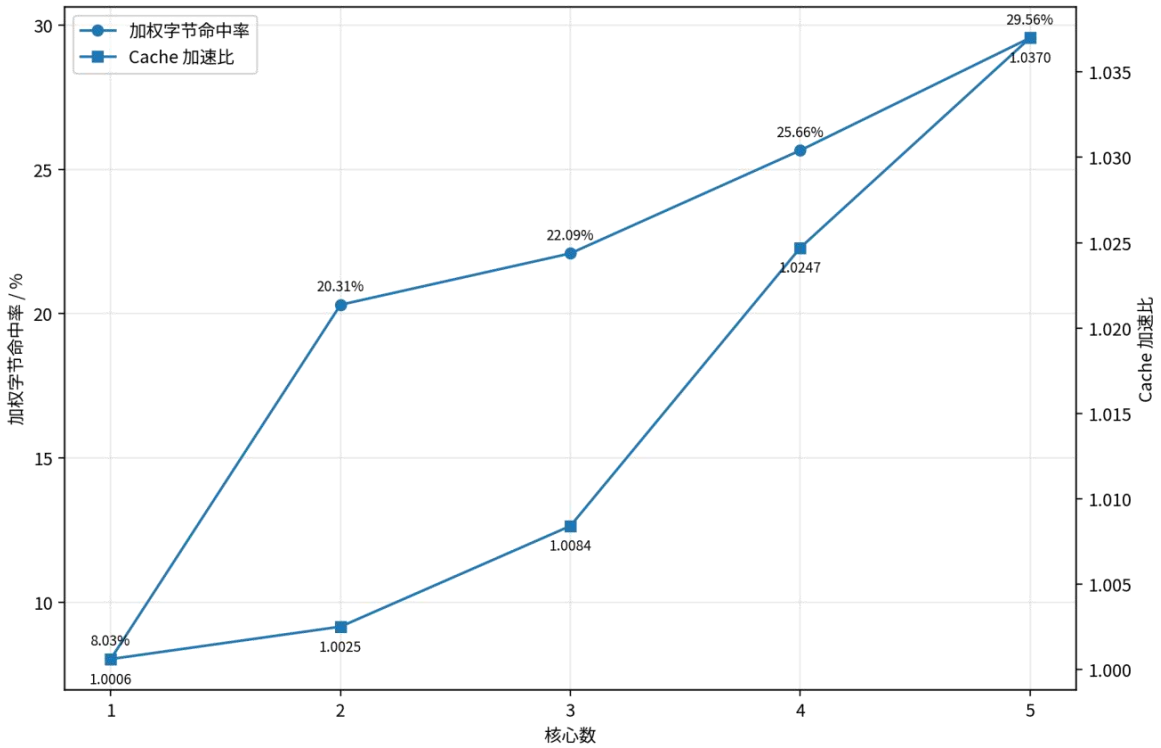



**图 7-2 问题三命中率与 Cache 加速比随核数变化图**

## 7.2 误差分析

误差分析从下界差距、最优性认证与核验一致性三方面说明解的质量边界。

**表 7-3 下界差距与最优性认证统计**

| 指标 | 问题二 | 问题三 |
|---|---|---|
| gap_upper_bound 均值 | 0.3441 | 0.2719 |
| gap_upper_bound 中位数 | 0.1443 | 0.0952 |
| gap_upper_bound 最大值 | 2.3600 | 2.7784 |
| 近最优/最优认证条数 | 18/400 | 7/500 |

(1)下界差距。对问题二与问题三，求解器维护全局下界$LB_{global}$，并以$gap_upper_bound = T/LB_{global} - 1$作为”完工时间与理论下界差距的上界”。表 7-3 显示，问题二的该指标中位数为 0.1447、均值 0.3454；问题三中位数为 0.0951、均值 0.2737。绝大多数用例的完工时间距理论下界在 10%～15%以内，说明解的整体质量稳定；但存在少量最大达 2.36（问题二）/2.78（问题三）的长尾，对应个别结构复杂、通信密集的用例，仍有进一步改进空间。该差距并非实验误差，而是启发式搜索相对全局最优的上界估计。

(2)最优性认证。问题二有 18 条（占 4.5%）结果被认证为 near_optimal，问题三有 7 条（占 1.4%）被认证为 certified_optimal，即在这些用例上完工时间严格达

到下界、解被证明最优。虽然认证比例有限，但它表明下界模型在部分用例上是紧的，为其余用例的差距估计提供了可信标尺。

(3)核验一致性。三个问题的所有结果均经官方评估器复核，verified 字段为真；自建模拟与官方 Step1/Step2/Step3 逐操作展开结果一致（assert_official 通过），因此不存在”自建评估与官方口径”之间的系统性偏差。代理时长仅用于候选排序，与实际模拟时长之间存在偏差，但它不进入最终指标，故不构成结果误差，只影响搜索效率。

# 八、模型评价与推广

## 8.1 模型的优点

(1)统一建模框架，三问递进而非重复。本文将子图划分、核心映射与核内排序设为联合决策变量，直接刻画并行度、Task 边界、片上驻留与共享带宽之间的耦合。三问沿用同一套符号、评估流程与求解骨架，仅逐级替换执行语义：问题一以子图为独立 Task，问题二合并同核子图并细化跨核同步，问题三加入共享 Cache 状态转移。该处理保证了问题间可比性，也避免为每个场景重复建模，并便于后续替换通信或缓存语义进行推广。

(2)参数与目标设置合理。采用字典序目标$min(T, A)_{lex}$，先最小化完工时间、再最小化额外搬运量，与题目评估口径完全一致；代理时长、下界与精确模拟严格分离，各司其职，避免把代理值误写为真实执行指标。

(3)候选生成与剪枝利用图结构，兼顾覆盖与开销。求解时利用拓扑深度、安全链、弱连通分量、驻留亲和等信息构造候选方案，并以安全时间下界、资源下界和路径下界剪枝。该策略不是盲目扩大搜索，而是将计算资源集中在更可能缩短完工时间或减少搬运的候选上，因此在搜索预算内能覆盖不同图结构。

(4)结果质量优良。三个问题均以官方核内展开与离散事件模拟为准，结果经原版官方评测器复核通过；多核加速比整体接近线性且稳定，问题三的 Cache 优化在几乎不产生负收益的前提下带来额外提升，输出方案效率高、稳定可靠。

## 8.2 模型的不足

(1)本文搜索不保证全局最优。求解基于有限候选与有限邻域，未证明所有测试用例达到全局最优；部分复杂图上解与下界之间的相对差距仍较大，说明候选生成和邻域调整尚未覆盖最优解附近区域。后续可通过引入更紧下界或结构感知邻域来缩小该差距。

(2)模型适用范围较窄。本文结果对应题目给定硬件配置与搜索预算，未对其他核数范围、异构核心或更大规模计算图进行系统检验；若硬件参数或图规模超出该范围，加速比与 Cache 收益不能直接外推，需重新标定参数并重跑模拟验证。

(3)部分因素处理有所简化。代理模型的线性化近似可能改变候选评估优先级，影响搜索路径；Cache 命中收益受访问相位、FIFO 淘汰、同步与 Pipe 排队共同影响，静态共享张量数量与命中率之间并非线性关系，本文仅做了有限刻画。

## 8.3 模型的推广与改进

在推广方面，本文“结构候选生成—下界剪枝—官方语义精确评估”的统一框架具有较强的可移植性：将通信模型替换为异构带宽或网络拓扑后，可推广到异构多核、多模型并发与分布式推理等场景；面向异构加速器，可参考异构数据流加速器中资源划分与层级调度的协同优化思想[15]，进一步将不同核心的计算能力、数据流方式及存储资源差异纳入映射决策。将 Cache 替换策略由 FIFO 替换为 LRU、预取等策略，即可复用到更一般的片上存储协同优化问题。在改进方面，可结合参考文献[6][9]进一步考虑抢占式调度与异构核心，结合文献[10][12]引入更细粒度的图划分与分层调度，结合文献[13]考虑数据布局感知的映射；同时引入灵敏度驱动

的自适应搜索预算，使计算资源向差距较大的用例倾斜，进一步提升解质量与求解效率的平衡。

# 九、结论

本文以统一的多核 NPU 调度框架处理三个硬件场景。问题一在场景 A 下以子图为独立 Task，通过子图划分、核心分配与同核排序缩短完工时间；问题二在场景 B 下将同核子图合并为 Task，引入同核驻留复用，并把跨核等待细化为算子级同步；问题三在问题二基础上加入共享只读 FIFO L2 Cache，通过事件级状态转移刻画命中、填充与替换对执行时间的影响。

全量实验表明，增加核心数可降低归一化完工时间，但收益受计算图结构、通信边界、缓存容量与带宽竞争共同制约，并呈边际递减。2～5 核下，问题一平均加速比分别为 1.9344、2.6896、3.3808、3.9607，问题二分别为 2.1176、2.9634、3.7346、4.3491；两问题差距随核数增加而扩大，说明同核复用与算子级同步在高核数下收益更明显。问题三的平均 Cache 加速比由 1 核的 1.000654 升至 5 核的 1.039698，共享 L2 带来额外收益，但幅度有限，且不同用例差异明显。灵敏度分析表明，加速比对核心数敏感但边际递减，对计算图结构高度敏感；误差分析中，约半数配置与下界的相对差距不超过 15%。上述结论以官方核验结果为准，在给定硬件配置与搜索预算下，所得方案具有稳定的高质量调度性能。

# 参考文献


[1] Jung H, Ji H, Pushchin A, Ostapenko M, et al. Accelerating Deep Neural Net works on Mobile Multicore NPUs[C]//Proceedings of the 21st ACM/IEEE Inter national Symposium on Code Generation and Optimization. 2023: 236-248.

[2] Gao C, Wang Y, Liu C, Wang M, Chen W, Han Y, Zhang L. Layer-Puzzle: Allocating and Scheduling Multi-task on Multi-core NPUs by Using Layer Het erogeneity[C]//2023 Design, Automation & Test in Europe Conference & Exhi bition (DATE). 2023: 1-6.

[3] Min H, Kwon J, Egger B. Flexer: Out-of-Order Scheduling for Multi-NPUs[C] //Proceedings of the 21st ACM/IEEE International Symposium on Code Genera tion and Optimization. 2023: 212-223.

[4] Sousa R, Pereira M, Kwon Y, Kim T, Jung N, Kim C S, Frank M, Araujo G. Tensor Slicing and Optimization for Multicore NPUs[J]. Journal of Parallel a nd Distributed Computing, 2023, 175: 66-79.

[5] Lee Y, Kim H, Yu Y, Cho M, Seo J, Park Y. Discovering Efficient Fused L ayer Configurations for Executing Multi-Workloads on Multi-Core NPUs[C]//20 24 Design, Automation & Test in Europe Conference & Exhibition (DATE). 2 024: 1-6.

[6] Niu W, Sanim M M R, Shu Z, Guan J, Shen X, Yin M, et al. SmartMem: L ayout Transformation Elimination and Adaptation for Efficient DNN Execution on Mobile[C]//Proceedings of the 29th ACM International Conference on Arc hitectural Support for Programming Languages and Operating Systems. Volume 3. 2024: 916-931.

[7] Lee S, Kim B, Choi Y, Lee H-J. HopScotch: A Holistic Approach to Data L ayout-Aware Mapping on NPUs for High-Performance DNN Inference[J]. ACM Transactions on Architecture and Code Optimization, 2025, 22(3): 1-26.

[8] Xiang Y, Wu Z, Yao H, Xiong X, Yang F, et al. Aries: A DNN Inference Sc heduling Framework for Multi-core Accelerators[C]//Proceedings of the 2024 5t h International Conference on Computing, Networks and Internet of Things. 2 024: 186-191.

[9] Yang W-H, Shin Y, Woo O, Park G, Ham H, Kang J. PyTorchSim: A Comprehensive, Fast, and Accurate NPU Simulation Framework[C]//Proceedings of the 58th IEEE/ACM International Symposium on Microarchitecture. 2025: 1363-1380.

[10] Zhu T, Feng D, Feng E, Xia Y, Chen H. Systematic Exploration of Multi-core Architectures for Efficient LLM Serving using ArguSim[J]. IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems, 2026.

[11] Y. Choi and M. Rhu, "PREMA: A Predictive Multi-Task Scheduling Algorithm For Preemptible Neural Processing Units," 2020 IEEE International Symposium on High Performance Computer Architecture (HPCA), San Diego, CA, USA, 2020, pp. 220-233,

[12] Baek, E., Kwon, D., & Kim, J. (2020). A Multi-Neural Network Acceleration Architecture. In Proceedings - 2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture, ISCA 2020 (pp. 940-963).

[13] Y. H. Oh et al., "Layerweaver: Maximizing Resource Utilization of Neural Processing Units via Layer-Wise Scheduling," 2021 IEEE International Symposium on High-Performance Computer Architecture (HPCA), Seoul, Korea (South), 2021, pp. 584-597

[14] H. Kwon, L. Lai, M. Pellauer, T. Krishna, Y. -H. Chen and V. Chandra, "Heterogeneous Dataflow Accelerators for Multi-DNN Workloads," 2021 IEEE International Symposium on High-Performance Computer Architecture (HPCA), Seoul, Korea (South), 2021, pp. 71-83.

[15] S. -C. Kao and T. Krishna, "MAGMA: An Optimization Framework for Mapping Multiple DNNs on Multiple Accelerator Cores," 2022 IEEE International Symposium on High-Performance Computer Architecture (HPCA), Seoul, Korea, Republic of, 2022, pp. 814-830.

[16] S. Zeng et al., "Serving Multi-DNN Workloads on FPGAs: A Coordinated Architecture, Scheduling, and Mapping Perspective," in IEEE Transactions on Computers, vol. 72, no. 5, pp. 1314-1328, 1 May 2023.

[17] Lixiang Han, Zimu Zhou, and Zhenjiang Li. 2024. Pantheon: Preemptible Multi-DNN Inference on Mobile Edge GPUs. In Proceedings of the 22nd Annual I

nternational Conference on Mobile Systems, Applications and Services (MOBI SYS '24). Association for Computing Machinery, New York, NY, USA, 465–4 78.

[18] Chunlin Li, Sen Liu, Kun Jiang, Mengjie Yang, Zihao Zhang, Bingxin Wang, Liang Zhao, Chen Chen, and Shaohua Wan. 2025. DNN Inference Acceleratio n Based on Adaptive Task Partitioning and Offloading in Embedded VEC. AC M Trans. Embed. Comput. Syst. 24, 4, Article 54 (July 2025), 35 pages.

# 附录 A 问题一逐用例结果

| 用例 | 类型 | 2 核 Makespan | 2 核额外搬运 | 3 核 Makespan | 3 核额外搬运 | 4 核 Makespan | 4 核额外搬运 | 5 核 Makespan | 5 核额外搬运 |
|---|---|---|---|---|---|---|---|---|---|
| case_001 | conv_shallow | 116,868 | 1,152 | 78,545 | 2,304 | 58,984 | 3,456 | 47,502 | 4,608 |
| case_002 | mlp | 130,518 | 1,585,152 | 89,748 | 56,832 | 68,731 | 56,832 | 60,659 | 640,512 |
| case_003 | mixed | 381,983 | 3,337,114 | 310,029 | 3,507,750 | 294,374 | 4,056,252 | 272,152 | 4,563,278 |
| case_004 | gated | 199,074 | 110,592 | 134,043 | 221,184 | 104,188 | 331,776 | 88,341 | 442,368 |
| case_005 | transformer | 69,131 | 1,055,190 | 61,796 | 1,168,190 | 59,610 | 1,350,310 | 55,199 | 1,434,124 |
| case_006 | mixed | 46,012 | 47,360 | 31,152 | 17,280 | 29,117 | 197,532 | 27,153 | 481,132 |
| case_007 | gated | 507,494 | 27,648 | 338,094 | 55,296 | 254,398 | 82,944 | 203,834 | 110,592 |
| case_008 | attention | 244,163 | 1,990,656 | 163,256 | 1,327,104 | 122,957 | 995,328 | 100,500 | 811,008 |
| case_009 | mixed | 89,625 | 45,860 | 85,278 | 1,213,430 | 79,751 | 671,522 | 71,426 | 1,282,612 |
| case_010 | mixed | 43,700 | 36,232 | 30,749 | 69,256 | 29,856 | 96,768 | 25,063 | 309,792 |
| case_011 | gemm_reduce | 92,646 | 180,224 | 69,625 | 540,672 | 46,880 | 540,672 | 43,852 | 1,175,552 |
| case_012 | mixed | 29,120 | 154,772 | 20,774 | 46,124 | 16,527 | 59,904 | 13,825 | 58,756 |
| case_013 | gemm_shallow | 132,198 | 8,192 | 88,351 | 16,384 | 66,982 | 24,576 | 54,363 | 32,768 |
| case_014 | mixed | 8,796,858 | 93,759,672 | 5,917,013 | 80,268,672 | 4,490,885 | 77,824,296 | 3,648,682 | 94,745,784 |
| case_015 | mixed | 87,183 | 290,056 | 60,807 | 306,448 | 48,821 | 855,836 | 42,412 | 732,152 |
| case_016 | vector_chain | 4,550,194 | 119,554,128 | 3,712,676 | 119,554,740 | 3,241,241 | 119,555,344 | 3,223,182 | 119,555,952 |
| case_017 | mixed | 81,898 | 86,400 | 53,825 | 557,568 | 39,865 | 561,024 | 33,822 | 502,272 |
| case_018 | mixed | 501,748 | 2,556,928 | 337,638 | 4,374,272 | 257,387 | 6,449,664 | 199,640 | 7,299,072 |
| case_019 | mixed | 43,224 | 27,168 | 28,696 | 263,200 | 24,099 | 51,744 | 19,894 | 57,892 |
| case_020 | vector_elementwise | 266,580 | 0 | 177,950 | 0 | 133,704 | 0 | 107,650 | 0 |
| case_021 | gemm_reduce | 3,099,240 | 4,608,000 | 2,102,932 | 9,355,264 | 1,550,324 | 4,608,000 | 1,243,902 | 7,372,800 |
| case_022 | mixed | 28,333 | 6,912 | 19,053 | 13,824 | 14,495 | 20,736 | 11,828 | 20,736 |
| case_023 | mixed | 60,205 | 281,280 | 39,576 | 280,992 | 30,795 | 561,604 | 25,191 | 280,992 |
| case_024 | vector_chain | 1,506,718 | 39,327,048 | 1,228,772 | 39,327,252 | 1,072,721 | 39,327,448 | 1,067,514 | 39,327,648 |
| case_025 | gated | 2,431,840 | 27,648 | 1,622,164 | 55,296 | 1,217,444 | 82,944 | 975,282 | 110,592 |
| case_026 | mixed | 49,670 | 193,776 | 38,033 | 259,544 | 38,033 | 259,544 | 38,033 | 259,544 |
| case_027 | gemm_reduce | 1,116,984 | 2,211,840 | 827,541 | 12,914,688 | 559,196 | 2,211,840 | 531,047 | 14,049,280 |
| case_028 | conv_deep | 20,350,357 | 39,450,720 | 13,602,485 | 39,640,320 | 10,313,001 | 44,406,144 | 8,268,563 | 44,935,680 |
| case_029 | vector_norm | 19,740 | 0 | 12,743 | 0 | 11,028 | 0 | 9,383 | 0 |

| 用例 | 类型 | 2 核 Makespan | 2 核额外搬运 | 3 核 Makespan | 3 核额外搬运 | 4 核 Makespan | 4 核额外搬运 | 5 核 Makespan | 5 核额外搬运 |
|---|---|---|---|---|---|---|---|---|---|
| case_030 | mixed | 1,393,015 | 10,006,528 | 932,872 | 10,086,400 | 696,592 | 11,737,088 | 558,469 | 15,542,272 |
| case_031 | mixed | 714,242 | 10,100,736 | 484,648 | 7,348,224 | 369,154 | 7,609,344 | 300,434 | 10,205,184 |
| case_032 | mixed | 20,000 | 30,864 | 14,016 | 63,856 | 10,946 | 24,720 | 9,966 | 24,720 |
| case_033 | mixed | 877,823 | 9,956,448 | 597,784 | 8,688,204 | 441,543 | 9,338,976 | 354,025 | 10,488,960 |
| case_034 | mixed | 396,717 | 4,509,696 | 268,499 | 5,752,832 | 200,764 | 5,133,824 | 166,667 | 4,713,472 |
| case_035 | mixed | 92,577 | 72,740 | 92,577 | 72,740 | 87,211 | 1,136,462 | 80,975 | 1,143,216 |
| case_036 | conv_shallow | 466,068 | 1,152 | 311,345 | 2,304 | 233,584 | 3,456 | 187,182 | 4,608 |
| case_037 | gemm_reduce | 105,374 | 204,800 | 74,038 | 573,440 | 53,320 | 614,400 | 43,013 | 942,080 |
| case_038 | mixed | 680,294 | 12,638,208 | 451,934 | 7,913,472 | 341,542 | 14,443,008 | 277,673 | 7,045,632 |
| case_039 | gemm_reduce | 708,430 | 9,166,848 | 478,013 | 11,911,168 | 368,961 | 14,536,704 | 315,610 | 9,457,664 |
| case_040 | mixed | 127,875 | 61,440 | 108,881 | 1,960,724 | 96,930 | 2,048,864 | 95,010 | 1,402,038 |
| case_041 | mixed | 2,976,418 | 36,857,856 | 1,985,376 | 35,656,704 | 1,492,661 | 36,307,968 | 1,186,283 | 52,371,456 |
| case_042 | mixed | 844,923 | 8,168,736 | 563,434 | 9,634,080 | 434,213 | 9,452,832 | 352,854 | 12,165,984 |
| case_043 | mixed | 349,546 | 894,746 | 267,860 | 419,598 | 247,606 | 5,315,576 | 228,246 | 5,417,590 |
| case_044 | conv_deep | 65,644 | 1,119,072 | 64,881 | 1,119,072 | 64,881 | 1,119,072 | 64,881 | 1,119,072 |
| case_045 | vector_elementwise | 63,059 | 0 | 42,601 | 0 | 31,878 | 0 | 26,164 | 0 |
| case_046 | conv_deep | 128,086 | 2,235,712 | 109,400 | 2,650,304 | 97,255 | 3,734,912 | 97,255 | 3,734,912 |
| case_047 | transformer | 231,041 | 1,700,082 | 190,135 | 2,946,734 | 176,064 | 3,138,264 | 165,994 | 3,355,138 |
| case_048 | transformer | 168,259 | 695,912 | 155,836 | 739,010 | 155,836 | 739,010 | 155,836 | 739,010 |
| case_049 | mixed | 142,498 | 1,234,894 | 121,810 | 1,268,250 | 113,277 | 2,268,944 | 110,507 | 2,310,466 |
| case_050 | mixed | 101,918 | 1,905,856 | 83,781 | 1,957,140 | 75,477 | 1,973,854 | 73,993 | 2,008,580 |
| case_051 | vector_chain | 357,955 | 9,045,258 | 291,220 | 9,045,308 | 254,211 | 9,045,350 | 253,855 | 9,045,396 |
| case_052 | mixed | 89,996 | 181,248 | 60,896 | 176,640 | 46,998 | 333,312 | 39,004 | 688,128 |
| case_053 | mixed | 754,655 | 8,602,230 | 485,118 | 10,844,388 | 468,293 | 7,051,908 | 406,490 | 7,873,010 |
| case_054 | mixed | 477,896 | 4,303,894 | 392,583 | 4,353,046 | 332,502 | 4,705,462 | 290,825 | 5,201,378 |
| case_055 | mixed | 53,209 | 115,712 | 41,193 | 259,684 | 31,297 | 206,340 | 25,973 | 284,696 |
| case_056 | mixed | 198,081 | 2,486,852 | 173,573 | 2,557,522 | 161,493 | 2,603,650 | 157,739 | 3,441,772 |
| case_057 | mixed | 30,430 | 229,152 | 21,045 | 14,112 | 16,655 | 55,872 | 14,055 | 14,112 |

| 用例 | 类型 | 2 核 Makespan | 2 核额外搬运 | 3 核 Makespan | 3 核额外搬运 | 4 核 Makespan | 4 核额外搬运 | 5 核 Makespan | 5 核额外搬运 |
|---|---|---|---|---|---|---|---|---|---|
| case_058 | gemm_reduce | 2,212,749 | 56,311,808 | 1,488,859 | 48,967,680 | 1,122,650 | 41,848,832 | 945,674 | 31,449,088 |
| case_059 | gemm_reduce | 369,318 | 360,448 | 254,073 | 1,171,456 | 185,216 | 1,081,344 | 150,782 | 2,072,576 |
| case_060 | mixed | 444,273 | 3,699,968 | 289,209 | 4,118,912 | 223,478 | 5,470,468 | 174,049 | 5,208,832 |
| case_061 | mixed | 39,299 | 34,560 | 28,409 | 43,008 | 23,716 | 119,040 | 22,283 | 83,712 |
| case_062 | mlp | 1,429,046 | 52,224 | 948,732 | 5,783,040 | 735,908 | 4,554,240 | 588,782 | 3,571,200 |
| case_063 | mlp | 536,738 | 36,864 | 358,989 | 36,864 | 274,327 | 1,704,960 | 222,074 | 99,840 |
| case_064 | transformer | 17,255 | 100,566 | 16,516 | 112,640 | 16,304 | 118,024 | 16,303 | 117,990 |
| case_065 | mixed | 26,928 | 13,968 | 18,165 | 27,792 | 14,276 | 127,896 | 11,648 | 69,408 |
| case_066 | mixed | 215,465 | 1,521,202 | 176,324 | 2,913,250 | 125,485 | 2,990,172 | 116,465 | 3,244,934 |
| case_067 | conv_deep | 30,692,166 | 22,405,248 | 20,633,772 | 26,582,656 | 15,560,141 | 29,848,448 | 12,794,176 | 34,132,992 |
| case_068 | mixed | 235,919 | 1,868,426 | 192,526 | 2,432,370 | 169,699 | 2,460,408 | 167,258 | 2,448,454 |
| case_069 | transformer | 14,068 | 273,738 | 11,913 | 312,236 | 11,913 | 312,236 | 11,648 | 360,624 |
| case_070 | mixed | 57,006 | 347,136 | 39,565 | 477,708 | 31,035 | 713,216 | 26,543 | 579,588 |
| case_071 | transformer | 11,198 | 189,834 | 10,279 | 246,764 | 9,325 | 249,838 | 9,287 | 250,606 |
| case_072 | mixed | 11,775,954 | 131,244,720 | 7,874,623 | 129,049,264 | 5,983,296 | 119,253,792 | 4,796,788 | 117,301,936 |
| case_073 | conv_deep | 4,409,246 | 6,219,136 | 2,952,283 | 10,070,528 | 2,313,552 | 13,737,984 | 1,835,693 | 17,689,344 |
| case_074 | mixed | 820,258 | 7,557,120 | 543,217 | 9,863,168 | 412,681 | 6,721,536 | 339,207 | 5,160,960 |
| case_075 | transformer | 913,136 | 12,112,182 | 754,460 | 13,211,828 | 574,605 | 13,667,890 | 531,340 | 14,815,408 |
| case_076 | mixed | 3,069,464 | 30,744,576 | 2,034,405 | 43,087,872 | 1,538,338 | 35,345,408 | 1,226,738 | 40,754,176 |
| case_077 | mixed | 250,735 | 57,344 | 168,802 | 573,440 | 125,726 | 1,018,046 | 100,187 | 1,132,734 |
| case_078 | conv_deep | 1,119,724 | 3,967,104 | 827,478 | 8,591,616 | 590,696 | 15,186,816 | 581,428 | 15,223,680 |
| case_079 | gemm_reduce | 15,036,377 | 28,344,320 | 10,000,179 | 46,739,456 | 7,428,384 | 37,806,080 | 5,965,867 | 61,992,960 |
| case_080 | gemm_reduce | 145,825 | 1,675,264 | 100,325 | 3,047,424 | 91,774 | 2,666,496 | 89,345 | 3,047,424 |
| case_081 | vector_norm | 76,860 | 0 | 51,943 | 0 | 39,588 | 0 | 30,943 | 0 |
| case_082 | transformer | 461,058 | 6,490,562 | 325,356 | 6,815,414 | 286,347 | 7,149,994 | 250,605 | 7,721,436 |
| case_083 | conv_deep | 454,902 | 5,415,232 | 335,002 | 4,974,784 | 278,428 | 5,790,496 | 237,528 | 6,182,144 |
| case_084 | attention | 856,125 | 17,316,288 | 836,168 | 3,308,544 | 626,804 | 2,488,320 | 503,420 | 1,990,656 |
| case_085 | transformer | 1,835,584 | 33,262,698 | 1,398,716 | 35,362,242 | 1,142,514 | 35,093,242 | 1,064,352 | 38,380,598 |

| 用例 | 类型 | 2 核 Makespan | 2 核额外搬运 | 3 核 Makespan | 3 核额外搬运 | 4 核 Makespan | 4 核额外搬运 | 5 核 Makespan | 5 核额外搬运 |
|---|---|---|---|---|---|---|---|---|---|
| case_086 | transformer | 85,576 | 1,450,230 | 77,096 | 1,570,142 | 64,201 | 1,712,780 | 64,201 | 1,712,780 |
| case_087 | mixed | 1,980,492 | 11,013,120 | 1,375,796 | 16,405,084 | 1,052,173 | 11,704,324 | 869,941 | 20,092,008 |
| case_088 | mixed | 153,499 | 1,199,304 | 125,201 | 1,748,022 | 114,587 | 1,870,884 | 114,587 | 1,870,884 |
| case_089 | conv_deep | 346,763 | 641,280 | 243,214 | 1,158,528 | 178,160 | 716,160 | 147,432 | 954,880 |
| case_090 | conv_deep | 516,664 | 2,965,248 | 383,414 | 5,128,704 | 345,965 | 8,185,536 | 323,909 | 9,431,424 |
| case_091 | mixed | 4,541,346 | 79,045,120 | 3,023,255 | 75,340,288 | 2,286,848 | 87,420,928 | 1,866,311 | 72,501,760 |
| case_092 | conv_deep | 2,067,931 | 12,648,768 | 1,416,324 | 13,582,976 | 1,062,760 | 13,578,912 | 875,605 | 14,264,128 |
| case_093 | gemm_shallow | 37,827 | 8,192 | 25,103 | 16,384 | 19,779 | 24,576 | 15,876 | 32,768 |
| case_094 | mixed | 48,941 | 595,584 | 32,228 | 212,544 | 25,010 | 351,072 | 20,775 | 379,008 |
| case_095 | attention | 1,042,787 | 8,515,584 | 695,672 | 5,677,056 | 524,319 | 4,276,224 | 420,749 | 3,428,352 |
| case_096 | mixed | 42,941 | 534,620 | 31,115 | 159,576 | 24,595 | 135,708 | 23,762 | 135,708 |
| case_097 | gemm_reduce | 5,187,373 | 16,973,824 | 3,557,146 | 21,991,424 | 2,579,804 | 11,927,552 | 2,102,348 | 21,311,488 |
| case_098 | mixed | 89,551 | 0 | 67,033 | 1,792,434 | 56,857 | 1,792,440 | 51,939 | 1,792,444 |
| case_099 | mixed | 805,594 | 10,559,488 | 536,004 | 16,158,720 | 405,735 | 10,559,488 | 337,221 | 10,698,752 |
| case_100 | mixed | 79,057 | 466,576 | 57,811 | 72,192 | 51,786 | 301,584 | 51,786 | 301,584 |

# 附录 B 问题二逐用例结果

| 用例 | 类型 | 2 核 Makespan | 2 核额外搬运 | 3 核 Makespan | 3 核额外搬运 | 4 核 Makespan | 4 核额外搬运 | 5 核 Makespan | 5 核额外搬运 |
|---|---|---|---|---|---|---|---|---|---|
| case_001 | conv_shallow | 116,868 | 1,152 | 78,545 | 2,304 | 58,984 | 3,456 | 47,502 | 4,608 |
| case_002 | mlp | 125,995 | 1,211,904 | 84,116 | 38,400 | 65,838 | 1,171,968 | 54,349 | 279,552 |
| case_003 | mixed | 322,851 | 3,744,570 | 225,381 | 4,524,668 | 214,163 | 6,933,694 | 199,181 | 7,911,352 |
| case_004 | gated | 194,802 | 1,029,120 | 131,412 | 847,872 | 100,074 | 1,889,280 | 81,572 | 903,168 |
| case_005 | transformer | 54,698 | 1,071,590 | 46,664 | 959,340 | 43,305 | 1,309,712 | 41,480 | 1,309,712 |
| case_006 | mixed | 38,382 | 958,818 | 29,216 | 1,034,008 | 25,022 | 665,920 | 25,022 | 665,920 |
| case_007 | gated | 490,226 | 2,950,656 | 326,994 | 2,709,504 | 246,006 | 2,519,040 | 197,052 | 2,260,992 |
| case_008 | attention | 136,244 | 3,538,944 | 109,887 | 3,354,624 | 76,887 | 701,184 | 61,531 | 36,864 |
| case_009 | mixed | 87,215 | 515,782 | 81,400 | 778,402 | 64,446 | 773,414 | 54,054 | 715,642 |
| case_010 | mixed | 43,470 | 28,928 | 29,794 | 59,648 | 25,038 | 244,240 | 22,042 | 250,000 |
| case_011 | gemm_reduce | 92,646 | 180,224 | 62,905 | 2,119,680 | 46,880 | 540,672 | 42,013 | 1,320,960 |
| case_012 | mixed | 26,289 | 28,800 | 18,095 | 20,160 | 14,873 | 31,680 | 12,296 | 61,760 |
| case_013 | gemm_shallow | 132,198 | 8,192 | 88,351 | 16,384 | 66,982 | 24,576 | 54,363 | 32,768 |
| case_014 | mixed | 8,859,399 | 98,550,552 | 5,965,005 | 116,750,520 | 4,526,660 | 82,469,784 | 3,680,709 | 108,064,368 |
| case_015 | mixed | 80,126 | 232,756 | 53,135 | 437,504 | 43,894 | 247,552 | 39,265 | 329,728 |
| case_016 | vector_chain | 4,033,396 | 8,536 | 2,876,681 | 12,192 | 2,240,279 | 14,628 | 2,119,426 | 736,740 |
| case_017 | mixed | 64,826 | 87,552 | 44,682 | 175,104 | 35,117 | 259,200 | 29,598 | 339,888 |
| case_018 | mixed | 500,884 | 2,589,696 | 335,341 | 4,235,716 | 256,599 | 3,484,032 | 202,112 | 3,144,704 |
| case_019 | mixed | 30,917 | 27,168 | 22,376 | 24,576 | 15,199 | 84,512 | 15,199 | 84,512 |
| case_020 | vector_elementwise | 266,580 | 0 | 177,950 | 0 | 133,704 | 0 | 107,491 | 0 |
| case_021 | gemm_reduce | 3,135,518 | 4,165,632 | 2,115,499 | 9,654,272 | 1,550,324 | 4,608,000 | 1,250,168 | 11,431,936 |
| case_022 | mixed | 27,423 | 169,728 | 18,524 | 82,432 | 14,247 | 20,756 | 11,427 | 20,744 |
| case_023 | mixed | 50,145 | 149,920 | 33,474 | 213,280 | 25,299 | 281,280 | 22,149 | 281,568 |
| case_024 | vector_chain | 1,336,312 | 2,824 | 953,369 | 4,032 | 743,123 | 4,836 | 704,958 | 726,132 |
| case_025 | gated | 2,431,840 | 27,648 | 1,570,292 | 20,493,312 | 1,217,444 | 82,944 | 975,282 | 110,592 |
| case_026 | mixed | 46,418 | 106,496 | 36,849 | 193,992 | 36,849 | 193,992 | 36,849 | 193,992 |
| case_027 | gemm_reduce | 1,115,722 | 1,105,920 | 786,799 | 10,493,952 | 559,196 | 3,317,760 | 487,970 | 12,009,472 |
| case_028 | conv_deep | 20,281,968 | 18,098,376 | 13,505,429 | 10,563,840 | 10,145,012 | 72,038,784 | 8,141,472 | 16,939,008 |
| case_029 | vector_norm | 18,880 | 16,388 | 12,743 | 0 | 10,108 | 98,316 | 8,523 | 65,552 |

| 用例 | 类型 | 2 核 Makespan | 2 核额外搬运 | 3 核 Makespan | 3 核额外搬运 | 4 核 Makespan | 4 核额外搬运 | 5 核 Makespan | 5 核额外搬运 |
|---|---|---|---|---|---|---|---|---|---|
| case_030 | mixed | 1,399,895 | 4,944,896 | 933,969 | 9,071,616 | 699,591 | 8,632,320 | 561,192 | 11,644,416 |
| case_031 | mixed | 734,858 | 5,538,048 | 485,297 | 8,636,928 | 379,245 | 7,619,076 | 302,186 | 8,669,798 |
| case_032 | mixed | 17,455 | 6,216 | 11,785 | 12,360 | 9,770 | 6,224 | 9,770 | 6,224 |
| case_033 | mixed | 894,969 | 6,564,896 | 600,036 | 8,029,772 | 457,037 | 7,784,052 | 359,101 | 8,699,624 |
| case_034 | mixed | 407,272 | 2,354,688 | 270,444 | 2,757,632 | 206,613 | 2,970,112 | 171,615 | 3,386,368 |
| case_035 | mixed | 88,807 | 560,834 | 88,807 | 560,834 | 70,052 | 707,342 | 62,022 | 886,768 |
| case_036 | conv_shallow | 466,068 | 1,152 | 311,345 | 2,304 | 233,584 | 3,456 | 187,182 | 4,608 |
| case_037 | gemm_reduce | 105,374 | 204,800 | 72,587 | 811,008 | 53,320 | 614,400 | 43,013 | 942,080 |
| case_038 | mixed | 678,434 | 4,640,472 | 461,131 | 6,682,624 | 345,258 | 7,867,396 | 277,673 | 7,045,632 |
| case_039 | gemm_reduce | 700,857 | 10,575,872 | 478,013 | 11,911,168 | 368,961 | 14,536,704 | 322,273 | 9,183,232 |
| case_040 | mixed | 143,596 | 16,384 | 110,105 | 0 | 110,105 | 0 | 95,983 | 2,296,140 |
| case_041 | mixed | 3,002,310 | 25,144,332 | 2,073,353 | 24,876,732 | 1,573,048 | 27,927,352 | 1,268,970 | 35,681,128 |
| case_042 | mixed | 851,589 | 4,673,952 | 567,444 | 8,388,000 | 434,097 | 5,737,900 | 352,825 | 4,968,588 |
| case_043 | mixed | 353,177 | 385,024 | 270,609 | 7,983,996 | 242,196 | 9,007,746 | 207,184 | 4,469,328 |
| case_044 | conv_deep | 43,931 | 930,400 | 43,931 | 930,400 | 43,931 | 930,400 | 43,931 | 930,400 |
| case_045 | vector_elementwise | 63,059 | 0 | 42,601 | 0 | 31,871 | 0 | 26,164 | 0 |
| case_046 | conv_deep | 106,042 | 930,400 | 79,799 | 1,958,816 | 72,735 | 1,777,440 | 68,235 | 1,777,440 |
| case_047 | transformer | 168,403 | 1,329,830 | 138,089 | 1,888,620 | 118,704 | 2,994,866 | 108,433 | 2,994,866 |
| case_048 | transformer | 122,559 | 643,206 | 120,412 | 1,006,540 | 109,466 | 952,050 | 109,104 | 952,050 |
| case_049 | mixed | 103,661 | 1,148,594 | 81,601 | 1,897,672 | 67,339 | 2,053,998 | 64,642 | 2,053,998 |
| case_050 | mixed | 84,980 | 1,090,954 | 69,415 | 965,324 | 60,275 | 1,031,506 | 50,952 | 1,130,318 |
| case_051 | vector_chain | 318,295 | 668 | 227,413 | 952 | 178,014 | 1,140 | 171,053 | 722,128 |
| case_052 | mixed | 82,891 | 102,912 | 55,349 | 176,640 | 40,776 | 259,584 | 33,602 | 337,920 |
| case_053 | mixed | 687,143 | 5,498,458 | 475,495 | 8,589,376 | 411,764 | 12,151,738 | 371,299 | 6,756,898 |
| case_054 | mixed | 428,392 | 5,672,224 | 307,649 | 6,689,528 | 251,957 | 8,404,422 | 241,383 | 9,857,828 |
| case_055 | mixed | 52,306 | 81,920 | 36,063 | 138,240 | 28,910 | 94,776 | 22,075 | 148,480 |
| case_056 | mixed | 153,163 | 1,326,436 | 102,605 | 1,408,748 | 98,782 | 2,226,228 | 98,782 | 2,226,228 |
| case_057 | mixed | 30,170 | 20,640 | 20,705 | 14,112 | 16,485 | 27,936 | 13,794 | 14,160 |
| case_058 | gemm_reduce | 2,232,027 | 62,148,608 | 1,512,607 | 32,886,784 | 1,153,881 | 43,593,728 | 937,387 | 33,390,592 |
| case_059 | gemm_reduce | 369,318 | 360,448 | 253,759 | 1,179,648 | 185,216 | 1,081,344 | 150,736 | 2,072,576 |
| case_060 | mixed | 443,355 | 3,815,44 | 304,353 | 5,207,604 | 229,340 | 3,129,600 | 175,267 | 3,748,366 |

| 用例 | 类型 | 2 核 Makespan | 2 核额外搬运 | 3 核 Makespan | 3 核额外搬运 | 4 核 Makespan | 4 核额外搬运 | 5 核 Makespan | 5 核额外搬运 |
|---|---|---|---|---|---|---|---|---|---|
| | | | 4 | | | | | | |
| case_061 | mixed | 39,284 | 34,816 | 25,922 | 62,208 | 20,677 | 71,680 | 18,175 | 89,600 |
| case_062 | mlp | 1,428,520 | 21,504 | 964,751 | 93,696 | 755,574 | 102,912 | 588,261 | 92,160 |
| case_063 | mlp | 536,212 | 15,360 | 358,175 | 1,336,320 | 275,334 | 78,336 | 221,953 | 81,408 |
| case_064 | transformer | 12,342 | 56,870 | 11,914 | 69,676 | 11,880 | 69,676 | 11,854 | 69,676 |
| case_065 | mixed | 21,026 | 135,092 | 14,406 | 27,792 | 11,597 | 50,832 | 9,851 | 64,800 |
| case_066 | mixed | 190,070 | 2,322,136 | 140,906 | 2,939,652 | 130,902 | 4,042,596 | 122,042 | 4,383,568 |
| case_067 | conv_deep | 30,487,075 | 9,431,424 | 20,375,398 | 17,584,896 | 15,320,907 | 16,931,968 | 12,698,256 | 17,622,528 |
| case_068 | mixed | 216,658 | 2,858,612 | 173,252 | 3,646,208 | 147,211 | 2,072,228 | 144,463 | 2,057,772 |
| case_069 | transformer | 13,130 | 176,806 | 9,477 | 269,960 | 9,477 | 269,960 | 9,477 | 269,960 |
| case_070 | mixed | 54,121 | 129,536 | 40,413 | 259,072 | 30,291 | 333,312 | 27,664 | 462,848 |
| case_071 | transformer | 10,195 | 118,438 | 10,129 | 141,478 | 8,017 | 219,850 | 7,346 | 254,828 |
| case_072 | mixed | 11,814,263 | 124,530,392 | 7,945,629 | 114,002,720 | 5,989,294 | 122,293,792 | 4,863,110 | 121,488,048 |
| case_073 | conv_deep | 4,240,707 | 3,721,600 | 2,863,798 | 7,443,200 | 2,231,456 | 11,164,800 | 1,830,446 | 14,978,560 |
| case_074 | mixed | 822,510 | 7,077,888 | 553,814 | 7,614,464 | 414,482 | 5,939,200 | 332,704 | 4,997,134 |
| case_075 | transformer | 649,943 | 2,703,046 | 607,622 | 7,307,148 | 429,372 | 5,924,946 | 390,370 | 10,320,376 |
| case_076 | mixed | 3,069,464 | 30,744,576 | 2,061,609 | 35,117,056 | 1,550,155 | 35,703,808 | 1,238,120 | 37,194,752 |
| case_077 | mixed | 250,735 | 57,344 | 169,583 | 131,072 | 133,327 | 663,552 | 107,273 | 196,608 |
| case_078 | conv_deep | 1,024,634 | 2,148,480 | 718,901 | 6,751,488 | 520,115 | 6,445,440 | 519,288 | 6,654,984 |
| case_079 | gemm_reduce | 14,880,881 | 30,818,304 | 9,990,705 | 57,745,408 | 7,624,447 | 30,470,144 | 6,075,976 | 47,677,440 |
| case_080 | gemm_reduce | 145,224 | 1,650,688 | 98,625 | 761,856 | 81,962 | 1,773,568 | 77,726 | 2,650,112 |
| case_081 | vector_norm | 76,316 | 24,620 | 51,943 | 0 | 39,588 | 0 | 30,943 | 0 |
| case_082 | transformer | 339,752 | 1,261,606 | 247,188 | 2,553,228 | 205,097 | 3,223,250 | 188,738 | 5,376,182 |
| case_083 | conv_deep | 428,195 | 1,557,088 | 288,936 | 5,294,272 | 218,574 | 2,856,736 | 185,285 | 3,730,816 |
| case_084 | attention | 686,336 | 9,437,184 | 458,172 | 9,179,136 | 384,148 | 8,957,952 | 374,547 | 8,709,120 |
| case_085 | transformer | 1,649,767 | 11,636,202 | 1,160,320 | 18,491,276 | 838,246 | 10,155,890 | 782,278 | 25,638,488 |
| case_086 | transformer | 61,298 | 917,286 | 58,737 | 917,286 | 48,137 | 1,704,434 | 48,137 | 1,704,434 |
| case_087 | mixed | 1,985,234 | 9,999,360 | 1,385,253 | 10,722,816 | 1,069,661 | 11,796,480 | 888,504 | 10,460,160 |
| case_088 | mixed | 140,104 | 2,024,234 | 115,725 | 1,341,964 | 103,304 | 1,634,742 | 103,304 | 1,634,742 |
| case_089 | conv_deep | 299,270 | 238,720 | 209,009 | 1,215,232 | 152,132 | 716,160 | 123,062 | 954,880 |
| case_090 | conv_deep | 449,122 | 2,166,144 | 324,455 | 4,622,592 | 282,572 | 7,507,584 | 251,942 | 6,471,288 |

| 用例 | 类型 | 2 核 Makespan | 2 核额外搬运 | 3 核 Makespan | 3 核额外搬运 | 4 核 Makespan | 4 核额外搬运 | 5 核 Makespan | 5 核额外搬运 |
|---|---|---|---|---|---|---|---|---|---|
| case_091 | mixed | 4,590,046 | 57,827,296 | 3,070,809 | 63,099,200 | 2,315,529 | 67,391,136 | 1,874,503 | 71,279,104 |
| case_092 | conv_deep | 1,999,168 | 4,119,136 | 1,346,879 | 4,543,680 | 1,014,119 | 6,722,336 | 824,147 | 3,942,784 |
| case_093 | gemm_shallow | 37,739 | 12,288 | 25,103 | 16,384 | 19,779 | 24,576 | 15,851 | 36,864 |
| case_094 | mixed | 41,271 | 153,128 | 30,040 | 226,880 | 23,012 | 307,008 | 19,355 | 258,912 |
| case_095 | attention | 573,788 | 16,588,800 | 474,153 | 16,404,480 | 467,555 | 16,220,160 | 420,017 | 405,504 |
| case_096 | mixed | 38,859 | 86,272 | 27,899 | 67,392 | 23,399 | 123,440 | 22,936 | 215,476 |
| case_097 | gemm_reduce | 5,562,361 | 11,821,056 | 3,542,953 | 6,176,768 | 2,624,122 | 14,241,792 | 2,094,984 | 12,140,544 |
| case_098 | mixed | 89,333 | 2,006,858 | 72,826 | 131,120 | 62,007 | 1,328,302 | 54,448 | 1,555,310 |
| case_099 | mixed | 804,687 | 5,858,048 | 536,168 | 4,625,152 | 401,809 | 5,733,888 | 323,728 | 7,618,560 |
| case_100 | mixed | 72,760 | 104,960 | 51,491 | 203,272 | 51,491 | 203,272 | 51,491 | 203,272 |

# 附录 C 问题三无 L2 与只读 L2 Cache 逐用例结果

| 用例 | 类型 | 核数 | 无 L2 Makespan | 无 L2 额外搬运/B | 只读 Cache Makespan | 只读 Cache 额外搬运/B | Cache 命中率 |
|---|---|---|---|---|---|---|---|
| case_001 | conv_shallow | 1 | 233,110 | 0 | 233,110 | 0 | 0.00% |
| case_001 | conv_shallow | 2 | 116,868 | 1,152 | 116,868 | 1,152 | 0.00% |
| case_001 | conv_shallow | 3 | 78,545 | 2,304 | 78,545 | 2,304 | 0.00% |
| case_001 | conv_shallow | 4 | 58,984 | 3,456 | 58,984 | 3,456 | 0.00% |
| case_001 | conv_shallow | 5 | 47,502 | 4,608 | 47,502 | 4,608 | 0.00% |
| case_002 | mlp | 1 | 248,877 | 0 | 248,877 | 0 | 0.00% |
| case_002 | mlp | 2 | 126,394 | 1,216,512 | 126,034 | 1,216,512 | 25.48% |
| case_002 | mlp | 3 | 83,878 | 1,044,480 | 83,878 | 1,044,480 | 29.46% |
| case_002 | mlp | 4 | 65,583 | 1,267,200 | 65,462 | 1,267,200 | 28.49% |
| case_002 | mlp | 5 | 53,669 | 279,552 | 53,669 | 279,552 | 9.63% |
| case_003 | mixed | 1 | 543,734 | 44,068 | 543,734 | 44,068 | 0.00% |
| case_003 | mixed | 2 | 315,085 | 3,230,462 | 314,660 | 3,230,462 | 10.87% |
| case_003 | mixed | 3 | 222,760 | 4,561,464 | 220,843 | 4,561,464 | 14.02% |
| case_003 | mixed | 4 | 202,122 | 6,366,986 | 197,111 | 6,366,986 | 21.75% |
| case_003 | mixed | 5 | 190,232 | 7,518,340 | 176,702 | 7,518,340 | 24.39% |
| case_004 | gated | 1 | 386,363 | 307,200 | 386,363 | 307,200 | 0.00% |
| case_004 | gated | 2 | 194,802 | 1,029,120 | 194,802 | 1,029,120 | 4.49% |
| case_004 | gated | 3 | 132,672 | 1,299,456 | 130,769 | 1,299,456 | 23.57% |
| case_004 | gated | 4 | 100,429 | 1,480,704 | 100,022 | 1,480,704 | 9.44% |
| case_004 | gated | 5 | 81,107 | 866,304 | 81,107 | 866,304 | 2.74% |
| case_005 | transformer | 1 | 84,089 | 0 | 84,089 | 0 | 0.00% |
| case_005 | transformer | 2 | 48,802 | 1,090,790 | 48,541 | 1,090,790 | 9.07% |
| case_005 | transformer | 3 | 48,802 | 1,090,790 | 48,541 | 1,090,790 | 9.07% |
| case_005 | transformer | 4 | 54,379 | 2,450,258 | 48,473 | 2,450,258 | 30.41% |
| case_005 | transformer | 5 | 54,379 | 2,450,258 | 48,473 | 2,450,258 | 30.41% |
| case_006 | mixed | 1 | 70,315 | 881,742 | 70,293 | 881,742 | 10.17% |
| case_006 | mixed | 2 | 37,955 | 445,856 | 37,955 | 445,856 | 9.95% |
| case_006 | mixed | 3 | 30,082 | 1,144,498 | 27,084 | 1,144,498 | 28.65% |
| case_006 | mixed | 4 | 27,927 | 973,842 | 24,320 | 973,842 | 21.38% |
| case_006 | mixed | 5 | 21,889 | 675,436 | 20,527 | 675,436 | 19.92% |
| case_007 | gated | 1 | 979,808 | 5,210,112 | 979,808 | 5,210,112 | 21.94% |
| case_007 | gated | 2 | 490,226 | 2,950,656 | 490,226 | 2,950,656 | 0.39% |
| case_007 | gated | 3 | 326,994 | 2,703,360 | 326,994 | 2,703,360 | 0.00% |
| case_007 | gated | 4 | 245,978 | 2,525,184 | 245,978 | 2,525,184 | 1.33% |
| case_007 | gated | 5 | 197,052 | 2,260,992 | 196,870 | 2,260,992 | 2.23% |
| case_008 | attention | 1 | 269,325 | 3,760,128 | 269,325 | 3,760,128 | 0.00% |
| case_008 | attention | 2 | 136,146 | 3,538,944 | 136,146 | 3,538,944 | 0.00% |
| case_008 | attention | 3 | 97,687 | 1,364,736 | 97,687 | 1,364,736 | 0.00% |
| case_008 | attention | 4 | 79,422 | 552,960 | 79,422 | 552,960 | 0.00% |
| case_008 | attention | 5 | 60,991 | 36,864 | 60,991 | 36,864 | 0.00% |
| case_009 | mixed | 1 | 136,781 | 232,450 | 136,505 | 232,450 | 16.87% |
| case_009 | mixed | 2 | 83,397 | 570,206 | 83,395 | 570,206 | 10.61% |
| case_009 | mixed | 3 | 66,245 | 802,036 | 66,225 | 802,036 | 13.24% |

| 用例 | 类型 | 核数 | 无 L2 Makespan | 无 L2 额外搬运/B | 只读 Cache Makespan | 只读 Cache 额外搬运/B | Cache 命中率 |
|---|---|---|---|---|---|---|---|
| case_009 | mixed | 4 | 62,252 | 705,454 | 62,240 | 705,454 | 21.58% |
| case_009 | mixed | 5 | 52,738 | 683,910 | 52,630 | 683,910 | 18.75% |
| case_010 | mixed | 1 | 71,075 | 450,664 | 71,005 | 450,664 | 0.67% |
| case_010 | mixed | 2 | 41,124 | 413,980 | 41,124 | 413,980 | 2.87% |
| case_010 | mixed | 3 | 29,964 | 441,600 | 29,672 | 441,600 | 6.23% |
| case_010 | mixed | 4 | 23,831 | 284,576 | 23,392 | 284,576 | 32.51% |
| case_010 | mixed | 5 | 21,730 | 341,400 | 20,689 | 341,400 | 44.77% |
| case_011 | gemm_reduce | 1 | 184,697 | 0 | 184,697 | 0 | 0.00% |
| case_011 | gemm_reduce | 2 | 92,646 | 180,224 | 92,646 | 180,224 | 0.00% |
| case_011 | gemm_reduce | 3 | 63,828 | 2,129,920 | 63,828 | 2,129,920 | 48.47% |
| case_011 | gemm_reduce | 4 | 46,880 | 540,672 | 46,880 | 540,672 | 0.00% |
| case_011 | gemm_reduce | 5 | 44,975 | 2,054,144 | 41,763 | 2,054,144 | 54.88% |
| case_012 | mixed | 1 | 46,935 | 0 | 46,935 | 0 | 0.00% |
| case_012 | mixed | 2 | 26,289 | 28,800 | 26,289 | 28,800 | 5.62% |
| case_012 | mixed | 3 | 18,825 | 20,160 | 18,825 | 20,160 | 4.00% |
| case_012 | mixed | 4 | 15,675 | 159,004 | 15,450 | 159,004 | 9.97% |
| case_012 | mixed | 5 | 12,206 | 71,616 | 12,129 | 71,616 | 10.80% |
| case_013 | gemm_shallow | 1 | 263,949 | 0 | 263,949 | 0 | 0.00% |
| case_013 | gemm_shallow | 2 | 132,198 | 8,192 | 132,198 | 8,192 | 0.00% |
| case_013 | gemm_shallow | 3 | 88,351 | 16,384 | 88,351 | 16,384 | 0.00% |
| case_013 | gemm_shallow | 4 | 66,982 | 24,576 | 66,946 | 24,576 | 0.90% |
| case_013 | gemm_shallow | 5 | 54,363 | 32,768 | 54,117 | 32,768 | 1.20% |
| case_014 | mixed | 1 | 17,497,996 | 83,349,144 | 17,486,682 | 83,349,144 | 4.29% |
| case_014 | mixed | 2 | 8,856,434 | 64,759,200 | 8,809,357 | 64,759,200 | 27.67% |
| case_014 | mixed | 3 | 5,944,331 | 73,890,948 | 5,891,208 | 73,890,948 | 33.48% |
| case_014 | mixed | 4 | 4,474,149 | 80,649,492 | 4,443,382 | 80,649,492 | 32.00% |
| case_014 | mixed | 5 | 3,632,919 | 76,579,160 | 3,586,894 | 76,579,160 | 35.77% |
| case_015 | mixed | 1 | 145,126 | 327,776 | 145,126 | 327,776 | 27.03% |
| case_015 | mixed | 2 | 82,491 | 740,894 | 82,354 | 740,894 | 15.53% |
| case_015 | mixed | 3 | 54,126 | 634,112 | 51,948 | 634,112 | 15.94% |
| case_015 | mixed | 4 | 45,634 | 231,168 | 45,634 | 231,168 | 8.97% |
| case_015 | mixed | 5 | 40,489 | 806,416 | 36,382 | 806,416 | 36.65% |
| case_016 | vector_chain | 1 | 7,715,523 | 0 | 7,715,523 | 0 | 0.00% |
| case_016 | vector_chain | 2 | 4,033,396 | 8,536 | 4,033,396 | 8,536 | 0.00% |
| case_016 | vector_chain | 3 | 2,876,681 | 12,192 | 2,876,681 | 12,192 | 0.15% |
| case_016 | vector_chain | 4 | 2,240,279 | 14,628 | 2,240,279 | 14,628 | 0.30% |
| case_016 | vector_chain | 5 | 2,117,853 | 15,844 | 2,117,853 | 15,844 | 0.45% |
| case_017 | mixed | 1 | 126,440 | 88,144 | 126,414 | 88,144 | 14.13% |
| case_017 | mixed | 2 | 64,328 | 87,552 | 64,307 | 87,552 | 2.22% |
| case_017 | mixed | 3 | 44,777 | 175,104 | 44,722 | 175,104 | 19.25% |
| case_017 | mixed | 4 | 34,824 | 260,352 | 34,824 | 260,352 | 23.46% |
| case_017 | mixed | 5 | 29,598 | 339,888 | 28,733 | 339,888 | 24.80% |
| case_018 | mixed | 1 | 990,694 | 907,264 | 990,642 | 907,264 | 21.92% |
| case_018 | mixed | 2 | 499,642 | 2,589,696 | 499,506 | 2,589,696 | 49.78% |
| case_018 | mixed | 3 | 333,117 | 4,235,716 | 332,883 | 4,235,716 | 56.42% |

| 用例 | 类型 | 核数 | 无 L2 Makespan | 无 L2 额外搬运/B | 只读 Cache Makespan | 只读 Cache 额外搬运/B | Cache 命中率 |
|---|---|---|---|---|---|---|---|
| case_018 | mixed | 4 | 256,599 | 3,484,032 | 256,495 | 3,484,032 | 57.92% |
| case_018 | mixed | 5 | 200,867 | 3,563,556 | 200,711 | 3,563,556 | 66.49% |
| case_019 | mixed | 1 | 60,447 | 208,896 | 60,447 | 208,896 | 10.27% |
| case_019 | mixed | 2 | 29,329 | 27,168 | 29,329 | 27,168 | 9.19% |
| case_019 | mixed | 3 | 22,304 | 24,576 | 22,304 | 24,576 | 8.04% |
| case_019 | mixed | 4 | 22,268 | 24,576 | 22,268 | 24,576 | 8.04% |
| case_019 | mixed | 5 | 15,088 | 135,820 | 15,088 | 135,820 | 12.75% |
| case_020 | vector_elementwise | 1 | 532,746 | 0 | 532,746 | 0 | 0.00% |
| case_020 | vector_elementwise | 2 | 266,580 | 0 | 266,580 | 0 | 0.00% |
| case_020 | vector_elementwise | 3 | 177,950 | 0 | 177,950 | 0 | 0.00% |
| case_020 | vector_elementwise | 4 | 133,704 | 0 | 133,704 | 0 | 0.00% |
| case_020 | vector_elementwise | 5 | 107,491 | 0 | 107,491 | 0 | 0.00% |
| case_021 | gemm_reduce | 1 | 6,259,983 | 13,959,168 | 6,259,983 | 13,959,168 | 4.77% |
| case_021 | gemm_reduce | 2 | 3,134,908 | 4,165,632 | 3,134,908 | 4,165,632 | 0.00% |
| case_021 | gemm_reduce | 3 | 2,117,033 | 9,629,696 | 2,114,147 | 9,629,696 | 55.49% |
| case_021 | gemm_reduce | 4 | 1,550,324 | 4,608,000 | 1,550,324 | 4,608,000 | 0.00% |
| case_021 | gemm_reduce | 5 | 1,249,494 | 10,203,136 | 1,247,828 | 10,203,136 | 50.31% |
| case_022 | mixed | 1 | 54,123 | 0 | 54,123 | 0 | 0.00% |
| case_022 | mixed | 2 | 27,499 | 6,924 | 27,499 | 6,924 | 3.14% |
| case_022 | mixed | 3 | 18,347 | 13,824 | 18,347 | 13,824 | 6.08% |
| case_022 | mixed | 4 | 13,801 | 13,824 | 13,801 | 13,824 | 6.08% |
| case_022 | mixed | 5 | 11,427 | 13,836 | 11,427 | 13,836 | 4.05% |
| case_023 | mixed | 1 | 90,478 | 0 | 90,478 | 0 | 0.00% |
| case_023 | mixed | 2 | 47,606 | 194,976 | 47,606 | 194,976 | 8.63% |
| case_023 | mixed | 3 | 33,474 | 213,280 | 33,474 | 213,280 | 17.99% |
| case_023 | mixed | 4 | 24,632 | 281,280 | 24,275 | 281,280 | 11.84% |
| case_023 | mixed | 5 | 20,006 | 197,184 | 19,990 | 197,184 | 5.28% |
| case_024 | vector_chain | 1 | 2,555,343 | 0 | 2,555,343 | 0 | 0.00% |
| case_024 | vector_chain | 2 | 1,336,312 | 2,824 | 1,336,312 | 2,824 | 0.00% |
| case_024 | vector_chain | 3 | 953,369 | 4,032 | 953,369 | 4,032 | 0.05% |
| case_024 | vector_chain | 4 | 743,123 | 4,836 | 743,123 | 4,836 | 0.10% |
| case_024 | vector_chain | 5 | 703,521 | 5,236 | 703,521 | 5,236 | 0.15% |
| case_025 | gated | 1 | 4,863,026 | 0 | 4,863,026 | 0 | 0.00% |
| case_025 | gated | 2 | 2,431,840 | 27,648 | 2,431,840 | 27,648 | 0.00% |
| case_025 | gated | 3 | 1,622,164 | 55,296 | 1,622,164 | 55,296 | 0.00% |
| case_025 | gated | 4 | 1,217,444 | 82,944 | 1,217,444 | 82,944 | 0.00% |
| case_025 | gated | 5 | 975,282 | 110,592 | 975,282 | 110,592 | 0.00% |
| case_026 | mixed | 1 | 89,196 | 245,760 | 89,092 | 245,760 | 2.64% |
| case_026 | mixed | 2 | 46,007 | 98,304 | 46,007 | 98,304 | 0.00% |
| case_026 | mixed | 3 | 37,294 | 199,740 | 37,294 | 199,740 | 28.43% |
| case_026 | mixed | 4 | 37,294 | 199,740 | 37,294 | 199,740 | 28.43% |
| case_026 | mixed | 5 | 36,898 | 555,984 | 36,888 | 555,984 | 32.64% |
| case_027 | gemm_reduce | 1 | 2,229,905 | 0 | 2,229,905 | 0 | 0.00% |
| case_027 | gemm_reduce | 2 | 1,115,722 | 737,280 | 1,115,722 | 737,280 | 0.00% |
| case_027 | gemm_reduce | 3 | 773,119 | 14,856,192 | 772,439 | 14,856,192 | 23.30% |

| 用例 | 类型 | 核数 | 无 L2 Makespan | 无 L2 额外搬运/B | 只读 Cache Makespan | 只读 Cache 额外搬运/B | Cache 命中率 |
|---|---|---|---|---|---|---|---|
| case_027 | gemm_reduce | 4 | 559,196 | 1,843,200 | 559,196 | 1,843,200 | 0.00% |
| case_027 | gemm_reduce | 5 | 467,177 | 6,498,304 | 466,823 | 6,498,304 | 33.34% |
| case_028 | conv_deep | 1 | 40,647,084 | 14,300,160 | 40,647,084 | 14,300,160 | 0.00% |
| case_028 | conv_deep | 2 | 20,320,752 | 18,438,216 | 20,302,066 | 18,438,216 | 20.43% |
| case_028 | conv_deep | 3 | 13,516,554 | 13,040,496 | 13,516,554 | 13,040,496 | 0.00% |
| case_028 | conv_deep | 4 | 10,178,658 | 18,792,192 | 10,161,834 | 18,792,192 | 32.53% |
| case_028 | conv_deep | 5 | 8,141,472 | 16,939,008 | 8,108,574 | 16,939,008 | 52.24% |
| case_029 | vector_norm | 1 | 37,555 | 0 | 37,555 | 0 | 0.00% |
| case_029 | vector_norm | 2 | 18,880 | 16,388 | 18,880 | 16,388 | 0.00% |
| case_029 | vector_norm | 3 | 12,743 | 0 | 12,743 | 0 | 0.00% |
| case_029 | vector_norm | 4 | 10,108 | 98,316 | 10,103 | 98,316 | 7.69% |
| case_029 | vector_norm | 5 | 8,523 | 65,552 | 8,523 | 65,552 | 0.00% |
| case_030 | mixed | 1 | 2,800,487 | 5,581,824 | 2,800,279 | 5,581,824 | 36.60% |
| case_030 | mixed | 2 | 1,398,984 | 7,418,944 | 1,398,307 | 7,418,944 | 41.74% |
| case_030 | mixed | 3 | 931,703 | 7,190,948 | 931,488 | 7,190,948 | 51.70% |
| case_030 | mixed | 4 | 698,575 | 5,590,020 | 698,462 | 5,590,020 | 52.49% |
| case_030 | mixed | 5 | 559,453 | 6,793,708 | 559,453 | 6,793,708 | 44.35% |
| case_031 | mixed | 1 | 1,408,643 | 3,876,352 | 1,408,343 | 3,876,352 | 23.88% |
| case_031 | mixed | 2 | 710,417 | 5,287,618 | 710,019 | 5,287,618 | 45.94% |
| case_031 | mixed | 3 | 483,203 | 5,377,310 | 483,164 | 5,377,310 | 40.23% |
| case_031 | mixed | 4 | 365,550 | 8,672,916 | 365,516 | 8,672,916 | 51.13% |
| case_031 | mixed | 5 | 301,187 | 8,438,114 | 300,667 | 8,438,114 | 41.56% |
| case_032 | mixed | 1 | 34,141 | 10,240 | 34,141 | 10,240 | 3.54% |
| case_032 | mixed | 2 | 17,227 | 6,216 | 17,227 | 6,216 | 0.74% |
| case_032 | mixed | 3 | 11,755 | 12,360 | 11,755 | 12,360 | 4.24% |
| case_032 | mixed | 4 | 9,714 | 6,224 | 9,645 | 6,224 | 1.46% |
| case_032 | mixed | 5 | 9,714 | 6,224 | 9,645 | 6,224 | 1.46% |
| case_033 | mixed | 1 | 1,742,774 | 5,069,332 | 1,742,404 | 5,069,332 | 20.07% |
| case_033 | mixed | 2 | 875,745 | 6,495,284 | 875,669 | 6,495,284 | 32.64% |
| case_033 | mixed | 3 | 596,025 | 8,023,732 | 594,676 | 8,023,732 | 30.03% |
| case_033 | mixed | 4 | 453,020 | 8,526,772 | 450,639 | 8,526,772 | 41.05% |
| case_033 | mixed | 5 | 359,988 | 8,922,340 | 357,577 | 8,922,340 | 42.05% |
| case_034 | mixed | 1 | 807,601 | 1,801,762 | 807,483 | 1,801,762 | 20.95% |
| case_034 | mixed | 2 | 406,396 | 5,517,312 | 405,196 | 5,517,312 | 25.37% |
| case_034 | mixed | 3 | 270,360 | 4,267,520 | 265,122 | 4,267,520 | 41.28% |
| case_034 | mixed | 4 | 199,816 | 3,464,712 | 197,513 | 3,464,712 | 42.12% |
| case_034 | mixed | 5 | 162,203 | 3,517,440 | 156,717 | 3,517,440 | 39.34% |
| case_035 | mixed | 1 | 151,302 | 450,576 | 151,302 | 450,576 | 7.14% |
| case_035 | mixed | 2 | 88,361 | 543,426 | 88,361 | 543,426 | 6.67% |
| case_035 | mixed | 3 | 79,490 | 980,492 | 79,164 | 980,492 | 30.20% |
| case_035 | mixed | 4 | 70,052 | 838,422 | 69,986 | 838,422 | 16.26% |
| case_035 | mixed | 5 | 64,594 | 683,082 | 64,242 | 683,082 | 15.93% |
| case_036 | conv_shallow | 1 | 931,510 | 0 | 931,510 | 0 | 0.00% |
| case_036 | conv_shallow | 2 | 466,068 | 1,152 | 466,068 | 1,152 | 0.00% |
| case_036 | conv_shallow | 3 | 311,345 | 2,304 | 311,345 | 2,304 | 0.00% |

| 用例 | 类型 | 核数 | 无 L2 Makespan | 无 L2 额外搬运/B | 只读 Cache Makespan | 只读 Cache 额外搬运/B | Cache 命中率 |
|---|---|---|---|---|---|---|---|
| case_036 | conv_shallow | 4 | 233,584 | 3,456 | 233,584 | 3,456 | 0.00% |
| case_036 | conv_shallow | 5 | 187,182 | 4,608 | 187,182 | 4,608 | 0.00% |
| case_037 | gemm_reduce | 1 | 210,001 | 0 | 210,001 | 0 | 0.00% |
| case_037 | gemm_reduce | 2 | 105,374 | 204,800 | 105,374 | 204,800 | 0.00% |
| case_037 | gemm_reduce | 3 | 71,349 | 1,210,368 | 71,349 | 1,210,368 | 61.04% |
| case_037 | gemm_reduce | 4 | 53,320 | 614,400 | 53,320 | 614,400 | 0.00% |
| case_037 | gemm_reduce | 5 | 43,013 | 942,080 | 43,013 | 942,080 | 42.22% |
| case_038 | mixed | 1 | 1,345,349 | 4,475,392 | 1,345,188 | 4,475,392 | 23.23% |
| case_038 | mixed | 2 | 676,424 | 6,162,944 | 676,257 | 6,162,944 | 22.25% |
| case_038 | mixed | 3 | 454,069 | 4,686,236 | 454,030 | 4,686,236 | 42.57% |
| case_038 | mixed | 4 | 343,034 | 6,170,624 | 341,388 | 6,170,624 | 59.72% |
| case_038 | mixed | 5 | 280,538 | 8,956,936 | 273,051 | 8,956,936 | 40.55% |
| case_039 | gemm_reduce | 1 | 1,387,607 | 18,788,352 | 1,387,555 | 18,788,352 | 1.15% |
| case_039 | gemm_reduce | 2 | 699,623 | 8,937,472 | 699,623 | 8,937,472 | 6.19% |
| case_039 | gemm_reduce | 3 | 470,705 | 11,673,600 | 469,274 | 11,673,600 | 40.40% |
| case_039 | gemm_reduce | 4 | 380,904 | 17,510,400 | 355,032 | 17,510,400 | 51.15% |
| case_039 | gemm_reduce | 5 | 347,811 | 14,811,136 | 308,064 | 14,811,136 | 53.96% |
| case_040 | mixed | 1 | 241,103 | 0 | 241,103 | 0 | 0.00% |
| case_040 | mixed | 2 | 141,530 | 16,400 | 141,521 | 16,400 | 1.33% |
| case_040 | mixed | 3 | 108,284 | 1,171,270 | 107,566 | 1,171,270 | 21.70% |
| case_040 | mixed | 4 | 96,187 | 1,477,894 | 94,909 | 1,477,894 | 32.32% |
| case_040 | mixed | 5 | 87,822 | 1,802,604 | 85,856 | 1,802,604 | 36.52% |
| case_041 | mixed | 1 | 5,960,033 | 32,114,688 | 5,959,586 | 32,114,688 | 6.93% |
| case_041 | mixed | 2 | 2,993,461 | 28,197,888 | 2,990,288 | 28,197,888 | 20.72% |
| case_041 | mixed | 3 | 1,997,980 | 34,007,040 | 1,995,291 | 34,007,040 | 25.23% |
| case_041 | mixed | 4 | 1,493,190 | 32,652,288 | 1,491,718 | 32,652,288 | 25.54% |
| case_041 | mixed | 5 | 1,195,037 | 32,544,768 | 1,192,343 | 32,544,768 | 30.61% |
| case_042 | mixed | 1 | 1,637,474 | 3,726,624 | 1,637,300 | 3,726,624 | 10.89% |
| case_042 | mixed | 2 | 837,278 | 6,661,696 | 835,257 | 6,661,696 | 30.96% |
| case_042 | mixed | 3 | 563,953 | 8,368,416 | 563,371 | 8,368,416 | 40.82% |
| case_042 | mixed | 4 | 420,891 | 3,432,296 | 420,852 | 3,432,296 | 46.06% |
| case_042 | mixed | 5 | 355,344 | 13,061,376 | 345,698 | 13,061,376 | 52.14% |
| case_043 | mixed | 1 | 656,877 | 3,104,800 | 654,317 | 3,104,800 | 8.81% |
| case_043 | mixed | 2 | 332,593 | 523,264 | 332,593 | 523,264 | 26.78% |
| case_043 | mixed | 3 | 243,991 | 438,916 | 243,991 | 438,916 | 10.23% |
| case_043 | mixed | 4 | 238,951 | 7,051,542 | 231,533 | 7,051,542 | 40.06% |
| case_043 | mixed | 5 | 209,984 | 4,460,710 | 197,532 | 4,460,710 | 43.02% |
| case_044 | conv_deep | 1 | 74,601 | 0 | 74,601 | 0 | 0.00% |
| case_044 | conv_deep | 2 | 49,413 | 935,008 | 43,142 | 935,008 | 44.56% |
| case_044 | conv_deep | 3 | 49,413 | 935,008 | 43,142 | 935,008 | 44.56% |
| case_044 | conv_deep | 4 | 49,413 | 935,008 | 43,142 | 935,008 | 44.56% |
| case_044 | conv_deep | 5 | 50,676 | 1,581,952 | 37,817 | 1,581,952 | 42.37% |
| case_045 | vector_elementwise | 1 | 125,614 | 0 | 125,614 | 0 | 0.00% |
| case_045 | vector_elementwise | 2 | 63,059 | 0 | 63,059 | 0 | 0.00% |
| case_045 | vector_elementwise | 3 | 42,601 | 0 | 42,601 | 0 | 0.00% |

| 用例 | 类型 | 核数 | 无 L2 Makespan | 无 L2 额外搬运/B | 只读 Cache Makespan | 只读 Cache 额外搬运/B | Cache 命中率 |
|---|---|---|---|---|---|---|---|
| case_045 | vector_elementwise | 4 | 31,871 | 0 | 31,871 | 0 | 0.00% |
| case_045 | vector_elementwise | 5 | 26,164 | 0 | 26,164 | 0 | 0.00% |
| case_046 | conv_deep | 1 | 210,387 | 0 | 210,387 | 0 | 0.00% |
| case_046 | conv_deep | 2 | 107,977 | 935,520 | 106,065 | 935,520 | 12.40% |
| case_046 | conv_deep | 3 | 86,232 | 2,233,536 | 77,258 | 2,233,536 | 50.21% |
| case_046 | conv_deep | 4 | 101,350 | 3,257,024 | 68,708 | 3,257,024 | 68.91% |
| case_046 | conv_deep | 5 | 88,823 | 3,037,568 | 56,442 | 3,037,568 | 64.46% |
| case_047 | transformer | 1 | 297,583 | 0 | 297,583 | 0 | 0.00% |
| case_047 | transformer | 2 | 169,449 | 1,337,350 | 169,448 | 1,337,350 | 10.84% |
| case_047 | transformer | 3 | 133,645 | 1,852,268 | 133,449 | 1,852,268 | 15.59% |
| case_047 | transformer | 4 | 109,597 | 2,421,170 | 108,294 | 2,421,170 | 27.99% |
| case_047 | transformer | 5 | 109,597 | 2,421,170 | 108,294 | 2,421,170 | 27.99% |
| case_048 | transformer | 1 | 172,063 | 0 | 172,063 | 0 | 0.00% |
| case_048 | transformer | 2 | 121,414 | 643,206 | 121,414 | 643,206 | 14.05% |
| case_048 | transformer | 3 | 120,479 | 1,173,356 | 120,325 | 1,173,356 | 17.32% |
| case_048 | transformer | 4 | 107,174 | 952,050 | 106,852 | 952,050 | 11.17% |
| case_048 | transformer | 5 | 107,874 | 952,050 | 107,519 | 952,050 | 11.18% |
| case_049 | mixed | 1 | 180,092 | 9,232 | 180,092 | 9,232 | 0.00% |
| case_049 | mixed | 2 | 100,578 | 1,127,350 | 100,535 | 1,127,350 | 4.97% |
| case_049 | mixed | 3 | 78,883 | 1,892,172 | 77,051 | 1,892,172 | 13.75% |
| case_049 | mixed | 4 | 63,471 | 2,046,958 | 62,439 | 2,046,958 | 16.12% |
| case_049 | mixed | 5 | 62,115 | 2,046,958 | 61,325 | 2,046,958 | 16.22% |
| case_050 | mixed | 1 | 141,849 | 681,388 | 141,162 | 681,388 | 15.11% |
| case_050 | mixed | 2 | 76,697 | 713,770 | 76,508 | 713,770 | 19.49% |
| case_050 | mixed | 3 | 64,517 | 1,614,884 | 62,115 | 1,614,884 | 24.80% |
| case_050 | mixed | 4 | 57,466 | 1,894,530 | 54,091 | 1,894,530 | 26.51% |
| case_050 | mixed | 5 | 56,963 | 1,894,530 | 53,513 | 1,894,530 | 24.29% |
| case_051 | vector_chain | 1 | 607,628 | 0 | 607,628 | 0 | 0.00% |
| case_051 | vector_chain | 2 | 318,295 | 668 | 318,295 | 668 | 0.00% |
| case_051 | vector_chain | 3 | 227,413 | 952 | 227,413 | 952 | 0.01% |
| case_051 | vector_chain | 4 | 178,014 | 1,140 | 178,014 | 1,140 | 0.02% |
| case_051 | vector_chain | 5 | 169,680 | 1,232 | 169,680 | 1,232 | 0.04% |
| case_052 | mixed | 1 | 153,974 | 0 | 153,974 | 0 | 0.00% |
| case_052 | mixed | 2 | 79,240 | 78,336 | 79,240 | 78,336 | 8.82% |
| case_052 | mixed | 3 | 55,133 | 343,604 | 55,133 | 343,604 | 18.48% |
| case_052 | mixed | 4 | 40,676 | 259,584 | 40,676 | 259,584 | 5.17% |
| case_052 | mixed | 5 | 33,467 | 337,920 | 33,467 | 337,920 | 12.57% |
| case_053 | mixed | 1 | 1,288,582 | 478,218 | 1,288,582 | 478,218 | 0.44% |
| case_053 | mixed | 2 | 685,024 | 5,684,370 | 681,933 | 5,684,370 | 14.27% |
| case_053 | mixed | 3 | 473,361 | 8,589,376 | 469,422 | 8,589,376 | 13.74% |
| case_053 | mixed | 4 | 407,205 | 10,085,850 | 393,682 | 10,085,850 | 32.50% |
| case_053 | mixed | 5 | 381,111 | 14,394,776 | 351,347 | 14,394,776 | 32.35% |
| case_054 | mixed | 1 | 801,689 | 611,990 | 801,689 | 611,990 | 0.00% |
| case_054 | mixed | 2 | 420,402 | 4,463,698 | 420,402 | 4,463,698 | 2.76% |
| case_054 | mixed | 3 | 307,649 | 6,689,528 | 305,901 | 6,689,528 | 9.25% |

| 用例 | 类型 | 核数 | 无 L2 Makespan | 无 L2 额外搬运/B | 只读 Cache Makespan | 只读 Cache 额外搬运/B | Cache 命中率 |
|---|---|---|---|---|---|---|---|
| case_054 | mixed | 4 | 251,957 | 8,404,422 | 248,132 | 8,404,422 | 13.43% |
| case_054 | mixed | 5 | 241,387 | 9,858,084 | 230,192 | 9,858,084 | 17.70% |
| case_055 | mixed | 1 | 103,877 | 1,489,240 | 102,681 | 1,489,240 | 14.94% |
| case_055 | mixed | 2 | 51,873 | 81,920 | 51,873 | 81,920 | 2.43% |
| case_055 | mixed | 3 | 34,669 | 132,096 | 34,669 | 132,096 | 14.27% |
| case_055 | mixed | 4 | 30,161 | 495,952 | 29,163 | 495,952 | 16.13% |
| case_055 | mixed | 5 | 21,390 | 148,480 | 21,351 | 148,480 | 17.06% |
| case_056 | mixed | 1 | 253,682 | 145,096 | 253,682 | 145,096 | 13.78% |
| case_056 | mixed | 2 | 155,922 | 1,326,176 | 155,922 | 1,326,176 | 8.78% |
| case_056 | mixed | 3 | 105,289 | 1,409,580 | 104,501 | 1,409,580 | 17.45% |
| case_056 | mixed | 4 | 97,677 | 2,814,678 | 96,962 | 2,814,678 | 21.16% |
| case_056 | mixed | 5 | 103,184 | 4,302,248 | 97,210 | 4,302,248 | 21.91% |
| case_057 | mixed | 1 | 56,565 | 598,152 | 55,137 | 598,152 | 25.81% |
| case_057 | mixed | 2 | 30,276 | 54,496 | 30,276 | 54,496 | 6.05% |
| case_057 | mixed | 3 | 20,705 | 14,112 | 20,705 | 14,112 | 1.40% |
| case_057 | mixed | 4 | 16,481 | 27,968 | 16,481 | 27,968 | 7.66% |
| case_057 | mixed | 5 | 13,794 | 14,160 | 13,794 | 14,160 | 4.02% |
| case_058 | gemm_reduce | 1 | 4,493,031 | 30,846,976 | 4,478,999 | 30,846,976 | 23.76% |
| case_058 | gemm_reduce | 2 | 2,242,317 | 63,356,928 | 2,237,909 | 63,356,928 | 43.11% |
| case_058 | gemm_reduce | 3 | 1,507,683 | 29,433,856 | 1,507,363 | 29,433,856 | 6.23% |
| case_058 | gemm_reduce | 4 | 1,141,935 | 31,551,488 | 1,129,512 | 31,551,488 | 42.36% |
| case_058 | gemm_reduce | 5 | 936,945 | 30,875,648 | 928,663 | 30,875,648 | 5.77% |
| case_059 | gemm_reduce | 1 | 738,041 | 0 | 738,041 | 0 | 0.00% |
| case_059 | gemm_reduce | 2 | 369,318 | 360,448 | 369,318 | 360,448 | 0.00% |
| case_059 | gemm_reduce | 3 | 253,759 | 1,179,648 | 253,657 | 1,179,648 | 49.18% |
| case_059 | gemm_reduce | 4 | 185,216 | 1,081,344 | 185,216 | 1,081,344 | 0.00% |
| case_059 | gemm_reduce | 5 | 150,782 | 2,072,576 | 150,736 | 2,072,576 | 54.42% |
| case_060 | mixed | 1 | 875,822 | 2,453,676 | 875,067 | 2,453,676 | 44.36% |
| case_060 | mixed | 2 | 443,355 | 3,815,444 | 442,784 | 3,815,444 | 37.87% |
| case_060 | mixed | 3 | 270,421 | 2,878,720 | 270,341 | 2,878,720 | 34.49% |
| case_060 | mixed | 4 | 227,365 | 3,763,460 | 226,784 | 3,763,460 | 48.65% |
| case_060 | mixed | 5 | 160,359 | 4,076,050 | 157,954 | 4,076,050 | 45.57% |
| case_061 | mixed | 1 | 75,090 | 0 | 75,090 | 0 | 0.00% |
| case_061 | mixed | 2 | 37,768 | 31,744 | 37,768 | 31,744 | 4.43% |
| case_061 | mixed | 3 | 25,922 | 62,208 | 25,922 | 62,208 | 9.12% |
| case_061 | mixed | 4 | 20,644 | 76,544 | 20,486 | 76,544 | 6.56% |
| case_061 | mixed | 5 | 17,740 | 122,628 | 17,740 | 122,628 | 14.03% |
| case_062 | mlp | 1 | 2,854,809 | 0 | 2,854,809 | 0 | 0.00% |
| case_062 | mlp | 2 | 1,428,520 | 21,504 | 1,428,520 | 21,504 | 0.00% |
| case_062 | mlp | 3 | 955,171 | 297,984 | 955,171 | 297,984 | 0.00% |
| case_062 | mlp | 4 | 717,090 | 339,456 | 717,090 | 339,456 | 0.00% |
| case_062 | mlp | 5 | 578,617 | 353,280 | 578,617 | 353,280 | 0.00% |
| case_063 | mlp | 1 | 1,070,397 | 0 | 1,070,397 | 0 | 0.00% |
| case_063 | mlp | 2 | 536,188 | 15,360 | 536,188 | 15,360 | 0.00% |
| case_063 | mlp | 3 | 358,403 | 18,432 | 358,403 | 18,432 | 0.00% |

| 用例 | 类型 | 核数 | 无 L2 Makespan | 无 L2 额外搬运/B | 只读 Cache Makespan | 只读 Cache 额外搬运/B | Cache 命中率 |
|---|---|---|---|---|---|---|---|
| case_063 | mlp | 4 | 266,864 | 1,869,312 | 266,864 | 1,869,312 | 1.43% |
| case_063 | mlp | 5 | 221,953 | 76,800 | 221,953 | 76,800 | 0.00% |
| case_064 | transformer | 1 | 16,829 | 0 | 16,829 | 0 | 0.00% |
| case_064 | transformer | 2 | 12,389 | 56,292 | 12,389 | 56,292 | 14.53% |
| case_064 | transformer | 3 | 10,930 | 78,348 | 10,889 | 78,348 | 22.32% |
| case_064 | transformer | 4 | 10,579 | 78,348 | 10,531 | 78,348 | 30.20% |
| case_064 | transformer | 5 | 10,543 | 78,348 | 10,496 | 78,348 | 27.46% |
| case_065 | mixed | 1 | 38,211 | 12,296 | 38,211 | 12,296 | 0.00% |
| case_065 | mixed | 2 | 19,247 | 23,184 | 19,164 | 23,184 | 4.39% |
| case_065 | mixed | 3 | 14,394 | 27,792 | 14,173 | 27,792 | 4.26% |
| case_065 | mixed | 4 | 11,333 | 50,832 | 10,923 | 50,832 | 10.70% |
| case_065 | mixed | 5 | 9,797 | 68,896 | 9,383 | 68,896 | 14.08% |
| case_066 | mixed | 1 | 325,983 | 0 | 325,983 | 0 | 0.00% |
| case_066 | mixed | 2 | 181,340 | 2,010,440 | 180,767 | 2,010,440 | 13.97% |
| case_066 | mixed | 3 | 133,552 | 2,814,100 | 131,201 | 2,814,100 | 17.77% |
| case_066 | mixed | 4 | 116,716 | 3,189,234 | 113,321 | 3,189,234 | 21.08% |
| case_066 | mixed | 5 | 113,094 | 4,090,934 | 107,595 | 4,090,934 | 22.42% |
| case_067 | conv_deep | 1 | 60,662,445 | 6,786,048 | 60,662,445 | 6,786,048 | 0.00% |
| case_067 | conv_deep | 2 | 30,495,081 | 14,811,520 | 30,481,362 | 14,811,520 | 19.66% |
| case_067 | conv_deep | 3 | 20,182,961 | 27,140,864 | 20,124,437 | 27,140,864 | 28.57% |
| case_067 | conv_deep | 4 | 15,157,385 | 26,768,512 | 15,097,210 | 26,768,512 | 38.48% |
| case_067 | conv_deep | 5 | 12,137,530 | 61,951,488 | 12,074,188 | 61,951,488 | 30.39% |
| case_068 | mixed | 1 | 323,275 | 659,924 | 322,585 | 659,924 | 4.27% |
| case_068 | mixed | 2 | 196,903 | 2,883,634 | 196,247 | 2,883,634 | 8.91% |
| case_068 | mixed | 3 | 165,850 | 3,620,096 | 158,705 | 3,620,096 | 16.49% |
| case_068 | mixed | 4 | 160,130 | 4,432,124 | 150,016 | 4,432,124 | 15.58% |
| case_068 | mixed | 5 | 142,556 | 2,063,148 | 139,086 | 2,063,148 | 24.33% |
| case_069 | transformer | 1 | 21,861 | 0 | 21,861 | 0 | 0.00% |
| case_069 | transformer | 2 | 13,026 | 176,806 | 13,021 | 176,806 | 22.46% |
| case_069 | transformer | 3 | 9,477 | 269,960 | 9,436 | 269,960 | 34.28% |
| case_069 | transformer | 4 | 9,478 | 340,782 | 8,983 | 340,782 | 34.62% |
| case_069 | transformer | 5 | 9,478 | 340,782 | 8,983 | 340,782 | 34.62% |
| case_070 | mixed | 1 | 103,683 | 0 | 103,683 | 0 | 0.00% |
| case_070 | mixed | 2 | 52,876 | 101,888 | 52,876 | 101,888 | 19.53% |
| case_070 | mixed | 3 | 38,286 | 259,072 | 38,286 | 259,072 | 11.16% |
| case_070 | mixed | 4 | 32,323 | 347,648 | 30,959 | 347,648 | 20.24% |
| case_070 | mixed | 5 | 28,689 | 462,848 | 25,750 | 462,848 | 27.15% |
| case_071 | transformer | 1 | 16,433 | 0 | 16,433 | 0 | 0.00% |
| case_071 | transformer | 2 | 9,671 | 154,726 | 9,666 | 154,726 | 19.17% |
| case_071 | transformer | 3 | 8,048 | 219,338 | 8,019 | 219,338 | 29.67% |
| case_071 | transformer | 4 | 7,966 | 237,866 | 7,290 | 237,866 | 23.99% |
| case_071 | transformer | 5 | 7,435 | 281,296 | 6,412 | 281,296 | 27.98% |
| case_072 | mixed | 1 | 23,658,769 | 123,587,360 | 23,656,065 | 123,587,360 | 2.11% |
| case_072 | mixed | 2 | 11,830,081 | 121,251,128 | 11,817,956 | 121,251,128 | 7.91% |
| case_072 | mixed | 3 | 7,945,629 | 114,002,720 | 7,937,690 | 114,002,720 | 9.96% |

| 用例 | 类型 | 核数 | 无 L2 Makespan | 无 L2 额外搬运/B | 只读 Cache Makespan | 只读 Cache 额外搬运/B | Cache 命中率 |
|---|---|---|---|---|---|---|---|
| case_072 | mixed | 4 | 5,994,354 | 122,293,024 | 5,985,478 | 122,293,024 | 10.03% |
| case_072 | mixed | 5 | 4,846,959 | 71,866,822 | 4,819,330 | 71,866,822 | 18.60% |
| case_073 | conv_deep | 1 | 8,211,639 | 188,416 | 8,211,639 | 188,416 | 0.00% |
| case_073 | conv_deep | 2 | 4,188,797 | 6,016,384 | 4,156,691 | 6,016,384 | 42.52% |
| case_073 | conv_deep | 3 | 2,841,997 | 7,443,200 | 2,839,515 | 7,443,200 | 47.09% |
| case_073 | conv_deep | 4 | 2,143,833 | 17,091,712 | 2,121,213 | 17,091,712 | 39.47% |
| case_073 | conv_deep | 5 | 1,750,370 | 20,235,264 | 1,718,609 | 20,235,264 | 50.37% |
| case_074 | mixed | 1 | 1,615,877 | 5,783,562 | 1,615,877 | 5,783,562 | 1.51% |
| case_074 | mixed | 2 | 822,510 | 7,077,888 | 820,789 | 7,077,888 | 26.19% |
| case_074 | mixed | 3 | 542,606 | 7,516,160 | 540,831 | 7,516,160 | 24.45% |
| case_074 | mixed | 4 | 410,475 | 6,991,882 | 410,206 | 6,991,882 | 46.09% |
| case_074 | mixed | 5 | 331,814 | 4,821,002 | 332,174 | 4,821,002 | 33.56% |
| case_075 | transformer | 1 | 1,206,555 | 0 | 1,206,555 | 0 | 0.00% |
| case_075 | transformer | 2 | 645,044 | 2,703,046 | 645,044 | 2,703,046 | 13.78% |
| case_075 | transformer | 3 | 592,468 | 7,307,148 | 590,796 | 7,307,148 | 13.14% |
| case_075 | transformer | 4 | 434,416 | 5,931,602 | 434,298 | 5,931,602 | 27.71% |
| case_075 | transformer | 5 | 370,929 | 10,320,376 | 358,532 | 10,320,376 | 30.49% |
| case_076 | mixed | 1 | 6,126,869 | 35,655,680 | 6,125,887 | 35,655,680 | 7.72% |
| case_076 | mixed | 2 | 3,069,464 | 30,744,576 | 3,067,211 | 30,744,576 | 25.84% |
| case_076 | mixed | 3 | 2,059,036 | 35,182,704 | 2,054,626 | 35,182,704 | 33.18% |
| case_076 | mixed | 4 | 1,539,912 | 17,147,360 | 1,539,041 | 17,147,360 | 40.08% |
| case_076 | mixed | 5 | 1,237,040 | 37,702,800 | 1,230,229 | 37,702,800 | 36.90% |
| case_077 | mixed | 1 | 484,609 | 0 | 484,609 | 0 | 0.00% |
| case_077 | mixed | 2 | 246,403 | 57,344 | 246,403 | 57,344 | 2.61% |
| case_077 | mixed | 3 | 168,425 | 131,072 | 168,425 | 131,072 | 1.45% |
| case_077 | mixed | 4 | 141,327 | 1,832,932 | 140,426 | 1,832,932 | 12.31% |
| case_077 | mixed | 5 | 107,273 | 196,608 | 106,758 | 196,608 | 5.64% |
| case_078 | conv_deep | 1 | 2,050,746 | 24,576 | 2,050,746 | 24,576 | 0.00% |
| case_078 | conv_deep | 2 | 1,029,560 | 2,148,480 | 1,029,560 | 2,148,480 | 0.00% |
| case_078 | conv_deep | 3 | 717,252 | 6,241,536 | 710,596 | 6,241,536 | 51.06% |
| case_078 | conv_deep | 4 | 520,115 | 6,445,440 | 516,004 | 6,445,440 | 73.31% |
| case_078 | conv_deep | 5 | 486,419 | 11,205,240 | 455,615 | 11,205,240 | 56.04% |
| case_079 | gemm_reduce | 1 | 30,339,109 | 30,445,568 | 30,318,517 | 30,445,568 | 40.22% |
| case_079 | gemm_reduce | 2 | 14,853,842 | 27,643,904 | 14,848,850 | 27,643,904 | 5.09% |
| case_079 | gemm_reduce | 3 | 9,993,353 | 57,720,832 | 9,991,117 | 57,720,832 | 28.39% |
| case_079 | gemm_reduce | 4 | 7,635,624 | 30,445,568 | 7,635,624 | 30,445,568 | 0.00% |
| case_079 | gemm_reduce | 5 | 6,063,040 | 33,882,112 | 6,062,064 | 33,882,112 | 46.71% |
| case_080 | gemm_reduce | 1 | 289,813 | 1,400,832 | 289,813 | 1,400,832 | 7.63% |
| case_080 | gemm_reduce | 2 | 145,224 | 380,928 | 145,224 | 380,928 | 0.00% |
| case_080 | gemm_reduce | 3 | 99,401 | 778,240 | 98,003 | 778,240 | 14.07% |
| case_080 | gemm_reduce | 4 | 107,717 | 4,444,160 | 76,390 | 4,444,160 | 61.57% |
| case_080 | gemm_reduce | 5 | 80,273 | 2,650,112 | 65,548 | 2,650,112 | 34.24% |
| case_081 | vector_norm | 1 | 151,795 | 0 | 151,795 | 0 | 0.00% |
| case_081 | vector_norm | 2 | 76,316 | 24,620 | 76,316 | 24,620 | 2.30% |
| case_081 | vector_norm | 3 | 52,016 | 8,196 | 51,842 | 8,196 | 0.39% |

| 用例 | 类型 | 核数 | 无 L2 Makespan | 无 L2 额外搬运/B | 只读 Cache Makespan | 只读 Cache 额外搬运/B | Cache 命中率 |
|---|---|---|---|---|---|---|---|
| case_081 | vector_norm | 4 | 39,588 | 0 | 39,588 | 0 | 0.00% |
| case_081 | vector_norm | 5 | 30,943 | 0 | 30,943 | 0 | 0.00% |
| case_082 | transformer | 1 | 607,062 | 494,346 | 607,062 | 494,346 | 0.00% |
| case_082 | transformer | 2 | 326,188 | 1,261,606 | 326,186 | 1,261,606 | 13.40% |
| case_082 | transformer | 3 | 244,746 | 2,553,228 | 244,738 | 2,553,228 | 14.86% |
| case_082 | transformer | 4 | 187,056 | 3,223,250 | 185,947 | 3,223,250 | 30.42% |
| case_082 | transformer | 5 | 177,966 | 5,376,182 | 173,479 | 5,376,182 | 24.85% |
| case_083 | conv_deep | 1 | 837,256 | 131,072 | 837,256 | 131,072 | 0.00% |
| case_083 | conv_deep | 2 | 411,165 | 2,893,408 | 410,658 | 2,893,408 | 10.99% |
| case_083 | conv_deep | 3 | 288,224 | 5,294,272 | 283,789 | 5,294,272 | 28.07% |
| case_083 | conv_deep | 4 | 214,624 | 2,791,200 | 214,624 | 2,791,200 | 70.19% |
| case_083 | conv_deep | 5 | 205,974 | 5,538,176 | 174,824 | 5,538,176 | 56.38% |
| case_084 | attention | 1 | 1,371,036 | 9,667,584 | 1,371,036 | 9,667,584 | 0.00% |
| case_084 | attention | 2 | 686,336 | 9,437,184 | 686,336 | 9,437,184 | 0.00% |
| case_084 | attention | 3 | 458,172 | 9,179,136 | 458,172 | 9,179,136 | 0.00% |
| case_084 | attention | 4 | 384,138 | 8,957,952 | 384,138 | 8,957,952 | 0.00% |
| case_084 | attention | 5 | 372,532 | 7,603,776 | 372,532 | 7,603,776 | 0.00% |
| case_085 | transformer | 1 | 2,948,702 | 6,086,434 | 2,948,702 | 6,086,434 | 0.00% |
| case_085 | transformer | 2 | 1,649,767 | 11,636,202 | 1,649,763 | 11,636,202 | 10.02% |
| case_085 | transformer | 3 | 1,171,589 | 18,491,276 | 1,171,573 | 18,491,276 | 22.82% |
| case_085 | transformer | 4 | 838,246 | 10,155,890 | 837,423 | 10,155,890 | 32.81% |
| case_085 | transformer | 5 | 761,913 | 25,100,728 | 730,829 | 25,100,728 | 29.01% |
| case_086 | transformer | 1 | 106,323 | 66,052 | 106,323 | 66,052 | 0.00% |
| case_086 | transformer | 2 | 57,916 | 917,094 | 57,663 | 917,094 | 3.21% |
| case_086 | transformer | 3 | 58,195 | 2,197,324 | 56,875 | 2,197,324 | 16.50% |
| case_086 | transformer | 4 | 45,730 | 1,704,434 | 43,511 | 1,704,434 | 27.97% |
| case_086 | transformer | 5 | 44,764 | 1,704,434 | 42,600 | 1,704,434 | 26.19% |
| case_087 | mixed | 1 | 3,974,071 | 7,902,720 | 3,969,779 | 7,902,720 | 18.34% |
| case_087 | mixed | 2 | 1,985,234 | 9,999,360 | 1,984,074 | 9,999,360 | 26.84% |
| case_087 | mixed | 3 | 1,385,253 | 10,722,816 | 1,383,483 | 10,722,816 | 22.51% |
| case_087 | mixed | 4 | 1,069,661 | 10,483,200 | 1,069,661 | 10,483,200 | 29.44% |
| case_087 | mixed | 5 | 888,434 | 10,391,040 | 888,435 | 10,391,040 | 25.60% |
| case_088 | mixed | 1 | 216,113 | 308,936 | 216,061 | 308,936 | 1.03% |
| case_088 | mixed | 2 | 135,525 | 2,032,426 | 134,155 | 2,032,426 | 12.62% |
| case_088 | mixed | 3 | 116,585 | 2,882,368 | 113,347 | 2,882,368 | 11.19% |
| case_088 | mixed | 4 | 101,449 | 1,715,088 | 98,173 | 1,715,088 | 21.82% |
| case_088 | mixed | 5 | 101,812 | 1,715,088 | 97,861 | 1,715,088 | 20.08% |
| case_089 | conv_deep | 1 | 591,928 | 0 | 591,928 | 0 | 0.00% |
| case_089 | conv_deep | 2 | 295,856 | 238,720 | 295,856 | 238,720 | 0.00% |
| case_089 | conv_deep | 3 | 203,303 | 1,670,400 | 203,223 | 1,670,400 | 29.21% |
| case_089 | conv_deep | 4 | 148,856 | 716,160 | 148,856 | 716,160 | 0.00% |
| case_089 | conv_deep | 5 | 120,170 | 954,880 | 120,170 | 954,880 | 0.00% |
| case_090 | conv_deep | 1 | 841,279 | 0 | 841,279 | 0 | 0.00% |
| case_090 | conv_deep | 2 | 433,153 | 3,176,064 | 425,018 | 3,176,064 | 21.19% |
| case_090 | conv_deep | 3 | 317,356 | 5,791,872 | 302,462 | 5,791,872 | 44.90% |

| 用例 | 类型 | 核数 | 无 L2 Makespan | 无 L2 额外搬运/B | 只读 Cache Makespan | 只读 Cache 额外搬运/B | Cache 命中率 |
|---|---|---|---|---|---|---|---|
| case_090 | conv_deep | 4 | 312,778 | 9,123,456 | 246,151 | 9,123,456 | 60.52% |
| case_090 | conv_deep | 5 | 249,262 | 8,778,240 | 187,191 | 8,778,240 | 51.74% |
| case_091 | mixed | 1 | 9,164,404 | 56,822,400 | 9,147,895 | 56,822,400 | 7.80% |
| case_091 | mixed | 2 | 4,590,046 | 57,827,296 | 4,579,927 | 57,827,296 | 19.06% |
| case_091 | mixed | 3 | 3,059,420 | 60,739,264 | 3,053,259 | 60,739,264 | 23.89% |
| case_091 | mixed | 4 | 2,298,804 | 66,972,160 | 2,296,502 | 66,972,160 | 23.89% |
| case_091 | mixed | 5 | 1,848,452 | 53,598,748 | 1,836,261 | 53,598,748 | 37.42% |
| case_092 | conv_deep | 1 | 4,005,111 | 3,756,032 | 4,005,111 | 3,756,032 | 0.00% |
| case_092 | conv_deep | 2 | 2,006,630 | 4,008,544 | 2,006,630 | 4,008,544 | 0.40% |
| case_092 | conv_deep | 3 | 1,352,575 | 5,270,720 | 1,334,355 | 5,270,720 | 31.58% |
| case_092 | conv_deep | 4 | 1,013,982 | 6,016,800 | 1,006,274 | 6,016,800 | 41.54% |
| case_092 | conv_deep | 5 | 828,074 | 3,942,784 | 819,272 | 3,942,784 | 47.26% |
| case_093 | gemm_shallow | 1 | 74,205 | 0 | 74,205 | 0 | 0.00% |
| case_093 | gemm_shallow | 2 | 37,739 | 12,288 | 37,733 | 12,288 | 1.40% |
| case_093 | gemm_shallow | 3 | 25,103 | 16,384 | 25,103 | 16,384 | 0.00% |
| case_093 | gemm_shallow | 4 | 19,779 | 24,576 | 19,743 | 24,576 | 3.08% |
| case_093 | gemm_shallow | 5 | 15,876 | 32,768 | 15,630 | 32,768 | 4.05% |
| case_094 | mixed | 1 | 84,296 | 102,408 | 84,296 | 102,408 | 0.78% |
| case_094 | mixed | 2 | 40,947 | 146,216 | 40,812 | 146,216 | 15.33% |
| case_094 | mixed | 3 | 27,341 | 163,872 | 26,707 | 163,872 | 24.83% |
| case_094 | mixed | 4 | 21,550 | 325,440 | 20,906 | 325,440 | 19.13% |
| case_094 | mixed | 5 | 21,441 | 258,912 | 18,747 | 258,912 | 18.63% |
| case_095 | attention | 1 | 1,144,413 | 16,809,984 | 1,144,413 | 16,809,984 | 0.00% |
| case_095 | attention | 2 | 573,690 | 16,588,800 | 573,690 | 16,588,800 | 0.00% |
| case_095 | attention | 3 | 474,153 | 16,404,480 | 474,153 | 16,404,480 | 0.00% |
| case_095 | attention | 4 | 436,296 | 13,639,680 | 436,296 | 13,639,680 | 0.00% |
| case_095 | attention | 5 | 418,380 | 995,328 | 418,380 | 995,328 | 0.00% |
| case_096 | mixed | 1 | 63,707 | 155,648 | 63,707 | 155,648 | 1.41% |
| case_096 | mixed | 2 | 40,724 | 340,684 | 40,661 | 340,684 | 26.18% |
| case_096 | mixed | 3 | 26,274 | 89,944 | 26,274 | 89,944 | 15.89% |
| case_096 | mixed | 4 | 26,274 | 89,944 | 26,274 | 89,944 | 15.89% |
| case_096 | mixed | 5 | 26,771 | 875,096 | 24,667 | 875,096 | 29.33% |
| case_097 | gemm_reduce | 1 | 10,343,849 | 8,069,120 | 10,343,849 | 8,069,120 | 0.00% |
| case_097 | gemm_reduce | 2 | 5,228,922 | 14,221,312 | 5,224,762 | 14,221,312 | 20.30% |
| case_097 | gemm_reduce | 3 | 3,542,953 | 6,176,768 | 3,542,953 | 6,176,768 | 46.13% |
| case_097 | gemm_reduce | 4 | 2,626,908 | 14,221,312 | 2,626,908 | 14,221,312 | 4.17% |
| case_097 | gemm_reduce | 5 | 2,102,348 | 21,311,488 | 2,102,250 | 21,311,488 | 24.52% |
| case_098 | mixed | 1 | 170,571 | 341,076 | 170,153 | 341,076 | 7.71% |
| case_098 | mixed | 2 | 89,333 | 2,006,858 | 89,171 | 2,006,858 | 10.42% |
| case_098 | mixed | 3 | 62,941 | 2,271,138 | 62,865 | 2,271,138 | 12.95% |
| case_098 | mixed | 4 | 58,709 | 1,328,310 | 56,972 | 1,328,310 | 25.88% |
| case_098 | mixed | 5 | 54,513 | 1,555,310 | 51,281 | 1,555,310 | 27.11% |
| case_099 | mixed | 1 | 1,600,344 | 5,411,584 | 1,600,344 | 5,411,584 | 1.38% |
| case_099 | mixed | 2 | 803,473 | 5,778,816 | 802,393 | 5,778,816 | 36.57% |
| case_099 | mixed | 3 | 534,933 | 5,653,760 | 534,706 | 5,653,760 | 49.95% |

| 用例 | 类型 | 核数 | 无 L2 Makespan | 无 L2 额外搬运/B | 只读 Cache Makespan | 只读 Cache 额外搬运/B | Cache 命中率 |
|---|---|---|---|---|---|---|---|
| case_099 | mixed | 4 | 402,852 | 6,785,280 | 401,916 | 6,785,280 | 63.28% |
| case_099 | mixed | 5 | 325,977 | 8,179,456 | 322,113 | 8,179,456 | 68.71% |
| case_100 | mixed | 1 | 133,562 | 0 | 133,562 | 0 | 0.00% |
| case_100 | mixed | 2 | 72,903 | 226,640 | 72,903 | 226,640 | 9.28% |
| case_100 | mixed | 3 | 51,144 | 221,220 | 51,144 | 221,220 | 33.46% |
| case_100 | mixed | 4 | 51,144 | 221,220 | 51,144 | 221,220 | 33.46% |
| case_100 | mixed | 5 | 46,874 | 319,540 | 46,034 | 319,540 | 42.13% |

# 附录 D 代码说明

求解与实验全部使用 Python 3 编写，结果均经原版官方评估程序复核。各问题代码清单及其作用如下。

## D.1 问题一

| 文件 | 作用 |
|---|---|
| solve.py | 主求解入口，组织整体求解流程 |
| partition.py | 计算图切分与子图划分 |
| scheduling.py | 核内调度与 Task 排序 |
| execution.py | 完整时序与共享带宽竞争模拟 |
| benchmark.py | 批量评测，按 2 ~ 5 核生成指标 |
| plot_results.py | 结果绘图 |

## D.2 问题二

| 文件 | 作用 |
|---|---|
| solve.py | 阶段控制与候选流管理（Solution、Solver.solve） |
| partition.py | 分组代理、张量边界与下界（CostModel） |
| scheduling.py | 顺序、细分与合并（顺序邻域） |
| benchmark.py | 批量统计与瓶颈指标 |

## D.3 问题三

| 文件 | 作用 |
|---|---|
| model.py | 张量通信模型、安全下界、结构初始方案 |
| search.py | 核内重排、切分合并、瓶颈重切、迁移及 Cache 反馈邻域 |
| solve.py | 搜索控制、字典序选优、提交文件与官方核验 |
| diagnostics.py | 分析 Cache 事件、核心负载、带宽与通信等待 |
| benchmark.py | 用例覆盖统计、1 ~ 5 核实验、汇总 |
| sensitivity.py | 固定方案的容量/带宽灵敏度补充实验 |
| plot_results.py | 双配置曲线与 Cache 加速比图 |

# 附录 E 核心代码

## E1 问题一

partition.py
计算切图后的依赖与搬运代价:

```
def describe(graph, groups, settings, name):
    groups = canonical(groups)
    owner = {v: s for s, nodes in enumerate(groups) for v in nodes}
    if len(owner) != sum(map(len, groups)) or set(owner) != set(graph.ops):
        raise ValueError('划分必须恰好覆盖全部非 COPY 操作')
    preds = {s: set() for s in range(len(groups))}
    succs = {s: set() for s in preds}
    for v, following in graph.succs.items():
        for w in following:
            a, b = owner[v], owner[w]
            if a != b:
                preds[b].add(a)
                succs[a].add(b)
    order = topological(preds, succs)
    load = [[0] * 4 for _ in groups]
    ddr_work = [0] * len(groups)
    for v, op in graph.ops.items():
        load[owner[v]][PIPES.index(op['pipe'])] += max(1, op['cycles'])
    copy_bytes = 0
    read_time, write_time = {}, {}
    for tid, tensor in graph.tensors.items():
        ps = {owner[v] for v in graph.producers[tid]}
        cs = {owner[v] for v in graph.consumers[tid]}
        size = tensor['size']
        duration = max(1, math.ceil(size / settings.bandwidth))
        for s in cs - ps:
            load[s][0] += duration
            ddr_work[s] += duration
            read_time[s, tid] = duration
            copy_bytes += size
        for s in ps:
            if tid in graph.final or not cs or cs - {s}:
                load[s][1] += duration
                ddr_work[s] += duration
                write_time[s, tid] = duration
                copy_bytes += size
    finish, critical = {}, [0] * len(groups)
    for v in graph.order:
        s = owner[v]
        start = max((finish[u] for u in graph.preds[v] if owner[u] == s), default=0)
```

```
            start = max(start, max((read_time.get((s, t), 0)
                                    for t in graph.inputs[v]), default=0))
            finish[v] = start + max(1, graph.ops[v]['cycles'])
            end = finish[v] + max((write_time.get((s, t), 0)
                                   for t in graph.outputs[v]), default=0)
            critical[s] = max(critical[s], end)
        duration = [max(critical[s], max(load[s]), ddr_work[s])
                    for s in range(len(groups))]
        return Partition(groups, name, owner, preds, succs, order, duration,
                         copy_bytes, sum(ddr_work))
```

根据仿真瓶颈生成重切候选:

```
def feedback(graph, partition, schedules, result):
    if not partition.groups:
        return
    task = max(range(len(partition.groups)),
               key=lambda s: result['task_end'][s] - result['task_start'][s])
    nodes = partition.groups[task]
    if len(nodes) > 1:
        components = graph.connected(nodes)
        if len(components) > 1:
            parts = pack_components(components, graph, 2, 1)
        else:
            allowed = set(nodes)
            parts = chunks([[v] for v in graph.order if v in allowed], graph, 2)
        if len(parts) == 2:
            yield 'split_bottleneck', canonical(
                [g for s, g in enumerate(partition.groups) if s != task] + list(parts))
    pairs = [(a, b) for order in schedules for a, b in zip(order, order[1:])]
    if pairs:
        a, b = min(pairs, key=lambda ab: sum(
            result['task_end'][s] - result['task_start'][s] for s in ab))
        yield 'merge_neighbors', canonical(
            [g for s, g in enumerate(partition.groups) if s not in (a, b)]
            + [partition.groups[a] + partition.groups[b]])
```

scheduling.py

按依赖和预计完成时间进行列表调度:

```
def list_schedule(partition, cores, settings, durations=None, priority='critical'):
    durations = durations if durations is not None else partition.duration
    rank = {}
    for s in reversed(partition.order):
        rank[s] = durations[s] + max((rank[w] for w in partition.succs[s]), default=0)
    degree = {s: len(p) for s, p in partition.preds.items()}
    ready = [(-(rank[s] if priority == 'critical' else durations[s]), s)
             for s, d in degree.items() if not d]
```

```
    heapq.heapify(ready)
    orders = [[] for _ in range(cores)]
    available, owner, finish = [0] * cores, {}, {}
    while ready:
        _, s = heapq.heappop(ready)
        choices = []
        for c in range(cores):
            local = available[c] + (settings.same_wait if orders[c] else 0)
            dependency = max((finish[r] + (settings.cross_wait if owner[r] != c else 0)
                              for r in partition.preds[s]), default=0)
            end = max(local, dependency) + durations[s]
            choices.append((end, available[c], c))
        end, _, c = min(choices)
        orders[c].append(s)
        available[c] = finish[s] = end
        owner[s] = c
        for w in sorted(partition.succs[s]):
            degree[w] -= 1
            if not degree[w]:
                heapq.heappush(ready, (-(rank[w] if priority == 'critical' else
durations[w]), w))
    return orders, max(max(finish.values(), default=0), partition.ddr_work)
```

execution.py

执行事件模拟，处理 Task 等待、Pipe 并行及共享 DDR 带宽:

```
def simulate(self, compiled, schedules):
        part, cfg = compiled.partition, self.settings
        check_schedules(part, schedules)
        n, m = len(schedules), len(part.groups)
        owner = [0] * m
        for c, order in enumerate(schedules):
            for s in order:
                owner[s] = c
        task_start, task_end = [-1] * m, [-1] * m
        remaining = compiled.task_sizes.copy()
        task_degree = [len(part.preds[s]) for s in range(m)]
        cross_release = [0] * m
        position, active = [0] * n, [-1] * n
        previous = [None] * n
        cursor = [[0] * 4 for _ in range(n)]
        running = [[-1] * 4 for _ in range(n)]
        degree = compiled.degree.copy()
        starts, ends = [-1] * len(degree), [-1] * len(degree)
        busy = [[0] * 4 for _ in range(n)]
        pool, last_update = {}, 0
```

```
        def settle(now):
            nonlocal last_update
            elapsed = now - last_update
            while elapsed > 1e-9:
                ids = [v for v, w in pool.items() if w > 1e-9]
                if not ids:
                    break
                minimum = min(pool[v] for v in ids)
                delta = minimum * len(ids)
                if delta >= elapsed - 1e-9:
                    share = elapsed / len(ids)
                    for v in ids:
                        pool[v] = max(0.0, pool[v] - share)
                    break
                for v in ids:
                    pool[v] = max(0.0, pool[v] - minimum)
                elapsed -= delta
            last_update = now
        def project(now):
            ordered = sorted((max(0.0, w), v) for v, w in pool.items())
            elapsed, previous_work, count, i = float(now), 0.0, len(ordered), 0
            while i < len(ordered):
                w = ordered[i][0]
                elapsed += (w - previous_work) * count
                j = i
                while j < len(ordered) and abs(ordered[j][0] - w) <= 1e-9:
                    ends[ordered[j][1]] = int(math.ceil(elapsed - 1e-9))
                    j += 1
                count -= j - i
                previous_work, i = w, j
        def release(c, s):
            if task_degree[s]:
                return None
            local = 0 if previous[c] is None else previous[c] + cfg.same_wait
            return max(local, cross_release[s])
        now, completed = 0, 0
        while completed < m:
            settle(now)
            retired_ddr = False
            for c in range(n):
                for p in range(4):
                    v = running[c][p]
                    if v < 0 or ends[v] > now:
                        continue
                    running[c][p] = -1
                    cursor[c][p] += 1
```

```
                    busy[c][p] += ends[v] - starts[v]
                    remaining[compiled.task_of[v]] -= 1
                    if v in pool:
                        del pool[v]
                        retired_ddr = True
                    for w in compiled.successors[v]:
                        degree[w] -= 1
            if retired_ddr:
                project(now)
            for c in range(n):
                s = active[c]
                if s >= 0 and remaining[s] == 0:
                    task_end[s] = now
                    previous[c] = now
                    active[c] = -1
                    position[c] += 1
                    completed += 1
                    for w in part.succs[s]:
                        task_degree[w] -= 1
                        if owner[w] != c:
                            cross_release[w] = max(cross_release[w], now +
cfg.cross_wait)
            for c in range(n):
                if active[c] >= 0 or position[c] == len(schedules[c]):
                    continue
                s = schedules[c][position[c]]
                ready_at = release(c, s)
                if ready_at is not None and ready_at <= now:
                    active[c] = s
                    task_start[s] = now
                    cursor[c] = [0] * 4
            for c in range(n):
                s = active[c]
                if s < 0:
                    continue
                for p, order in enumerate(compiled.pipes[s]):
                    if running[c][p] >= 0 or cursor[c][p] == len(order):
                        continue
                    v = order[cursor[c][p]]
                    if degree[v]:
                        continue
                    running[c][p] = v
                    starts[v], ends[v] = now, now + compiled.work[v]
                    if compiled.ddr[v]:
                        settle(now)
                        pool[v] = float(compiled.work[v])
```

```
                        project(now)
            if completed == m:
                break
            next_times = [ends[v] for row in running for v in row if v >= 0]
            for c in range(n):
                if active[c] < 0 and position[c] < len(schedules[c]):
                    ready_at = release(c, schedules[c][position[c]])
                    if ready_at is not None and ready_at > now:
                        next_times.append(ready_at)
            if not next_times or min(next_times) <= now:
                raise RuntimeError('执行图未能推进；检查依赖、Pipe 顺序与任务顺
序')
            now = min(next_times)
        return dict(makespan=now, task_start=task_start, task_end=task_end,
                    op_start=starts, op_end=ends, owner=owner, pipe_busy=busy,
                    data_movement_bytes=compiled.movement,
                    ddr_full_bandwidth_work=sum(w for w, d in zip(compiled.work,
compiled.ddr) if d),
                    memory_peak_by_core={c: {
                        pos: max((compiled.peaks[s][pos] for s in schedules[c]), default=0)
                        for pos in cfg.capacity} for c in range(n)})

slove.py
精评并更新最优解：
def consider(compiled, orders, label, required=False):
            nonlocal best, duplicate_simulations
            key = compiled.partition.groups, tuple(map(tuple, orders))
            if key in evaluated and not required:
                duplicate_simulations += 1
                return None
            evaluated.add(key)
            begin = time.perf_counter()
            result = self.engine.simulate(compiled, orders)
            solution = Solution(compiled, [list(o) for o in orders], result, label)
            improved = best is None or solution.score < best.score
            log.append(dict(candidate=label, tasks=len(compiled.partition.groups),
                            makespan=solution.score[0],
added_copy_bytes=solution.score[1],
                            simulation_seconds=time.perf_counter() - begin,
improved=improved))
            if improved:
                best = solution
            return solution

下界剪枝与候选评估：
def try_partition(part):
```

```
            if time.perf_counter() >= deadline:
                return
            if lower_bound(part, cores) > best.score[0]:
                rejected.append(dict(candidate=part.name, reason='时间下界已经超过当
前最好结果'))
                return
            try:
                compiled = self.engine.prepare(part)
                prepared.add(part.groups)
                scheduled = set()
                for priority in ('critical', 'longest'):
                    orders, _ = list_schedule(part, cores, self.settings,
                                              compiled.durations, priority)
                    key = tuple(map(tuple, orders))
                    if key not in scheduled:
                        scheduled.add(key)
                        consider(compiled, orders, part.name + '/' + priority)
            except (ValueError, RuntimeError) as error:
                rejected.append(dict(candidate=part.name, reason=str(error)))
```

瓶颈反馈和调度邻域搜索：

```
def refine_best():
    for name, groups in feedback(self.graph, best.compiled.partition,
                                 best.schedules, best.result):
        if groups in prepared or time.perf_counter() >= deadline:
            continue
        try:
            try_partition(describe(self.graph, groups, self.settings, name))
        except ValueError as error:
            rejected.append(dict(candidate=name, reason=str(error)))

refine_best()
for orders in neighbors(best.compiled.partition, best.schedules, best.result, moves):
    if time.perf_counter() >= deadline:
        break
    consider(best.compiled, orders, 'schedule_move')
```

## E2 问题二

partition.py
估计计算、通信成本并给出候选下界:

```
def estimate(self, plan):
    graph, cfg = self.graph, self.settings
    owner, n = plan.assignment(), len(plan.blocks)
    load, traffic, ddr = [[0] * 4 for _ in range(n)], 0, 0
    for v, c in owner.items():
        load[c][self.pipe[v]] += max(1, graph.ops[v]['cycles'])
    for ps, cs, size, work, final in self.records.values():
        sources, targets = {owner[v] for v in ps}, {owner[v] for v in cs}
        reads, writes = [], []
        if not ps:
            reads.extend(targets)
        if ps and final:
            writes.extend(sources)
        for p in sources:
            for c in targets:
                if p != c:
                    writes.append(p)
                    reads.append(c)
        for c in reads:
            load[c][0] += work
        for c in writes:
            load[c][1] += work
        traffic += size * (len(reads) + len(writes))
        ddr += work * (len(reads) + len(writes))
    for v, w, size in self.direct:
        if owner[v] != owner[w]:
            work = max(1, math.ceil(size / cfg.bandwidth))
            load[owner[v]][1] += work
            load[owner[w]][0] += work
            ddr += 2 * work
            traffic += 2 * size
    resource_bound = max(ddr, max(map(max, load), default=0))
    finish = {}
    for v in graph.order:
        start = max((finish[p] + (cfg.copy_delay + 2 * self.transfer[v].get(p, 1)
                                  if owner[p] != owner[v] else 0)
                     for p in graph.preds[v]), default=0)
        finish[v] = start + max(1, graph.ops[v]['cycles'])
    estimate = max(resource_bound, max(finish.values(), default=0))
    self.bounds[plan] = estimate if self.path_bound_valid else resource_bound
return estimate, traffic
```

scheduling.py

生成核内顺序候选：

```
def order_candidates(model, plan):
    graph, n, owner = model.graph, len(plan.blocks), plan.assignment()
    for name, order in (('critical_runs', model.critical), ('release_runs', model.release)):
        yield name, runs(graph, owner, n, order, width=32)
    # 对已有子图细分，保留原来的核心分配和子图间顺序。
    for parts in (2, 4):
        for name, order in (('split_critical', model.critical), ('split_release',
model.release)):
            position = {v: i for i, v in enumerate(order)}
            blocks = []
            for core in plan.blocks:
                pieces = []
                for group in core:
                    nodes = sorted(group, key=position.get)
                    pieces.extend(nodes[len(nodes) * i // parts:len(nodes) * (i + 1) // parts]
                                  for i in range(parts))
                blocks.append(pieces)
            yield name + ('_4' if parts == 4 else ''), Plan.build(blocks)
        if parts == 2:
            pairs = [(model.unit(a)[2] + model.unit(b)[2], c, i)
                     for c, core in enumerate(plan.blocks) for i, (a, b) in
enumerate(zip(core, core[1:]))]
            if pairs:
                _, c, i = min(pairs)
                blocks = [list(core) for core in plan.blocks]
                blocks[c][i:i + 2] = [blocks[c][i] + blocks[c][i + 1]]
                candidate = Plan.build(blocks)
                try:
                    check_plan(graph, candidate)
                    yield 'merge_neighbors', candidate
                except ValueError:
                    pass
            groups = [(c, group) for c, core in enumerate(plan.blocks) for group in core]
            yield 'subgraph_critical', arrange(graph, groups, n, {v: -graph.rank[v] for v in
graph.ops})
            merged = Plan.build([[[v for v in graph.order if owner[v] == c]] for c in
range(n)])
            try:
                check_plan(graph, merged)
                yield 'merge_per_core', merged
            except ValueError:
                pass  # 核心级有环不影响原多子图方案的合法性。
```

针对负载、搬运和同步瓶颈生成迁移候选：

```
def move_candidates(model, solution, limit=24):
```

```
    graph, plan = model.graph, solution.plan
    owner = plan.assignment()
    finish = {row['core_id']: max((op['end'] for op in row['ops']), default=0)
              for row in solution.result['per_core_timeline']}
    bottleneck = max(finish, key=finish.get)
    items, _, _ = block_graph(graph, plan)
    block_of = {v: s for s, (_, group) in enumerate(items) for v in group}
    units = []
    for c, group in items:
        if c == bottleneck:
            work = sum(max(1, graph.ops[v]['cycles']) for v in group)
            units.append((work, group))
    units.sort(key=lambda x: (-x[0], x[1]))
    choices = [group for _, group in units[:2]]
    boundary = []
    for chain in model.chains:
        if len({owner[v] for v in chain}) != 1 or len({block_of[v] for v in chain}) != 1:
            continue
        c = owner[chain[0]]
        tensors = set().union(*(graph.inputs[v] | graph.outputs[v] for v in chain))
        cut = sum(graph.tensors[t]['size'] for t in tensors
                  if any(owner[v] != c for v in graph.producers[t] | graph.consumers[t]))
        work = sum(max(1, graph.ops[v]['cycles']) for v in chain)
        boundary.append((cut, work if c == bottleneck else 0, tuple(chain)))
    boundary.sort(key=lambda x: (-x[0], -x[1], x[2]))
    choices.extend(group for _, _, group in boundary[:4])
    proposals = []
    for nodes in dict.fromkeys(tuple(sorted(g)) for g in choices):
        source = owner[nodes[0]]
        for target in range(len(plan.blocks)):
            if target != source:
                proposals.append(('load', nodes, target))
    # 小张量也可能引入长同步等待；从关键路径和实际末尾算子寻找跨核边。
    end = {op['op_id']: op['end'] for row in solution.result['per_core_timeline']
           for op in row['ops'] if op['op_id'] in graph.ops}
    edges = [(v, w) for v in graph.order for w in sorted(graph.succs[v]) if owner[v] !=
owner[w]]
    critical = sorted(edges, key=lambda e: (-graph.rank[e[0]], e))[:4]
    late = sorted(edges, key=lambda e: (-end.get(e[1], 0), e))[:4]
    chains = {v: tuple(chain) for chain in model.chains for v in chain}
    for v, w in dict.fromkeys(critical + late):
        for node, target in ((v, owner[w]), (w, owner[v])):
            nodes = chains[node]
            if len({owner[u] for u in nodes}) == 1:
                proposals.append(('sync', nodes, target))
    proposals = list({(tuple(sorted(nodes)), target): (kind, nodes, target)
```

```
                        for kind, nodes, target in proposals}.values())
    if len(graph.ops) > 4096:
        loads = [[0] * 4 for _ in plan.blocks]
        for v, c in owner.items():
            loads[c][model.pipe[v]] += max(1, graph.ops[v]['cycles'])
        def hint(item):
            _, nodes, target = item
            source, work = owner[nodes[0]], model.unit(nodes)[0]
            peak = max(max(value + (work[p] if c == target else -work[p] if c == source
else 0)
                           for p, value in enumerate(row)) for c, row in enumerate(loads))
            members = set(nodes)
            tensors = set().union(*(graph.inputs[v] | graph.outputs[v] for v in nodes))
            cut = sum(model.records[t][3] * len({owner[v] for v in
                      (graph.producers[t] | graph.consumers[t]) - members} - {target}) for
t in tensors)
            return peak + 2 * cut, nodes[0], target
        proposals = [item for kind in ('load', 'sync') for item in
                     sorted((p for p in proposals if p[0] == kind), key=hint)[:max(4,
limit)]]
    seen, ranked = {plan}, []
    for kind, nodes, target in proposals:
        source = owner[nodes[0]]
        try:
            candidate = relocate(model, plan, nodes, target)
        except ValueError:
            continue
        if candidate in seen:
            continue
        seen.add(candidate)
        ranked.append((model.estimate(candidate),
f'{kind}_{source}_to_{target}_{nodes[0]}', candidate))
    ranked.sort(key=lambda item: (item[0], item[1]))
    # 分配一半名额给同步候选，避免单纯搬运量排序排除有用的小张量迁移。
    chosen = [item for item in ranked if item[1].startswith('sync_')][:limit // 2]
    chosen.extend(item for item in ranked if item not in chosen)
return [(name, plan) for _, name, plan in chosen[:limit]]

slove.py
检查、剪枝并精评候选
def evaluate(plan, name, stage):
            nonlocal best
            if plan is None or plan in attempted:
                return False
            attempted.add(plan)
            begin = time.perf_counter()
```

```
            called = False
            try:
                check_plan(self.graph, plan)
                if plan not in self.model.bounds:
                    self.model.estimate(plan)
                bound = self.model.bounds[plan]
                if bound > best.score[0]:
                    history.append(dict(stage=stage, candidate=name, status='pruned',
preview_call=False,
                                        lower_bound=bound, seconds=time.perf_counter() -
begin))
                    return False
                called = True
                result = preview(self.graph.raw, plan.as_dict(), self.settings)
                solution = Solution(plan, result, name)
                if stage in ('partition', 'refine'):
                    diverse, assignments = [], set()
                    for value in sorted(frontier + [solution], key=lambda s: s.score):
                        key = tuple(sorted(value.plan.assignment().items()))
                        if key not in assignments:
                            assignments.add(key)
                            diverse.append(value)
                    coarse = min((s for s in diverse if s.score[0] <= diverse[0].score[0] *
1.5),
                                 key=lambda s: (sum(map(len, s.plan.blocks)), s.score))
                    light = min((s for s in diverse if s.score[0] <= diverse[0].score[0] *
1.3),
                                key=lambda s: (s.score[1], s.score[0]))
                    frontier[:] = diverse[:2]
                    for value in (light, coarse):
                        if value not in frontier:
                            frontier.append(value)
                improved = solution.score < best.score
                history.append(dict(stage=stage, candidate=name, status='ok',
preview_call=True,
                                    makespan=solution.score[0],
added_bytes=solution.score[1],
                                    subgraphs=sum(map(len, plan.blocks)),
improved=improved,
                                    seconds=time.perf_counter() - begin))
                if improved:
                    best = solution
                return improved
            except EVALUATION_ERRORS as error:
                history.append(dict(stage=stage, candidate=name, status='rejected',
preview_call=called,
```

```
                                    error=f'{type(error).__name__}: {str(error)[:500]}',
                                    seconds=time.perf_counter() - begin))
                return False

搜索结构划分:
if cores > 1 and candidates and not near_optimal():
            pool, seen = [], set(attempted)
            for name, plan in self.model.candidates(cores):
                if plan is None or plan in seen:
                    continue
                seen.add(plan)
                pool.append((self.model.estimate(plan), name, plan))
            pool.sort(key=lambda item: (item[0], item[1]))
            # 先比较不同结构，避免代理误差使同类候选占满预算。
            chosen, families = [], set()
            for item in pool:
                family = item[1].split('_')[0]
                if family not in families:
                    chosen.append(item)
                    families.add(family)
            chosen.extend(item for item in pool if item not in chosen)
            # 完整独立分支不产生核间通信，在大图限时搜索中优先保留这个候
选。
            depth = max(self.graph.depth.values(), default=0)
            large = len(self.graph.ops) > 4096
            regular_deep = large and depth > 32 and len(self.graph.ops) >= depth *
len(self.graph.components) / 2
            primary = ('fork_reduce' if regular_deep and len(self.graph.components) >=
cores else
                       'components_1' if len(self.graph.components) >= cores else
                       'window_4' if depth * depth > 16 * len(self.graph.ops) else
                       'fork_reduce' if depth > 128 else 'levels_2')
            coarse_first = (primary == 'components_1' and
                            (depth > 32 or len(self.graph.ops) > 2 * depth *
len(self.graph.components)))
            chosen.sort(key=lambda item: (item[1] != primary,
                         not (coarse_first and item[1] == 'coarsen')))
            for _, name, plan in chosen[:candidates]:
                if near_optimal() or (history and time.perf_counter() - started >= budget *
0.5):
                    break
                evaluate(plan, name, 'partition')
            for round_id in range(2):
                if near_optimal() or time.perf_counter() - started >= budget * 0.6:
                    break
                seed = (min(frontier, key=lambda s: (sum(map(len, s.plan.blocks)),
```

```
s.score))
                        if round_id == 0 and not large else best)
                pool = [(self.model.estimate(plan), name, plan)
                        for name, plan in self.model.refinements(seed) if plan not in
attempted]
                pool.sort(key=lambda item: (item[0], item[1]))
                if large and pool:
                    close = sorted((item for item in pool[1:] if item[0][0] <= pool[0][0][0]
* 1.01),
                                   key=lambda item: (item[0][1], int(item[1].rsplit('_',
1)[1])))
                    pool = pool[:1] + close[:2] + [item for item in pool[1:] if item not in
close[:2]]
                for _, name, plan in pool[:3]:
                    if near_optimal() or time.perf_counter() - started >= budget * 0.65:
                        break
                    evaluate(plan, name, 'refine')

搜索顺序和迁移方案:
if cores > 1 and orders:
            count, rounds = 0, 0
            while count < orders and not near_optimal() and time.perf_counter() - started
< budget * 0.9:
                incumbent = best
                bases, seen = [best], {tuple(sorted(best.plan.assignment().items()))}
                for candidate in sorted(frontier, key=lambda s: s.score):
                    key = tuple(sorted(candidate.plan.assignment().items()))
                    if key not in seen and candidate.score[0] <= best.score[0] * 1.3:
                        seen.add(key)
                        bases.append(candidate)
                choices = [list(order_candidates(self.model, s.plan)) for s in bases]
                for base, row in zip(bases, choices):
                    depth = max(self.graph.depth.values(), default=0)
                    first = ('split_release' if base.result['cross_core_transfers'] else
                             'critical_runs' if depth > 32 else
                             'release_runs' if
base.result['data_movement_bytes']['spill_added_copy_bytes']
                             else 'split_release')
                    priority = ('merge_neighbors' if first == 'critical_runs'
                                and sum(map(len, base.plan.blocks)) > cores
                                and not base.result['cross_core_transfers'] else first)
                    row.sort(key=lambda item: item[0] != priority)
                options = [(f'{row[i][0]}@{base.origin}', row[i][1])
                           for i in range(max(map(len, choices))) for base, row in
zip(bases, choices)
                           if i < len(row)]
```

```
                    for name, plan in options:
                        if count >= orders or near_optimal() or time.perf_counter() - started >=
budget * 0.9:
                            break
                        if plan is not None and plan not in attempted:
                            evaluate(plan, name, 'order')
                            count += 1
                    rounds += 1
                    if best.score >= incumbent.score or rounds >= 2:
                        break
            ordered = best.score

            if cores > 1 and moves and not near_optimal() and time.perf_counter() - started <
budget:
                for name, plan in move_candidates(self.model, best, limit=moves):
                    if near_optimal() or time.perf_counter() - started >= budget:
                        break
                    evaluate(plan, name, 'move')
```

## E3 问题三

model.py
估计分核后的计算与必要通信成本:

```
def estimate(self, plan):
        owner = plan.assignment()
        return self._estimate(tuple(owner[v] for v in self.graph.order), len(plan.blocks))
    def _estimate(self, assignment, cores):
        # 此下界只依赖分核；子图分组/顺序的完整合法性仍逐候选检查。
        graph, cfg = self.graph, self.settings
        owner = dict(zip(graph.order, assignment))
        loads, writes, traffic = [[0] * 4 for _ in range(cores)], 0, 0
        for v, c in owner.items():
            loads[c][self.pipe[v]] += self.work[v]
        for ps, cs, size, ddr, fast, final in self.records.values():
            sources, targets = {owner[v] for v in ps}, {owner[v] for v in cs}
            readers = list(targets) if not ps else []
            writers = list(sources) if ps and final else []
            for src in sources:
                for dst in targets - {src}:
                    writers.append(src)
                    readers.append(dst)
            for c in readers:
                loads[c][0] += fast
            for c in writers:
                loads[c][1] += ddr
            writes += ddr * len(writers)
            traffic += size * (len(readers) + len(writers))
        for u, v, size in self.direct:
            if owner[u] != owner[v]:
                q = max(1, math.ceil(size / cfg.bandwidth))
                loads[owner[u]][1] += q
                loads[owner[v]][0] += min(q, max(1, math.ceil(size /
cfg.cache_bandwidth)))
                writes += q
                traffic += 2 * size
        bound = max(self.lower_bound(cores), writes, max(map(max, loads), default=0))
        if self.path_valid:
            finish = {}
            for v in graph.order:
                finish[v] = self.work[v] + max((finish[u] + (cfg.copy_delay +
self.transfer[v][u]
                    if owner[u] != owner[v] else 0) for u in graph.preds[v]), default=0)
            bound = max(bound, max(finish.values(), default=0))
        return bound, traffic
```

计算考虑同核 Pipe 顺序的安全下界:

```
def ordered_bound(self, plan):
        if not self.path_valid or sum(map(len, plan.blocks)) <= len(plan.blocks):
            return 0
        g, owner = self.graph, plan.assignment()
        early, finish = {}, {}
        def wait(u, v):
            return self.settings.copy_delay + self.transfer[v][u] if owner[u] != owner[v]
else 0
        for v in g.order:
            ready = max((self.records[t][4] for t in g.inputs[v] if not g.producers[t]),
default=0)
            early[v] = max(ready, max((finish[u] + wait(u, v) for u in g.preds[v]),
default=0))
            finish[v] = early[v] + self.work[v]
        successors = [[] for _ in g.order]
        degree, value = [0] * len(g.order), [finish[v] for v in g.order]
        def add(u, v, weight):
            successors[u].append((v, weight))
            degree[v] += 1
        for v in g.order:
            for u in g.preds[v]:
                add(self.position[u], self.position[v], self.work[v] + wait(u, v))
        for core in plan.blocks:
            previous = {}
            for group in core:
                pipes = defaultdict(list)
                for v in group:
                    pipes[self.pipe[v]].append(v)
                for pipe, nodes in pipes.items():
                    start, end = len(successors), len(successors) + 1
                    successors.extend(([], []))
                    degree.extend((0, 0))
                    value.extend((min(early[v] for v in nodes), 0))
                    add(start, end, sum(self.work[v] for v in nodes))
                    for v in nodes:
                        add(start, self.position[v], self.work[v])
                        add(self.position[v], end, 0)
                    if pipe in previous:
                        add(previous[pipe], start, 0)
                    previous[pipe] = end
        ready = deque(i for i, d in enumerate(degree) if not d)
        visited = 0
        while ready:
            u = ready.popleft()
            visited += 1
```

```
            for v, weight in successors[u]:
                value[v] = max(value[v], value[u] + weight)
                degree[v] -= 1
                if not degree[v]:
                    ready.append(v)
        if visited != len(value):
            raise ValueError('子图顺序与逐 Pipe 数据依赖形成环')
        return max(value, default=0)

search.py
根据真实执行时间线细分子图，并针对 Cache 热点调整访问相位:
def timeline_neighbors(model, solution):
    g, plan = model.graph, solution.plan
    owner, cores = plan.assignment(), len(plan.blocks)
    times = {op['op_id']: (op['start'], op['end'])
             for row in solution.result['per_core_timeline'] for op in row['ops']
             if op['op_id'] in g.ops}
    if not times:
        return
    fine = None
    for key in (0, 1):
        order = topological(g.preds, g.succs, {v: (times[v][key], v) for v in g.ops})
        for width in (4, 16, 1, 64):
            candidate = runs(g, owner, cores, order, width)
            if key == 0 and width == 4:
                fine = candidate
            yield f'timeline_{key}_{width}', candidate
    if fine is None:
        return
    consumers = defaultdict(set)
    for t, tensor in g.tensors.items():
        consumers[tensor.get('logical_tid', t)].update(g.consumers[t])
    requested, pressure, touched = set(), Counter(), defaultdict(set)
    for event in solution.result['cache_events']:
        if event['event'] != 'miss' or not 0 < event['size_bytes'] <=
model.settings.cache_capacity:
            continue
        t = event['tensor_id']
        touched[t].add(event['core_id'])
        if t in requested:
            pressure[t] += event['size_bytes']
        requested.add(t)
    items, preds, succs = block_graph(g, fine)
    locations = [(c, i) for c, core in enumerate(fine.blocks) for i in range(len(core))]
    indices = {location: s for s, location in enumerate(locations)}
    for t, _ in pressure.most_common(4):
```

```
          for c in sorted(touched[t]):
               index = next((i for i, group in enumerate(fine.blocks[c])
                             if consumers[t].intersection(group)), None)
               if index is None:
                    continue
               s = indices[c, index]
               low = max((locations[p][1] + 1 for p in preds[s] if items[p][0] == c),
default=0)
               high = min((locations[w][1] - 1 for w in succs[s] if items[w][0] == c),
                          default=len(fine.blocks[c]) - 1)
               for offset in (-1, 1, -2, 2, -4, 4):
                    target = index + offset
                    if not low <= target <= high:
                         continue
                    blocks = [list(core) for core in fine.blocks]
                    blocks[c].insert(target, blocks[c].pop(index))
                    yield f'phase_{t}_{c}_{offset}', Plan.build(blocks)

slove.py
剪枝、精评及维护优胜方案:
def consider(plan, name, stage, bound):
     nonlocal best, coarse_seed
     attempted.add(plan)
     before = time.perf_counter()
     entry = dict(stage=stage, candidate=name, lower_bound=bound, called=False)
     try:
          if bound <= best.score[0]:
               bound = max(bound, self.model.ordered_bound(plan))
               entry['lower_bound'] = bound
          if bound > best.score[0]:
               entry['status'] = 'pruned'
               return
          entry['called'] = True
          result = self.evaluate(plan.as_dict())
          value = Solution(plan, result, name)
          if name == 'levels_2':
               coarse_seed = value
          if bound > value.score[0]:
               raise AssertionError(f'候选下界错误: {bound} > {value.score[0]}')
          entry.update(status='evaluated', makespan=value.score[0],
added_bytes=value.score[1])
          if value.score < best.score:
               best = value
               entry['improved'] = True
          pool = sorted(frontier + [value], key=lambda s: s.score)
          selected = pool[:3] if cores == 1 else []
```

```
        assignments = {tuple(sorted(s.plan.assignment().items())) for s in selected}
        for s in pool:
            assignment = tuple(sorted(s.plan.assignment().items()))
            if assignment not in assignments and len(selected) < 3:
                selected.append(s)
                assignments.add(assignment)
        eligible = [s for s in pool if s.score[0] <= best.score[0] * 1.5]
        if eligible:
            coarse = min(eligible, key=lambda s: (sum(map(len, s.plan.blocks)), s.score))
            light = min(eligible, key=lambda s: (s.score[1], s.score[0]))
            for s in (coarse, light):
                if s not in selected:
                    selected.append(s)
        frontier[:] = selected
        except EVALUATION_ERRORS as error:
            entry.update(status='rejected', reason=str(error)[:180])
        finally:
            entry['seconds'] = time.perf_counter() - before
            history.append(entry)
```

对不同候选族排序和限额评估:

```
def run(proposals, stage, limit, fraction):
            deadline = budget * fraction
            if limit == 0 or optimal() or elapsed() >= deadline:
                return 0
            ranked, seen = [], set(attempted)
            for name, plan in proposals:
                if elapsed() >= deadline:
                    break
                if plan is None or plan in seen:
                    continue
                seen.add(plan)
                try:
                    check_plan(self.graph, plan)
                    score = self.model.estimate(plan)
                    ranked.append((score, name, plan))
                except EVALUATION_ERRORS as error:
                    history.append(dict(stage=stage, candidate=name, status='invalid',
                                        called=False, reason=str(error)[:180], seconds=0))
            def priority(item):
                (bound, traffic), name, plan = item
                fine = -sum(map(len, plan.blocks)) if name.split('_')[0] in ('components',
'dfs', 'topo') else 0
                return (bound + (traffic / (10 * self.settings.bandwidth)
                                 if stage in ('seed', 'stage') else 0), traffic, fine, name)
            ranked.sort(key=priority)
```

```
            first, rest, families = [], [], set()
            for item in ranked:
                family = item[1].split('_')[0]
                if family in ('runs', 'split'):
                    family = '_'.join(item[1].split('_')[:2])
                (rest if family in families else first).append(item)
                families.add(family)
            calls = 0
            ranked = first + rest
            if stage == 'seed' and cores > 1:
                depth = max(self.graph.depth.values(), default=0)
                primary = ('fork_reduce' if len(self.graph.ops) > 4096 and depth > 32 and
                           len(self.graph.ops) >= depth * len(self.graph.components) / 2
else
                           'components_1' if len(self.graph.components) >= cores else
'levels_2')
                ranked.sort(key=lambda item: (item[1] != primary,
                             not (primary == 'fork_reduce' and item[1] == 'forkrelease')))
            for (bound, _), name, plan in ranked:
                if calls >= limit or optimal() or elapsed() >= deadline:
                    break
                consider(plan, name, stage, bound)
                calls += int(history[-1]['called'])
            return calls
```

执行独立精修，并用有、无 L2 的固定基准条件阻止退化：

```
def refine(self, solution, budget=15.0, candidates=64):
        if not math.isfinite(budget) or budget < 0:
            raise ValueError('追加搜索时间须为有限非负数')
        if type(candidates) is not int or candidates < 0:
            raise ValueError('追加候选数须为非负整数')
        started = time.perf_counter()
        check_plan(self.graph, solution.plan)
        baseline = self.settings.without_cache(self.graph.raw, solution.plan.as_dict())
        best, best_without = solution, baseline
        seen, history, calls = {solution.plan}, [], 0
        prepared_seconds = time.perf_counter() - started
        started = time.perf_counter()
        for iteration in range(2):
            source = best
            for name, plan in timeline_neighbors(self.model, source):
                if calls >= candidates or time.perf_counter() - started >= budget:
                    break
                if plan is None or plan in seen:
                    continue
                seen.add(plan)
```

```
                    entry = dict(candidate=name, iteration=iteration, called=False)
                    try:
                        check_plan(self.graph, plan)
                        if self.model.estimate(plan)[0] > best.score[0]:
                            entry['status'] = 'pruned'
                            continue
                        calls += 1
                        entry['called'] = True
                        result = self.evaluate(plan.as_dict())
                        value = Solution(plan, result, name)
                        entry.update(makespan=value.score[0], added_bytes=value.score[1],
status='evaluated')
                        if value.score >= best.score or value.score[1] > solution.score[1]:
                            continue
                        without = self.settings.without_cache(self.graph.raw, plan.as_dict())
                        entry['no_l2_makespan'] = without['makespan']
                        # 始终与进入本阶段的固定基准比较，不能用不断抬高的比值阻
塞更快候选。
                        if (without['makespan'] > baseline['makespan'] or
                                without['makespan'] * solution.score[0] <
                                baseline['makespan'] * value.score[0]):
                            entry['status'] = 'guarded'
                            continue
                        best, best_without = value, without
                        entry['improved'] = True
                    except EVALUATION_ERRORS as error:
                        entry.update(status='rejected', reason=str(error)[:180])
                    finally:
                        history.append(entry)
                if best.plan == source.plan:
                    break
```

# 附录 F AI 工具使用说明

## F1 所用 AI 工具名称和版本

本文研究过程中使用的人工智能工具为 OpenAI ChatGPT（GPT-6），主要用于辅助方法检索、数据搜集、程序编写与调试，以及模型优化讨论。除上述辅助工作外，研究中的数据核验、模型构建、参数确定、程序运行、结果分析和最终结论均由研究团队独立完成。

## F2 AI 工具的使用目的和环节

在研究方法查找与方案设计环节，AI 工具主要用于根据研究问题梳理可选的数学模型、统计方法和算法思路，比较不同方法的适用条件、基本假设、优缺点及实现难度，并提供相关检索关键词和资料查找方向。AI 给出的内容仅作为方案讨论的参考，最终方法由团队结合题目要求、数据特征和已有文献重新判断。

在数据搜集与处理环节，AI 工具主要用于辅助制定数据搜集方案，包括推荐可能的数据来源、检索关键词、数据字段、时间范围和数据格式，并协助讨论缺失值、异常值、重复数据及量纲不一致等问题。实际使用的数据均由团队通过公开资料、题目附件或可靠数据库获取，并经过人工核验、清洗和记录，AI 生成的信息未未经核实直接作为数据依据。

在编程辅助与程序调试环节，AI 工具主要用于辅助编写 Python 程序框架、解释函数用法、检查语法错误、分析运行报错、完善数据处理流程和提出测试建议。AI 提供的代码均由团队结合实际数据进行修改、运行和验证，并通过结果复核、边界测试和重复计算确认程序的有效性。

在优化建模与结果讨论环节，AI 工具主要用于辅助分析决策变量、目标函数、约束条件和评价指标之间的关系，讨论线性规划、整数规划、启发式算法、敏感性分析和多情景比较等可能方案，并协助发现模型表达或程序实现中的潜在问题。最终的模型结构、参数设置、优化结果和结论均由团队独立推导、计算和解释。

## F3 关键交互记录

| 序号 | 使用环节 | 主要提示内容 | AI 回复及团队处理方式 |
|---|---|---|---|
| 1 | 方法查找与方案比较 | 根据研究目标和数据特征，列出可采用的建模方法，并比较其适用条件、优缺点和实现难度。 | AI 提供候选方法、检索关键词及比较思路。团队结合题目要求和参考资料重新筛选，并通过文献和计算实验确定最终方案。 |
| 2 | 数据搜集与处理 | 根据研究问题提出数据搜集方向，说明所需字段、数据来源、时间范围及可能的数据质量问题。 | AI 协助制定数据清单和处理流程。团队实际查找并下载数据，对来源、单位、完整性和合理性进行人工核验。 |
| 3 | 编程辅助与程序调试 | 检查程序中的语法错误、变量维度、循环逻辑和异常处理，并提出修改建议。 | AI 帮助定位潜在问题并提供示例代码。团队逐段运行程序，结合测试结果进行修改，并对关键输出进行重复验证。 |

| 序号 | 使用环节 | 主要提示内容 | AI 回复及团队处理方式 |
| --- | --- | --- | --- |
| 4 | 优化模型与结果讨论 | 分析目标函数和约束条件的设置是否合理，并讨论参数变化对优化结果的影响。 | AI 提供模型表达、敏感性分析和情景讨论的建议。团队根据数学推导、实际约束和计算结果重新检查模型，并独立确定参数和结论。 |

## F4 使用原则

AI 工具在本文中仅作为研究辅助工具，不代替研究团队进行事实判断、数据验证、模型推导和结论形成。对于 AI 生成的文献、数据来源、公式、代码和解释，团队均进行了必要的人工复核；未经核验的 AI 输出未直接用于论文中的数据、图表、公式或结论。